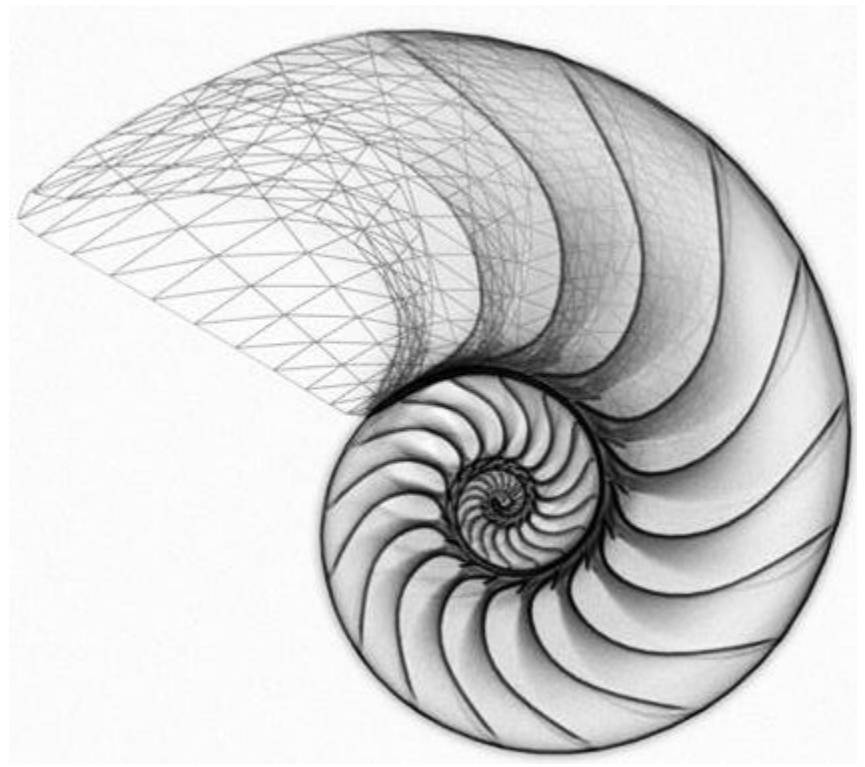


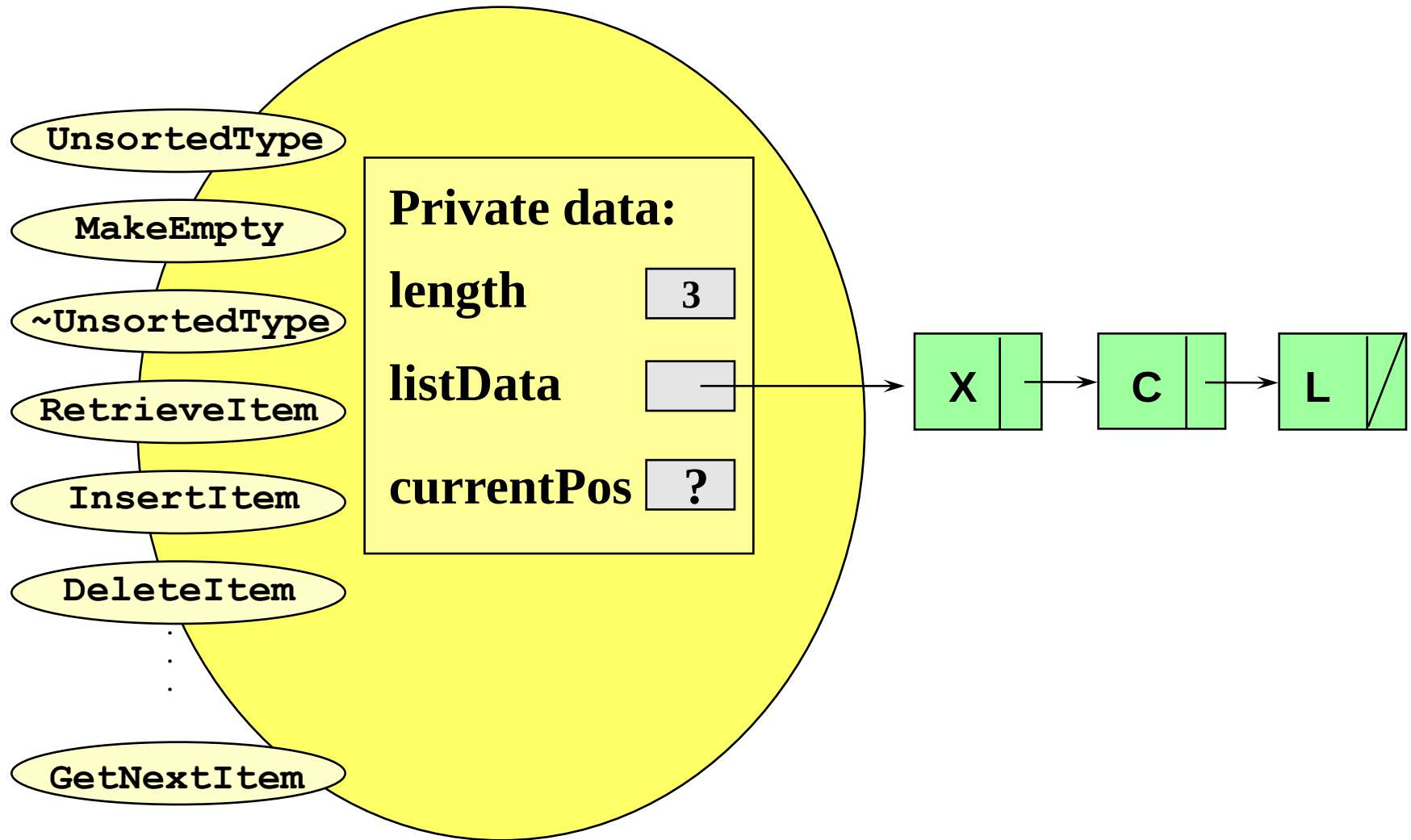


Lecture 09

Abstract Data Type Unsorted List and Sorted List (Linked-list-based Implementation)

CSE225: Data Structures and Algorithms

class UnsortedType<char>



unsortedlinkedlist.h

```
#ifndef UNSORTEDLINKEDLIST_H_INCLUDED
#define UNSORTEDLINKEDLIST_H_INCLUDED
```

```
template <class ItemType>
class UnsortedType
{
```

```
    struct NodeType
    {
        ItemType info;
        NodeType* next;
    };
```

```
public:
```

```
    UnsortedType();
    ~UnsortedType();
    bool IsFull();
    int LengthIs();
    void MakeEmpty();
    void RetrieveItem(ItemType& item, bool& found);
```

```
    void InsertItem(ItemType item);
    void DeleteItem(ItemType item);
    void ResetList();
    void GetNextItem(ItemType& item);
```

```
private:
```

```
    NodeType* listData;
    int length;
    NodeType* currentPos;
};
```

```
#endif // UNSORTEDLINKEDLIST_H_INCLUDED
```

unsortedlinkedlist.cpp

```
#include "unsortedlinkedlist.h"
#include<cstddef>
#include<new>

template <class ItemType>
UnsortedType<ItemType>::UnsortedType()
{
    length = 0;
    listData = NULL;
    currentPos = NULL;
}

template <class ItemType>
int UnsortedType<ItemType>::LengthIs()
{
    return length;
}
```

```
template<class ItemType>
bool UnsortedType<ItemType>::IsFull()
{
    NodeType* location;
    try
    {
        location = new NodeType;
        delete location;
        return false;
    }
    catch(std::bad_alloc& exception)
    {
        return true;
    }
}
```

unsortedlinkedlist.cpp

```
#include "unsortedlinkedlist.h"
#include<cstddef>
#include<new>

template <class ItemType>
UnsortedType<ItemType>::UnsortedType()
{
    length = 0;
    listData = NULL;
    currentPos = NULL;
}
```

O(1)

```
template <class ItemType>
int UnsortedType<ItemType>::LengthIs()
{
    return length;
}
```

O(1)

```
template<class ItemType>
bool UnsortedType<ItemType>::IsFull()
{
    NodeType* location;
    try
    {
        location = new NodeType;
        delete location;
        return false;
    }
    catch(std::bad_alloc& exception)
    {
        return true;
    }
}
```

O(1)

unsortedlinkedlist.cpp

```
template <class ItemType>
void UnsortedType<ItemType>::InsertItem(ItemType item)
{
    NodeType* location;
    location = new NodeType;
    location->info = item;
    location->next = listData;
    listData = location;
    length++;
}
```

unsortedlinkedlist.cpp

```
template <class ItemType>
void UnsortedType<ItemType>::InsertItem(ItemType item)
{
    NodeType* location;
    location = new NodeType;
    location->info = item;
    location->next = listData;
    listData = location;
    length++;
}
```

length 0 listData

InsertItem('X')

unsortedlinkedlist.cpp

```
template <class ItemType>
void UnsortedType<ItemType>::InsertItem(ItemType item)
{
    NodeType* location;
    location = new NodeType;
    location->info = item;
    location->next = listData;
    listData = location;
    length++;
}
```



InsertItem('X')

unsortedlinkedlist.cpp

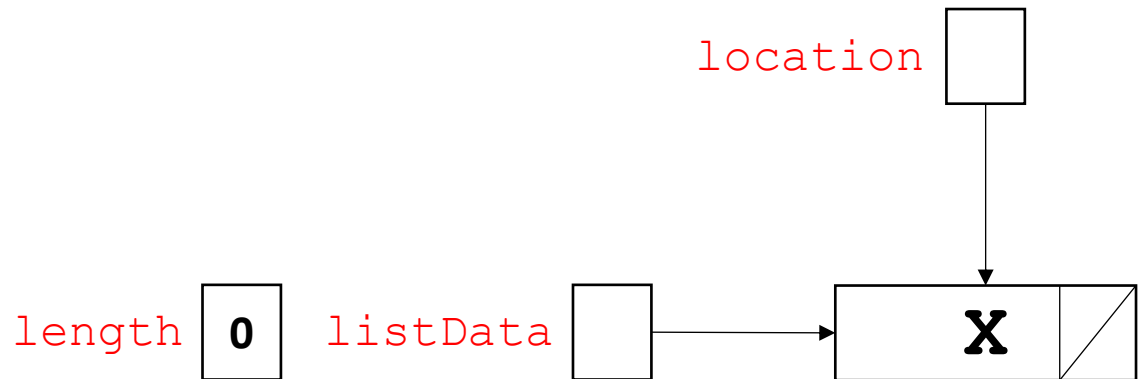
```
template <class ItemType>
void UnsortedType<ItemType>::InsertItem(ItemType item)
{
    NodeType* location;
    location = new NodeType;
    location->info = item;
    location->next = listData;
    listData = location;
    length++;
}
```



InsertItem('X')

unsortedlinkedlist.cpp

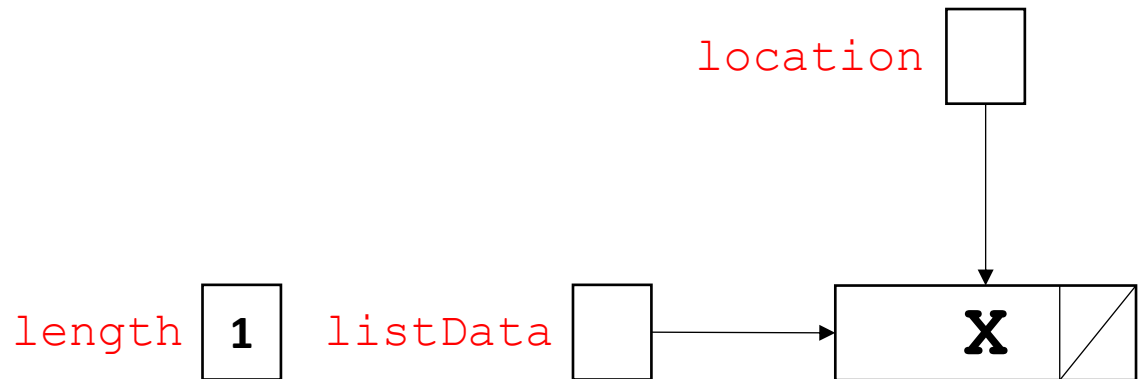
```
template <class ItemType>
void UnsortedType<ItemType>::InsertItem(ItemType item)
{
    NodeType* location;
    location = new NodeType;
    location->info = item;
    location->next = listData;
    listData = location;
    length++;
}
```



InsertItem('X')

unsortedlinkedlist.cpp

```
template <class ItemType>
void UnsortedType<ItemType>::InsertItem(ItemType item)
{
    NodeType* location;
    location = new NodeType;
    location->info = item;
    location->next = listData;
    listData = location;
    length++;
}
```



InsertItem('X')

unsortedlinkedlist.cpp

```
template <class ItemType>
void UnsortedType<ItemType>::InsertItem(ItemType item)
{
    NodeType* location;
    location = new NodeType;
    location->info = item;
    location->next = listData;
    listData = location;
    length++;
}
```



unsortedlinkedlist.cpp

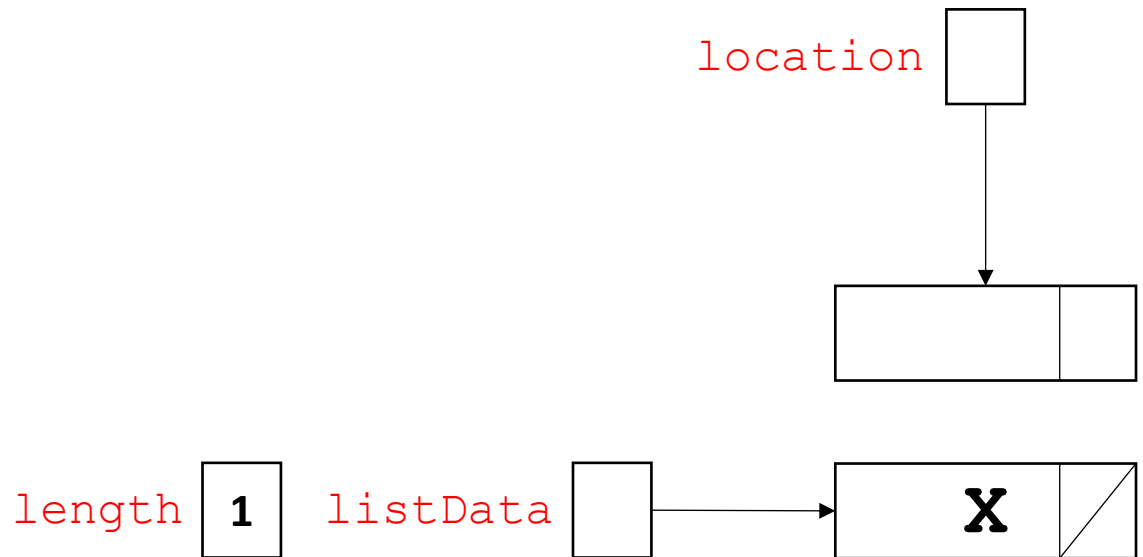
```
template <class ItemType>
void UnsortedType<ItemType>::InsertItem(ItemType item)
{
    NodeType* location;
    location = new NodeType;
    location->info = item;
    location->next = listData;
    listData = location;
    length++;
}
```



InsertItem('C')

unsortedlinkedlist.cpp

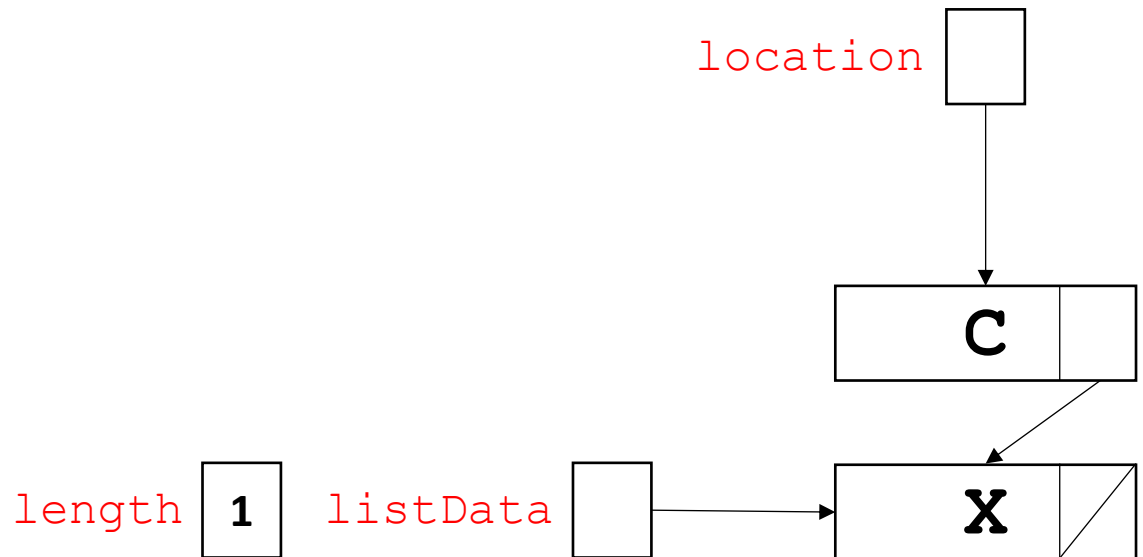
```
template <class ItemType>
void UnsortedType<ItemType>::InsertItem(ItemType item)
{
    NodeType* location;
    location = new NodeType;
    location->info = item;
    location->next = listData;
    listData = location;
    length++;
}
```



InsertItem('C')

unsortedlinkedlist.cpp

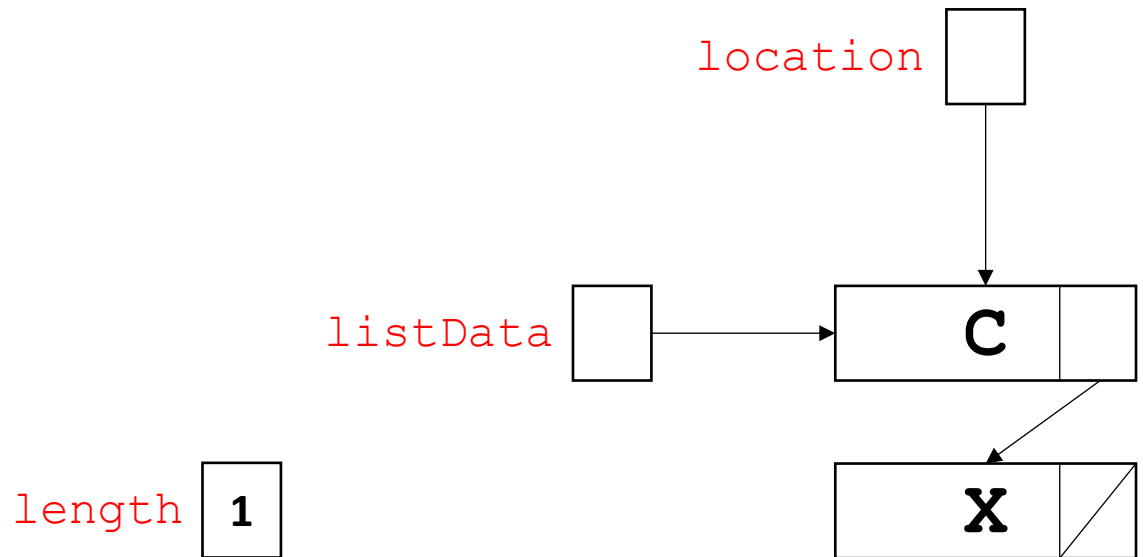
```
template <class ItemType>
void UnsortedType<ItemType>::InsertItem(ItemType item)
{
    NodeType* location;
    location = new NodeType;
    location->info = item;
    location->next = listData;
    listData = location;
    length++;
}
```



InsertItem('C')

unsortedlinkedlist.cpp

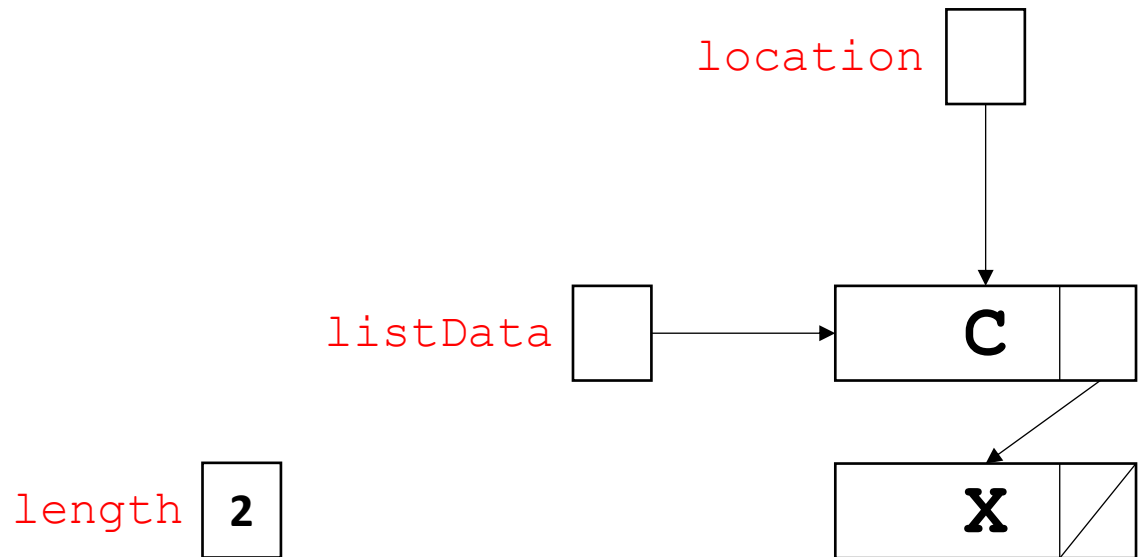
```
template <class ItemType>
void UnsortedType<ItemType>::InsertItem(ItemType item)
{
    NodeType* location;
    location = new NodeType;
    location->info = item;
    location->next = listData;
    listData = location;
    length++;
}
```



InsertItem('C')

unsortedlinkedlist.cpp

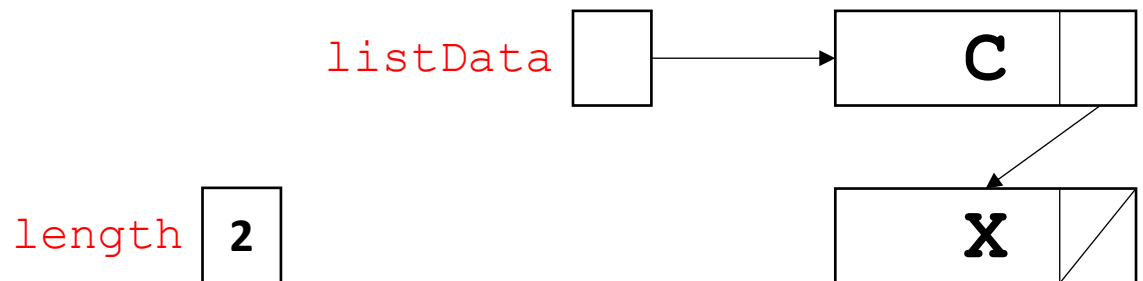
```
template <class ItemType>
void UnsortedType<ItemType>::InsertItem(ItemType item)
{
    NodeType* location;
    location = new NodeType;
    location->info = item;
    location->next = listData;
    listData = location;
    length++;
}
```



InsertItem('C')

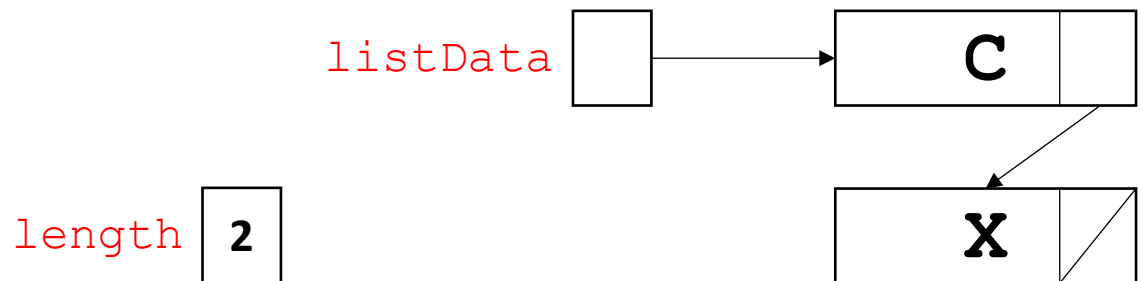
unsortedlinkedlist.cpp

```
template <class ItemType>
void UnsortedType<ItemType>::InsertItem(ItemType item)
{
    NodeType* location;
    location = new NodeType;
    location->info = item;
    location->next = listData;
    listData = location;
    length++;
}
```



unsortedlinkedlist.cpp

```
template <class ItemType>
void UnsortedType<ItemType>::InsertItem(ItemType item)
{
    NodeType* location;
    location = new NodeType;
    location->info = item;
    location->next = listData;
    listData = location;
    length++;
}
```



InsertItem('L')

unsortedlinkedlist.cpp

```
template <class ItemType>
void UnsortedType<ItemType>::InsertItem(ItemType item)
{
    NodeType* location;
    location = new NodeType;
    location->info = item;
    location->next = listData;
    listData = location;
    length++;
}
```

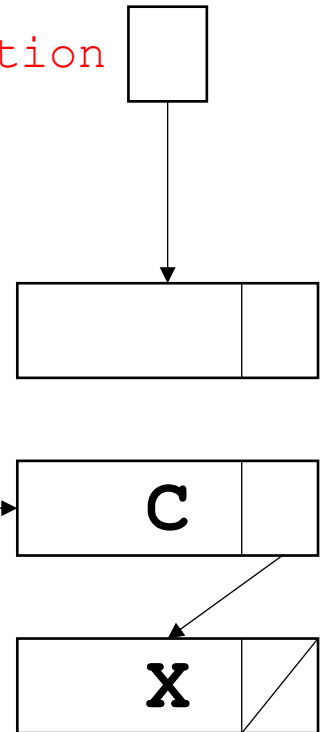
InsertItem('L')

length

2

listData

location



unsortedlinkedlist.cpp

```
template <class ItemType>
void UnsortedType<ItemType>::InsertItem(ItemType item)
{
    NodeType* location;
    location = new NodeType;
    location->info = item;
    location->next = listData;
    listData = location;
    length++;
}
```

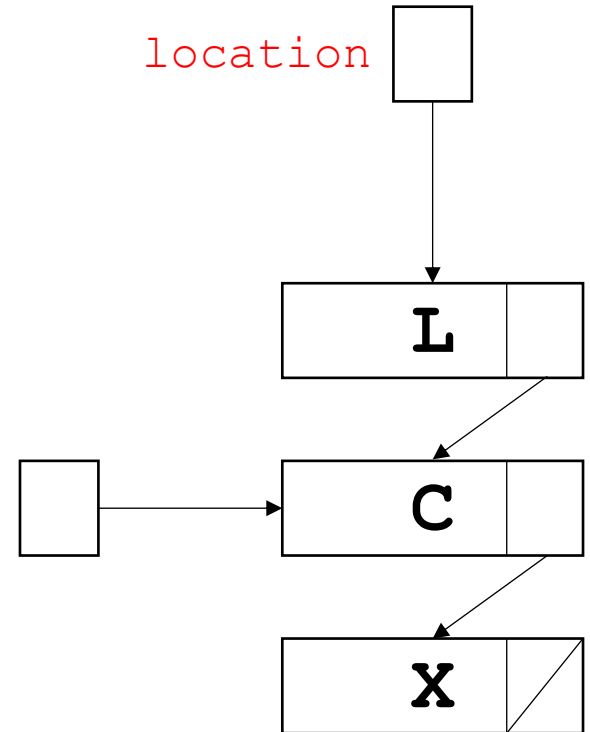
InsertItem('L')

length

2

listData

location



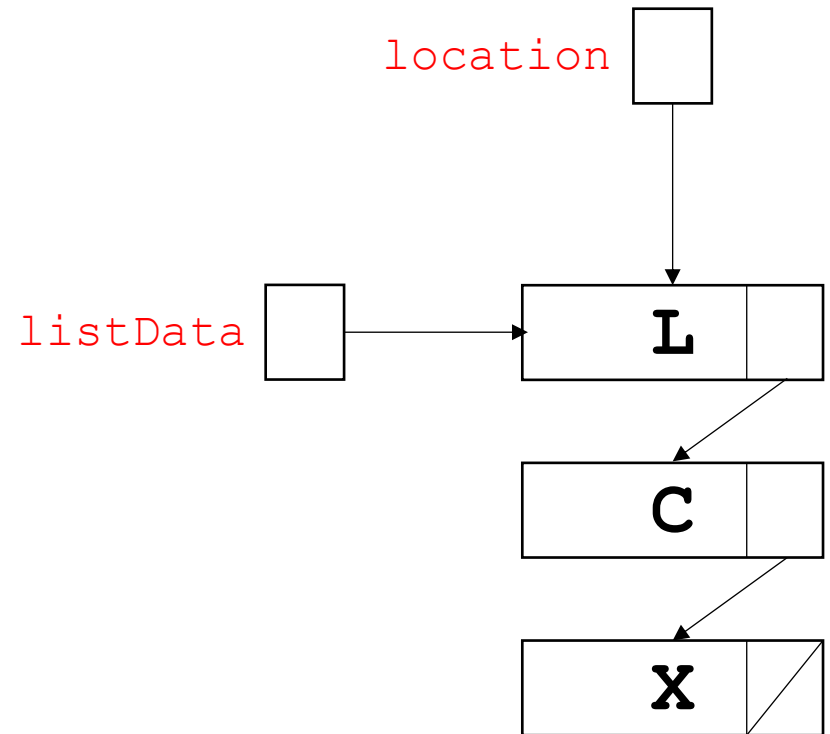
unsortedlinkedlist.cpp

```
template <class ItemType>
void UnsortedType<ItemType>::InsertItem(ItemType item)
{
    NodeType* location;
    location = new NodeType;
    location->info = item;
    location->next = listData;
    listData = location;
    length++;
}
```

length

2

InsertItem('L')



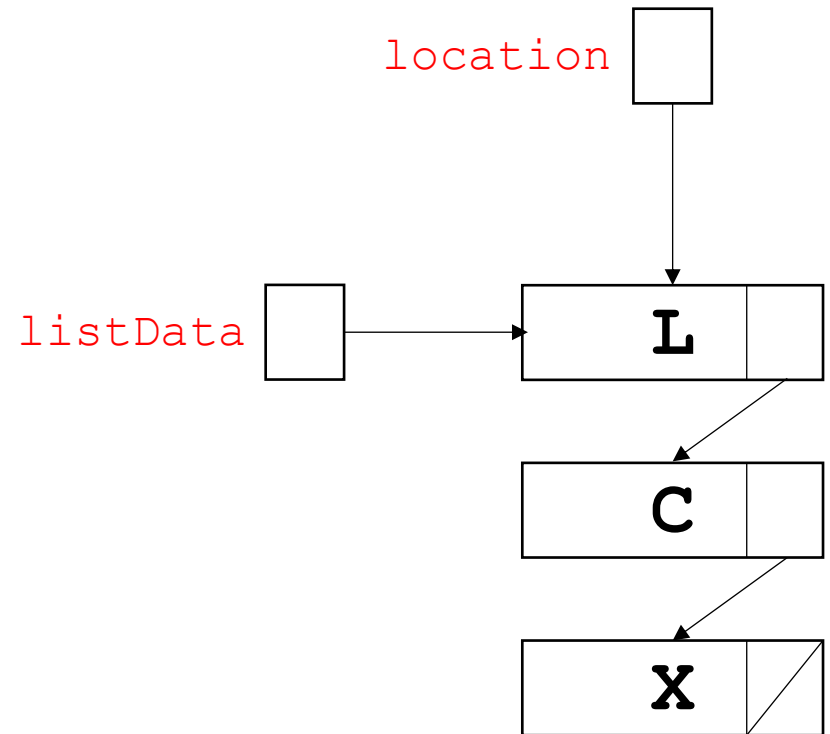
unsortedlinkedlist.cpp

```
template <class ItemType>
void UnsortedType<ItemType>::InsertItem(ItemType item)
{
    NodeType* location;
    location = new NodeType;
    location->info = item;
    location->next = listData;
    listData = location;
    length++;
}
```

length

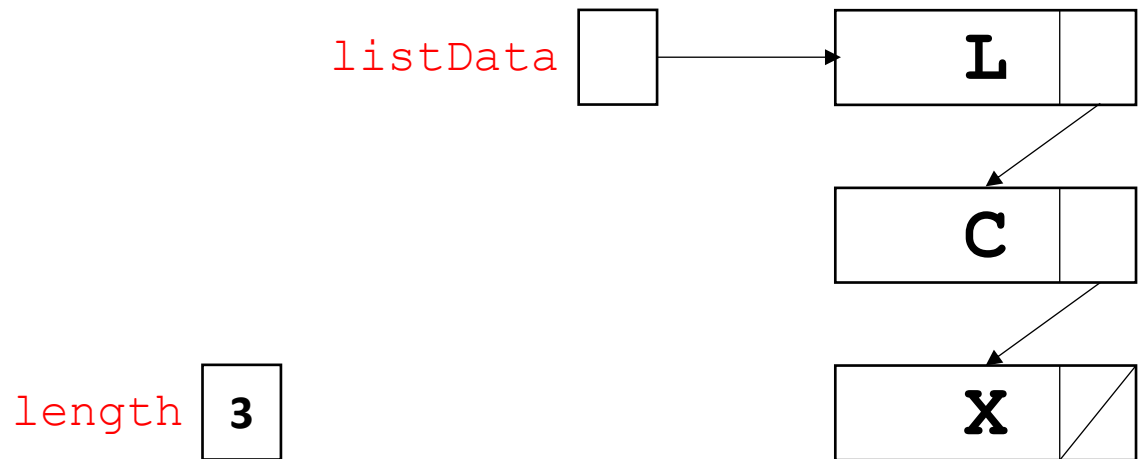
3

InsertItem('L')



unsortedlinkedlist.cpp

```
template <class ItemType>
void UnsortedType<ItemType>::InsertItem(ItemType item)
{
    NodeType* location;
    location = new NodeType;
    location->info = item;
    location->next = listData;
    listData = location;
    length++;
}
```



unsortedlinkedlist.cpp

```
template <class ItemType>
void UnsortedType<ItemType>::InsertItem(ItemType item)
{
    NodeType* location;
    location = new NodeType;
    location->info = item;
    location->next = listData;
    listData = location;
    length++;
}
```

O(1)

unsortedlinkedlist.cpp

```
template <class ItemType>
void UnsortedType<ItemType>::DeleteItem(ItemType item)
{
    NodeType* location = listData;
    NodeType* tempLocation;
    if (item == listData->info)
    {
        tempLocation = location;
        listData = listData->next; //Delete the first node
    }
    else
    {
        while (!(item==(location->next)->info))
        {
            location = location->next;
            tempLocation = location->next;
            location->next = (location->next)->next;
        }
        delete tempLocation;
        length--;
    }
}
```

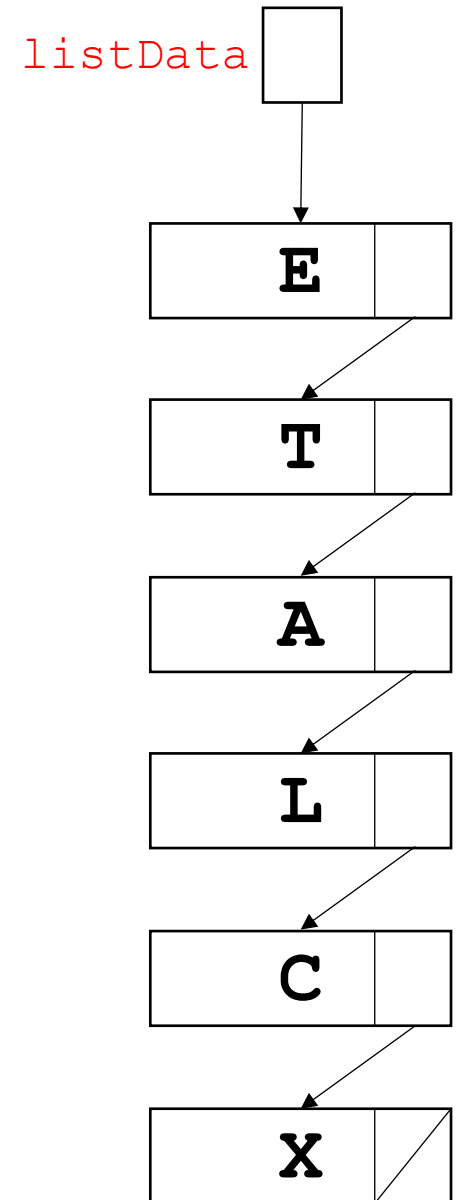
unsortedlinkedlist.cpp

```
template <class ItemType>
void UnsortedType<ItemType>::DeleteItem(ItemType item)
{
    NodeType* location = listData;
    NodeType* tempLocation;
    if (item == listData->info)
    {
        tempLocation = location;
        listData = listData->next;
    }
    else
    {
        while (!(item==(location->next)->info))
            location = location->next;
        tempLocation = location->next;
        location->next = (location->next)->next;
    }
    delete tempLocation;
    length--;
}
```

length

6

DeleteItem('L')



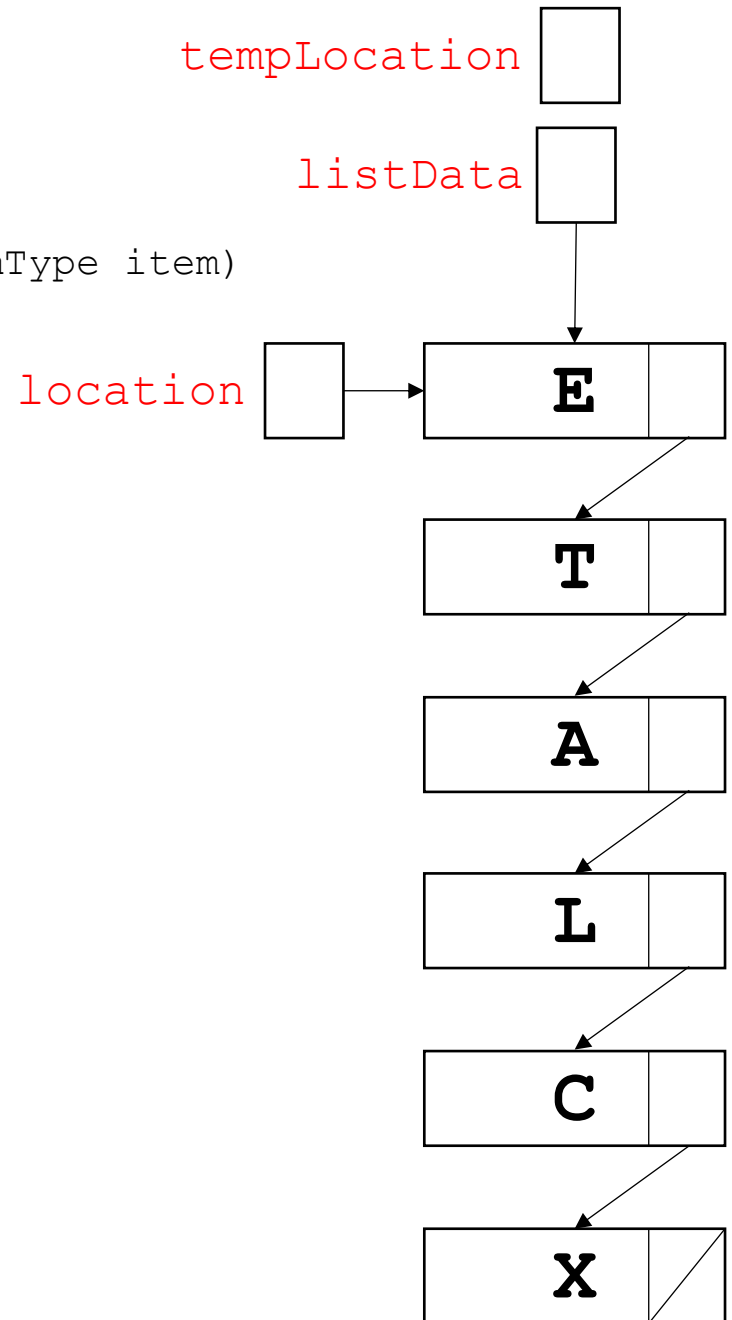
unsortedlinkedlist.cpp

```
template <class ItemType>
void UnsortedType<ItemType>::DeleteItem(ItemType item)
{
    NodeType* location = listData;
    NodeType* tempLocation;
    if (item == listData->info)
    {
        tempLocation = location;
        listData = listData->next;
    }
    else
    {
        while (!(item==(location->next)->info))
            location = location->next;
        tempLocation = location->next;
        location->next = (location->next)->next;
    }
    delete tempLocation;
    length--;
}
```

length

6

DeleteItem('L')



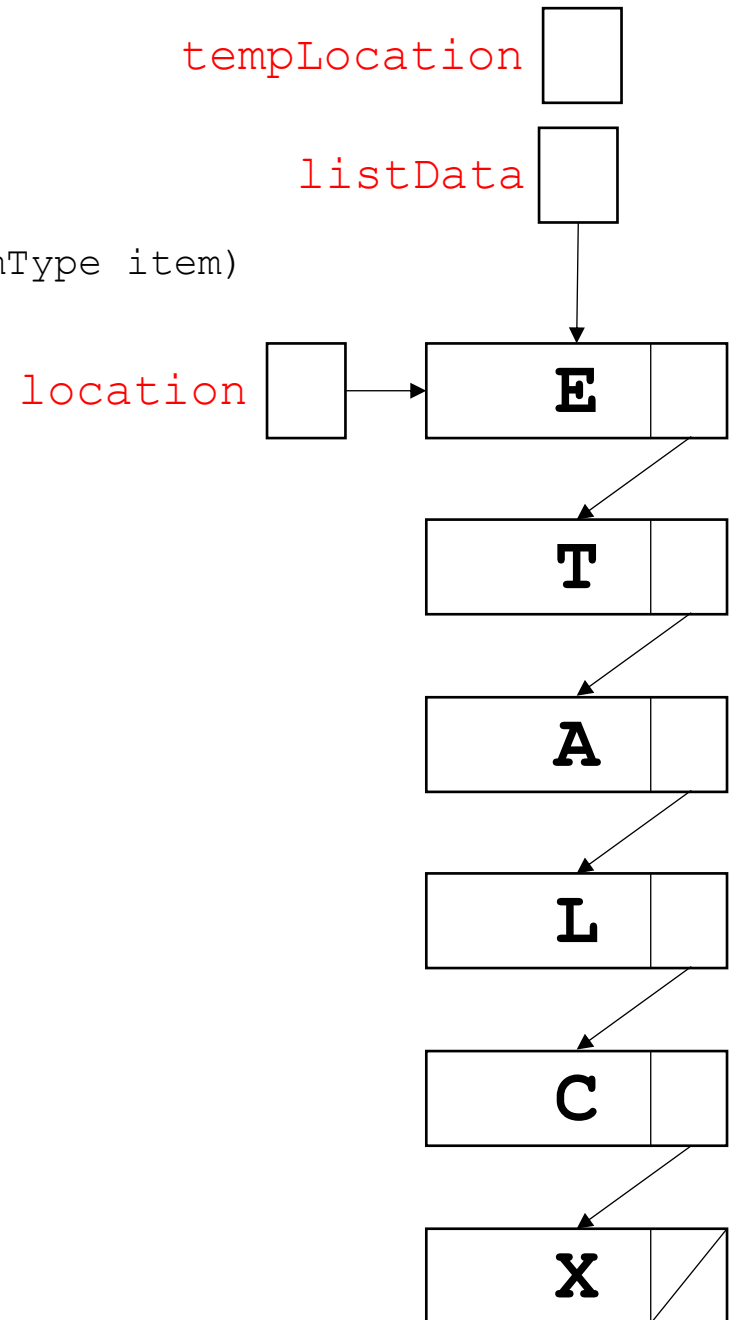
unsortedlinkedlist.cpp

```
template <class ItemType>
void UnsortedType<ItemType>::DeleteItem(ItemType item)
{
    NodeType* location = listData;
    NodeType* tempLocation;
    if (item == listData->info)
    {
        tempLocation = location;
        listData = listData->next;
    }
    else
    {
        while (!(item==(location->next)->info))
            location = location->next;
        tempLocation = location->next;
        location->next = (location->next)->next;
    }
    delete tempLocation;
    length--;
}
```

length

6

DeleteItem('L')



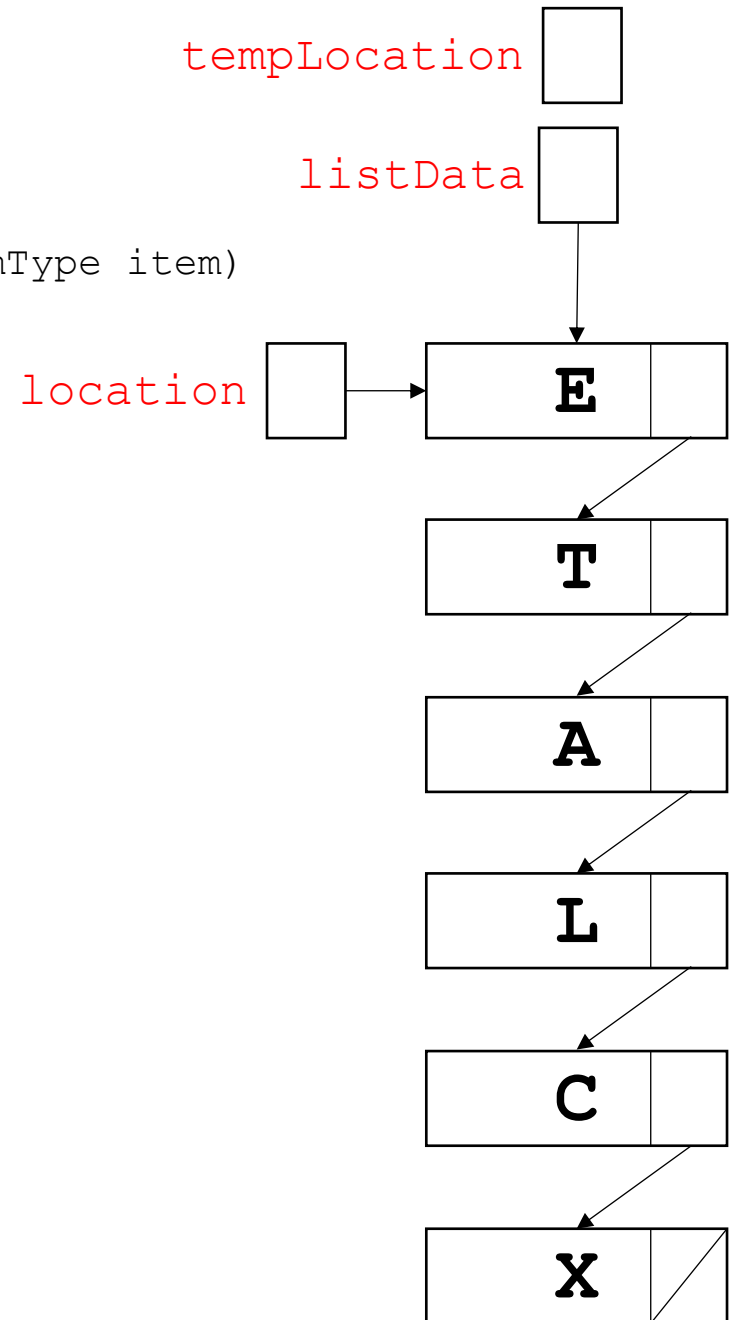
unsortedlinkedlist.cpp

```
template <class ItemType>
void UnsortedType<ItemType>::DeleteItem(ItemType item)
{
    NodeType* location = listData;
    NodeType* tempLocation;
    if (item == listData->info)
    {
        tempLocation = location;
        listData = listData->next;
    }
    else
    {
        while (!(item==(location->next)->info))
            location = location->next;
        tempLocation = location->next;
        location->next = (location->next)->next;
    }
    delete tempLocation;
    length--;
}
```

length

6

DeleteItem('L')



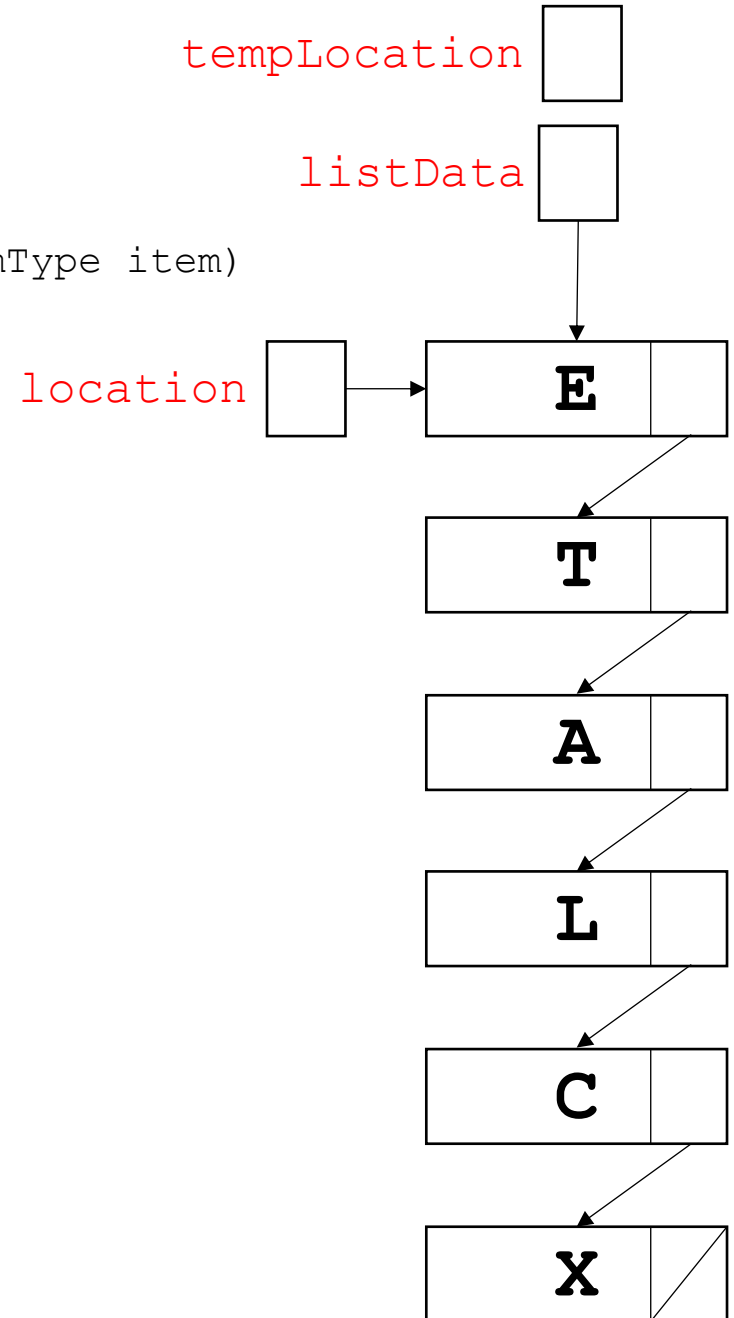
unsortedlinkedlist.cpp

```
template <class ItemType>
void UnsortedType<ItemType>::DeleteItem(ItemType item)
{
    NodeType* location = listData;
    NodeType* tempLocation;
    if (item == listData->info)
    {
        tempLocation = location;
        listData = listData->next;
    }
    else
    {
        while (!(item==(location->next)->info))
            location = location->next;
        tempLocation = location->next;
        location->next = (location->next)->next;
    }
    delete tempLocation;
    length--;
}
```

length

6

DeleteItem('L')



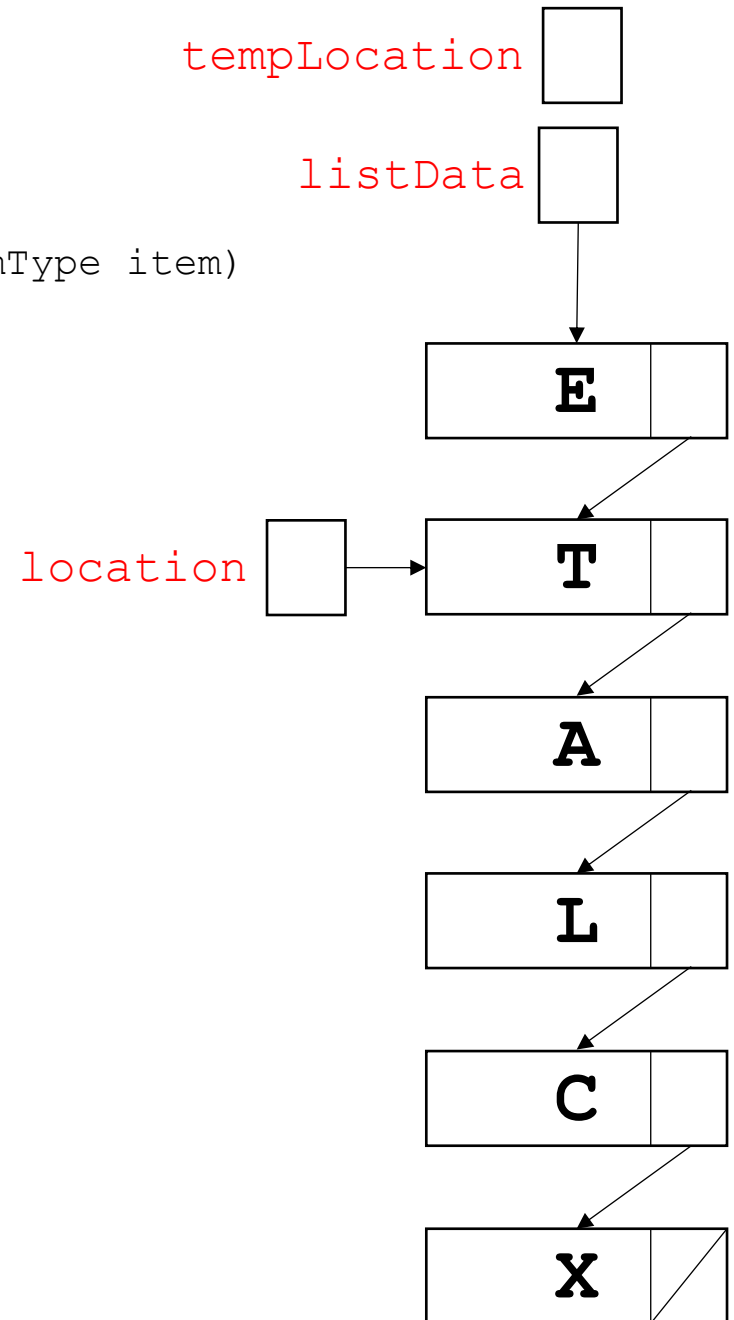
unsortedlinkedlist.cpp

```
template <class ItemType>
void UnsortedType<ItemType>::DeleteItem(ItemType item)
{
    NodeType* location = listData;
    NodeType* tempLocation;
    if (item == listData->info)
    {
        tempLocation = location;
        listData = listData->next;
    }
    else
    {
        while (!(item==(location->next)->info))
            location = location->next;
        tempLocation = location->next;
        location->next = (location->next)->next;
    }
    delete tempLocation;
    length--;
}
```

DeleteItem('L')

length

6



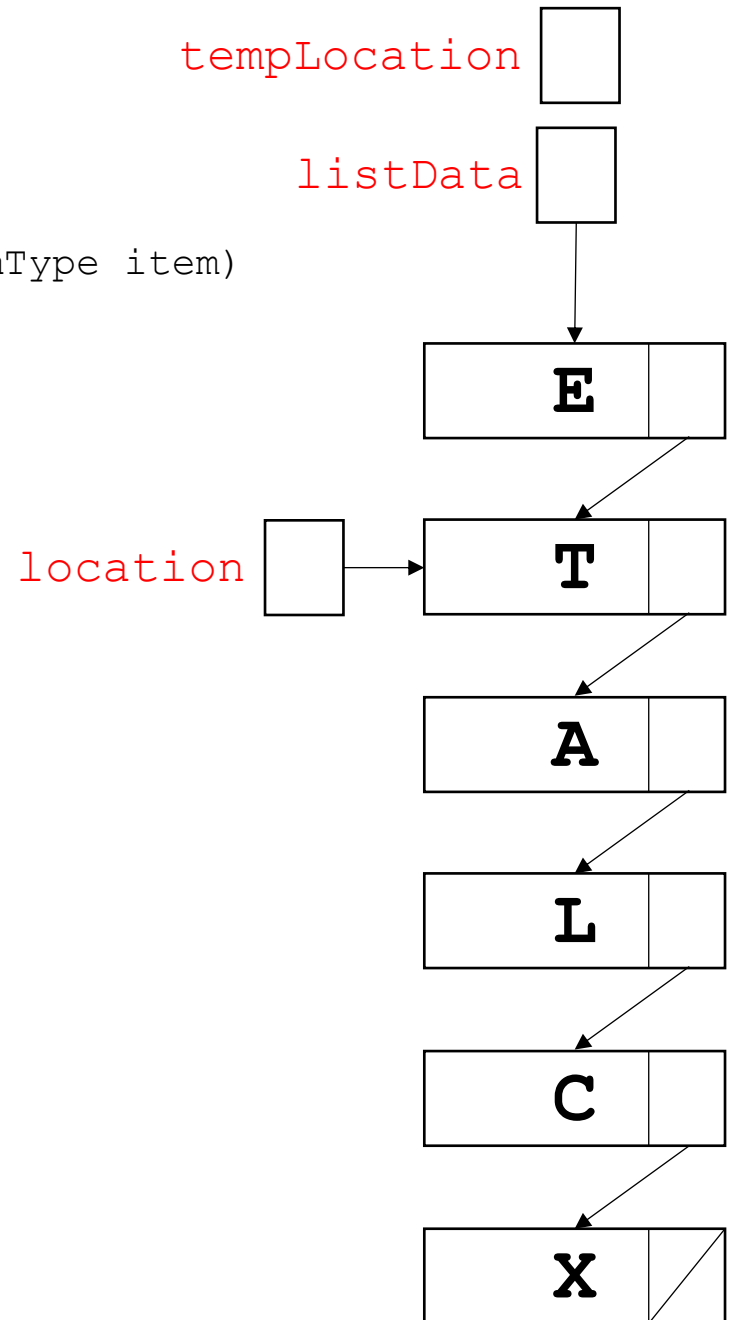
unsortedlinkedlist.cpp

```
template <class ItemType>
void UnsortedType<ItemType>::DeleteItem(ItemType item)
{
    NodeType* location = listData;
    NodeType* tempLocation;
    if (item == listData->info)
    {
        tempLocation = location;
        listData = listData->next;
    }
    else
    {
        while (!(item==(location->next)->info))
            location = location->next;
        tempLocation = location->next;
        location->next = (location->next)->next;
    }
    delete tempLocation;
    length--;
}
```

DeleteItem('L')

length

6



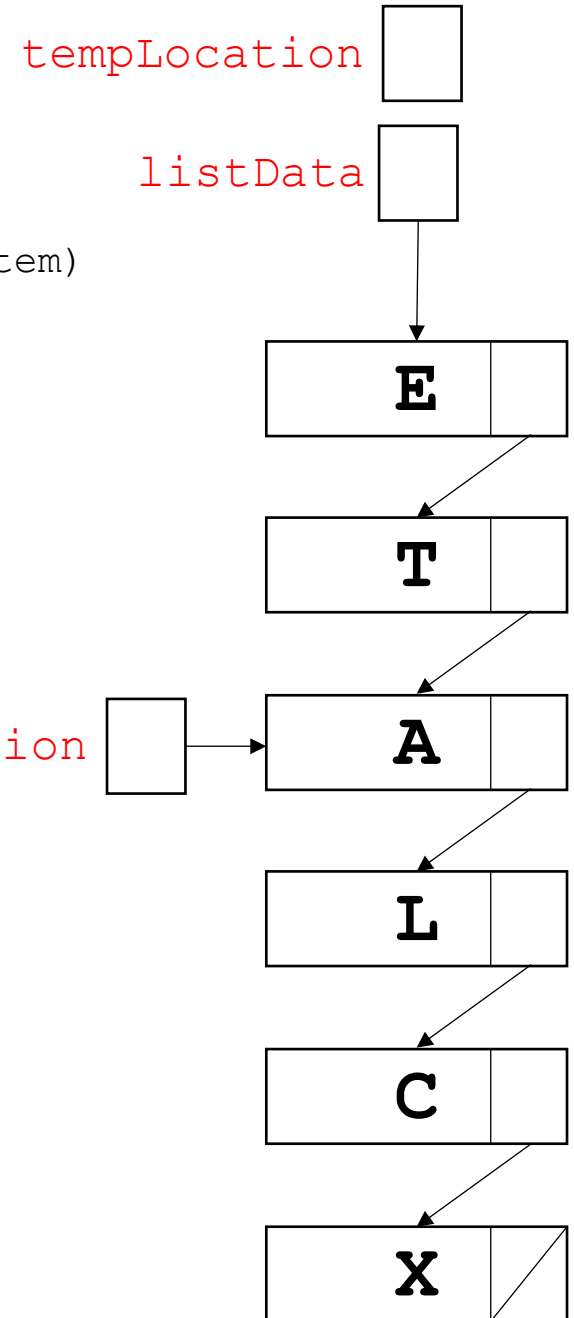
unsortedlinkedlist.cpp

```
template <class ItemType>
void UnsortedType<ItemType>::DeleteItem(ItemType item)
{
    NodeType* location = listData;
    NodeType* tempLocation;
    if (item == listData->info)
    {
        tempLocation = location;
        listData = listData->next;
    }
    else
    {
        while (!(item==(location->next)->info))
            location = location->next;
        tempLocation = location->next;
        location->next = (location->next)->next;
    }
    delete tempLocation;
    length--;
}
```

DeleteItem('L')

length

6



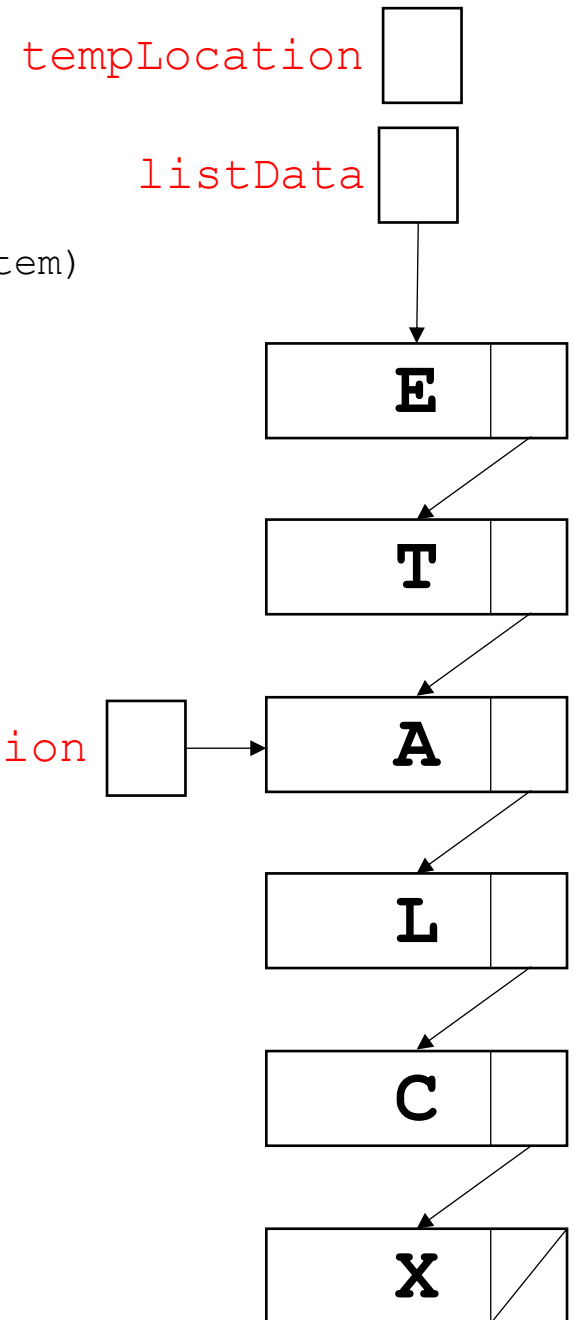
unsortedlinkedlist.cpp

```
template <class ItemType>
void UnsortedType<ItemType>::DeleteItem(ItemType item)
{
    NodeType* location = listData;
    NodeType* tempLocation;
    if (item == listData->info)
    {
        tempLocation = location;
        listData = listData->next;
    }
    else
    {
        while (!(item==(location->next)->info))
            location = location->next;
        tempLocation = location->next;
        location->next = (location->next)->next;
    }
    delete tempLocation;
    length--;
}
```

DeleteItem('L')

length

6



unsortedlinkedlist.cpp

```
template <class ItemType>
void UnsortedType<ItemType>::DeleteItem(ItemType item)
{
    NodeType* location = listData;
    NodeType* tempLocation;
    if (item == listData->info)
    {
        tempLocation = location;
        listData = listData->next;
    }
    else
    {
        while (!(item==(location->next)->info))
            location = location->next;
        tempLocation = location->next;
        location->next = (location->next)->next;
    }
    delete tempLocation;
    length--;
}
```

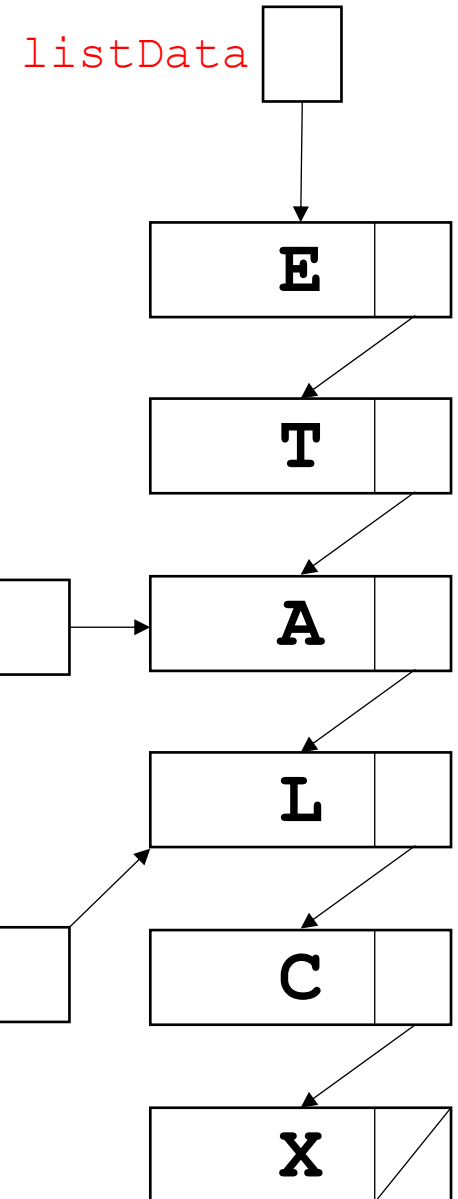
DeleteItem('L')

length

6

location

tempLocation



unsortedlinkedlist.cpp

```
template <class ItemType>
void UnsortedType<ItemType>::DeleteItem(ItemType item)
{
    NodeType* location = listData;
    NodeType* tempLocation;
    if (item == listData->info)
    {
        tempLocation = location;
        listData = listData->next;
    }
    else
    {
        while (!(item==(location->next)->info))
            location = location->next;
        tempLocation = location->next;
        location->next = (location->next)->next;
    }
    delete tempLocation;
    length--;
}
```

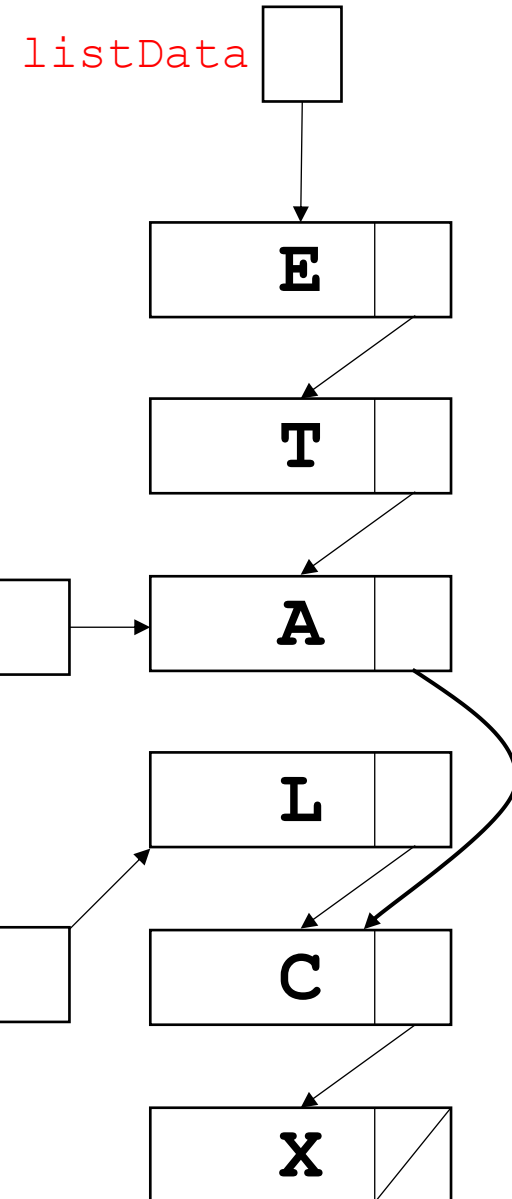
DeleteItem('L')

length

6

location

tempLocation



unsortedlinkedlist.cpp

```
template <class ItemType>
void UnsortedType<ItemType>::DeleteItem(ItemType item)
{
    NodeType* location = listData;
    NodeType* tempLocation;
    if (item == listData->info)
    {
        tempLocation = location;
        listData = listData->next;
    }
    else
    {
        while (!(item==(location->next)->info))
            location = location->next;
        tempLocation = location->next;
        location->next = (location->next)->next;
    }
    delete tempLocation;
    length--;
}
```

DeleteItem('L')

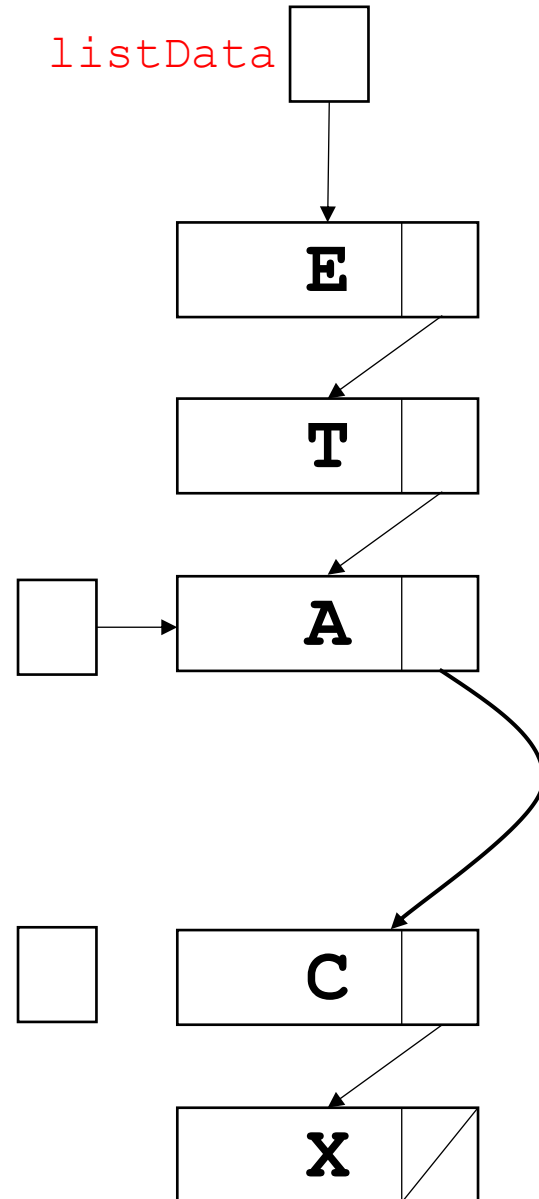
length

6

location

tempLocation

listData



unsortedlinkedlist.cpp

```
template <class ItemType>
void UnsortedType<ItemType>::DeleteItem(ItemType item)
{
    NodeType* location = listData;
    NodeType* tempLocation;
    if (item == listData->info)
    {
        tempLocation = location;
        listData = listData->next;
    }
    else
    {
        while (!(item==(location->next)->info))
            location = location->next;
        tempLocation = location->next;
        location->next = (location->next)->next;
    }
    delete tempLocation;
    length--;
}
```

DeleteItem('L')

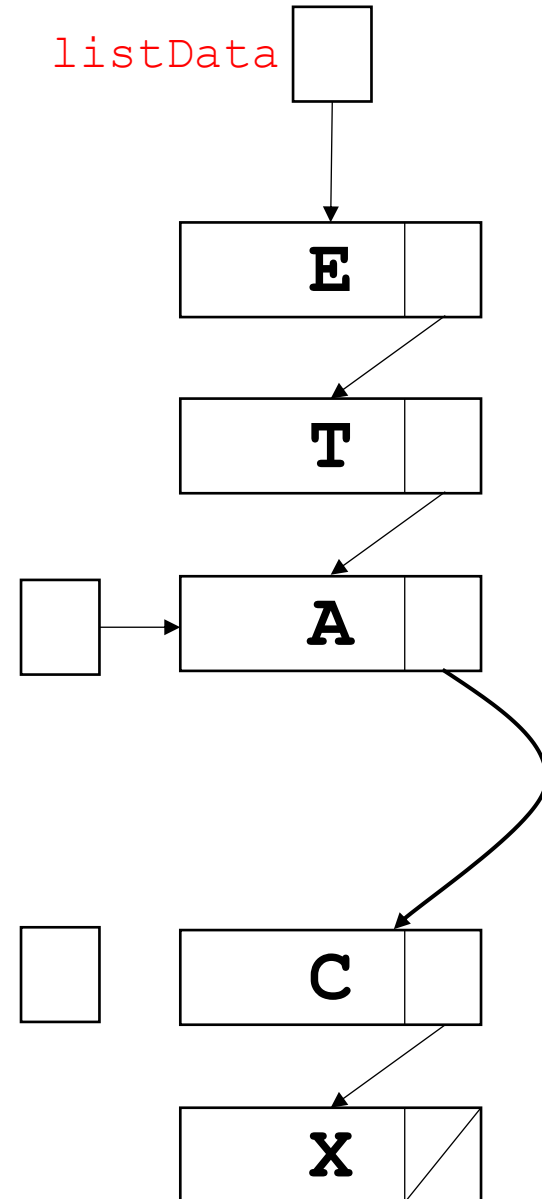
length

5

location

tempLocation

listData

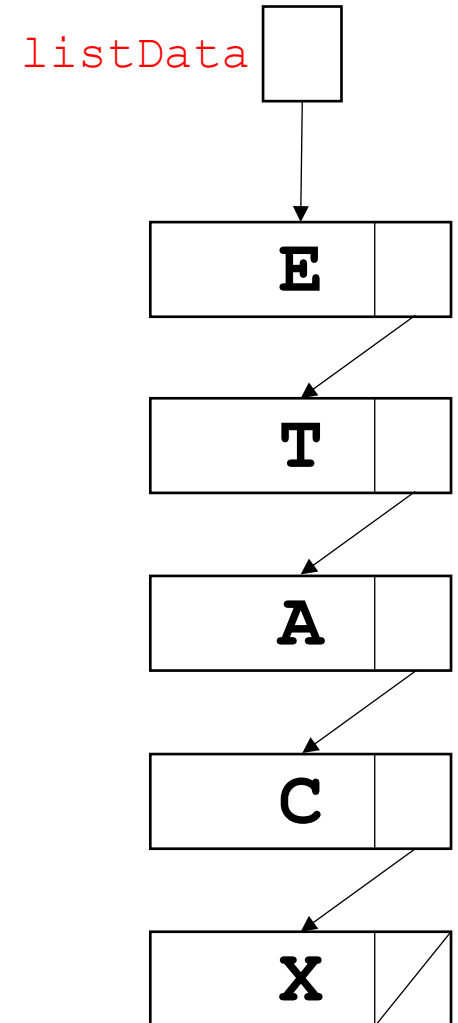


unsortedlinkedlist.cpp

```
template <class ItemType>
void UnsortedType<ItemType>::DeleteItem(ItemType item)
{
    NodeType* location = listData;
    NodeType* tempLocation;
    if (item == listData->info)
    {
        tempLocation = location;
        listData = listData->next;
    }
    else
    {
        while (!(item==(location->next)->info))
            location = location->next;
        tempLocation = location->next;
        location->next = (location->next)->next;
    }
    delete tempLocation;
    length--;
}
```

length

5



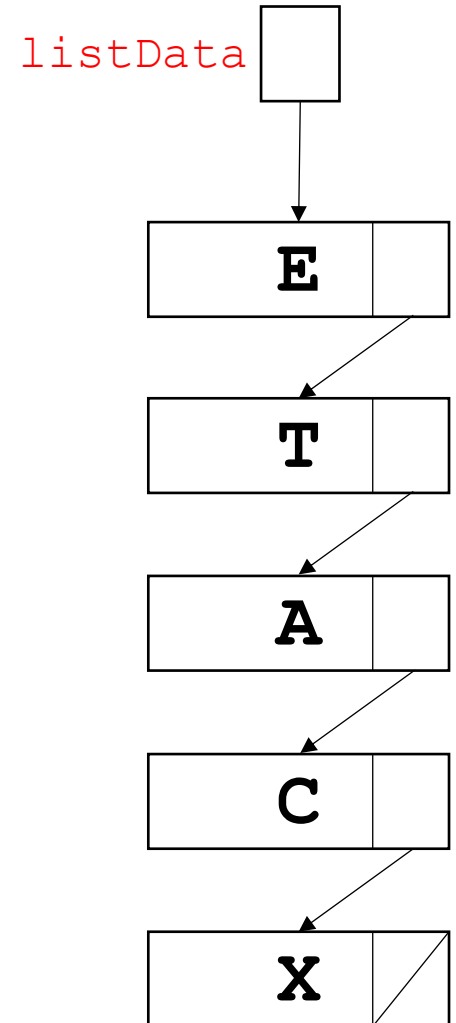
unsortedlinkedlist.cpp

```
template <class ItemType>
void UnsortedType<ItemType>::DeleteItem(ItemType item)
{
    NodeType* location = listData;
    NodeType* tempLocation;
    if (item == listData->info)
    {
        tempLocation = location;
        listData = listData->next;
    }
    else
    {
        while (!(item==(location->next)->info))
            location = location->next;
        tempLocation = location->next;
        location->next = (location->next)->next;
    }
    delete tempLocation;
    length--;
}
```

length

5

DeleteItem('E')



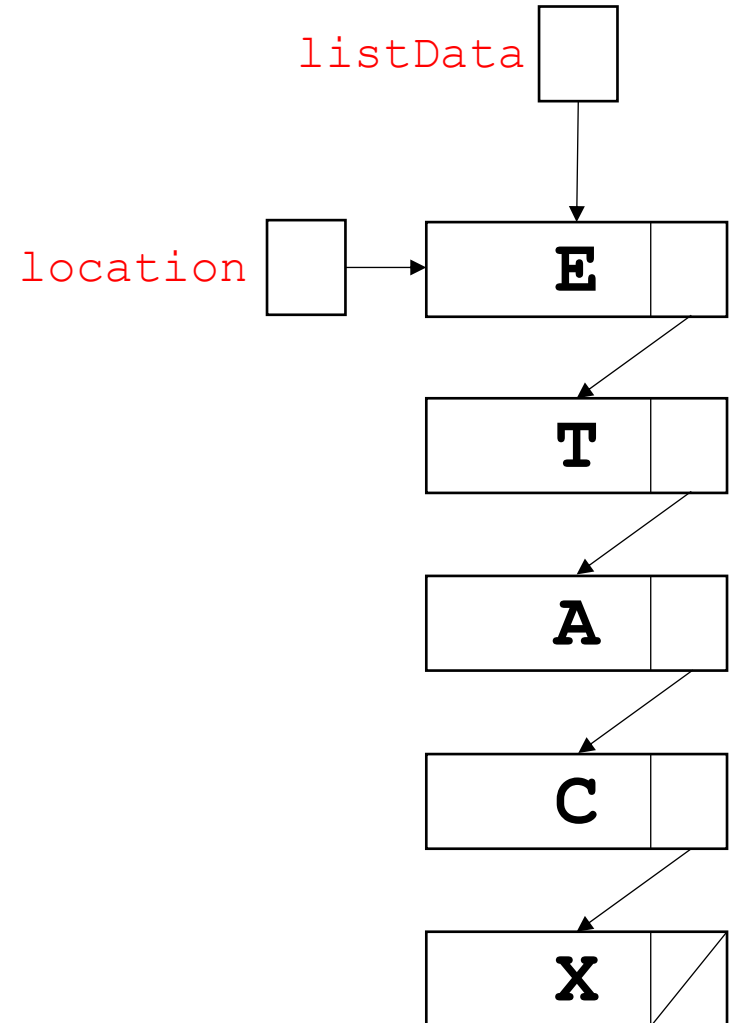
unsortedlinkedlist.cpp

tempLocation

```
template <class ItemType>
void UnsortedType<ItemType>::DeleteItem(ItemType item)
{
    NodeType* location = listData;
    NodeType* tempLocation;
    if (item == listData->info)
    {
        tempLocation = location;
        listData = listData->next;
    }
    else
    {
        while (!(item==(location->next)->info))
            location = location->next;
        tempLocation = location->next;
        location->next = (location->next)->next;
    }
    delete tempLocation;
    length--;
}
```

length

DeleteItem('E')



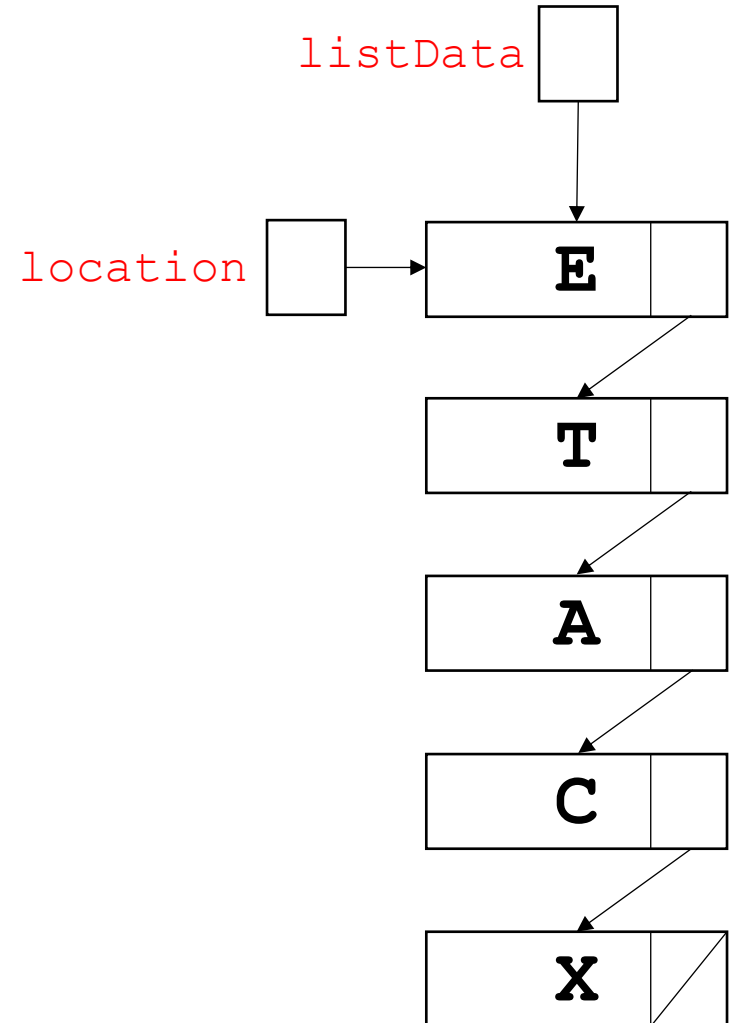
unsortedlinkedlist.cpp

tempLocation 

```
template <class ItemType>
void UnsortedType<ItemType>::DeleteItem(ItemType item)
{
    NodeType* location = listData;
    NodeType* tempLocation;
    if (item == listData->info)
    {
        tempLocation = location;
        listData = listData->next;
    }
    else
    {
        while (!(item==(location->next)->info))
            location = location->next;
        tempLocation = location->next;
        location->next = (location->next)->next;
    }
    delete tempLocation;
    length--;
}
```

length 

DeleteItem('E')



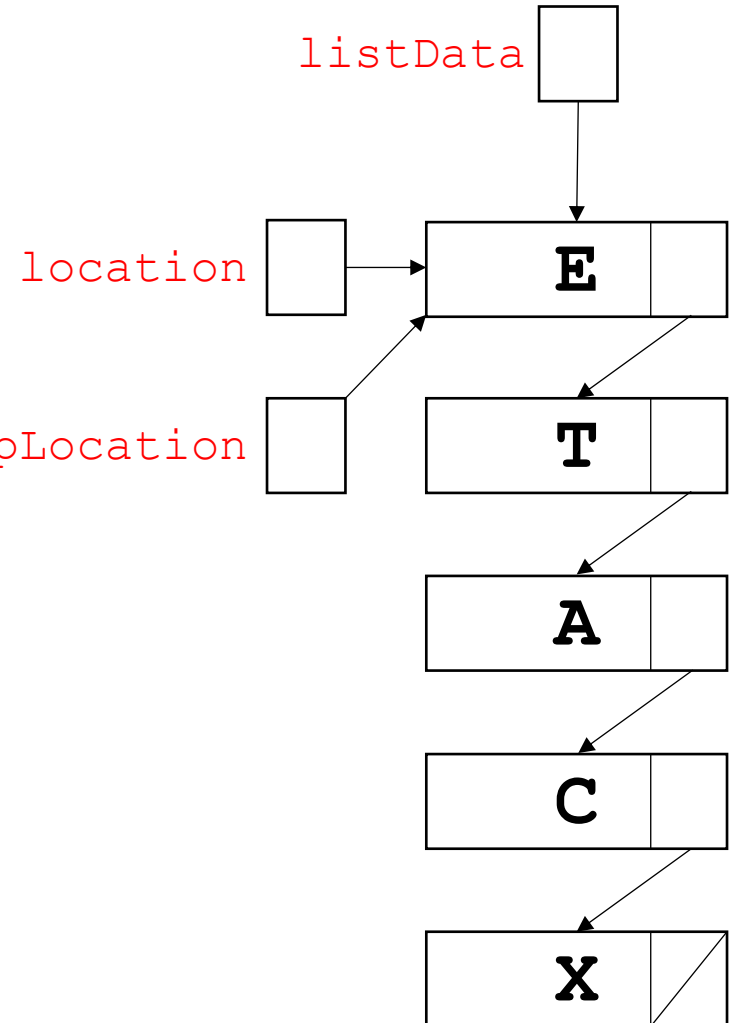
unsortedlinkedlist.cpp

```
template <class ItemType>
void UnsortedType<ItemType>::DeleteItem(ItemType item)
{
    NodeType* location = listData;
    NodeType* tempLocation;
    if (item == listData->info)
    {
        tempLocation = location;
        listData = listData->next;
    }
    else
    {
        while (!(item==(location->next)->info))
            location = location->next;
        tempLocation = location->next;
        location->next = (location->next)->next;
    }
    delete tempLocation;
    length--;
}
```

length

5

DeleteItem('E')



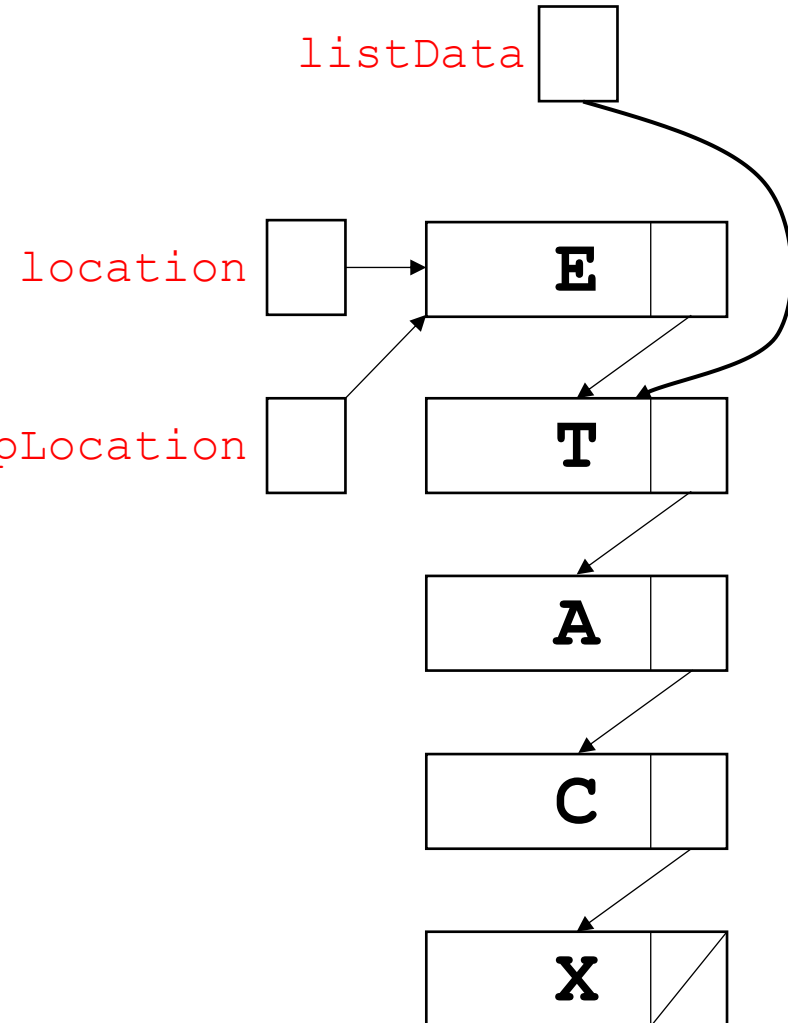
unsortedlinkedlist.cpp

```
template <class ItemType>
void UnsortedType<ItemType>::DeleteItem(ItemType item)
{
    NodeType* location = listData;
    NodeType* tempLocation;
    if (item == listData->info)
    {
        tempLocation = location;
        listData = listData->next;
    }
    else
    {
        while (!(item==(location->next)->info))
            location = location->next;
        tempLocation = location->next;
        location->next = (location->next)->next;
    }
    delete tempLocation;
    length--;
}
```

length

5

DeleteItem('E')



unsortedlinkedlist.cpp

```
template <class ItemType>
void UnsortedType<ItemType>::DeleteItem(ItemType item)
{
    NodeType* location = listData;
    NodeType* tempLocation;
    if (item == listData->info)
    {
        tempLocation = location;
        listData = listData->next;
    }
    else
    {
        while (!(item==(location->next)->info))
        {
            location = location->next;
            tempLocation = location->next;
            location->next = (location->next)->next;
        }
        delete tempLocation;
        length--;
    }
}
```

DeleteItem('E')

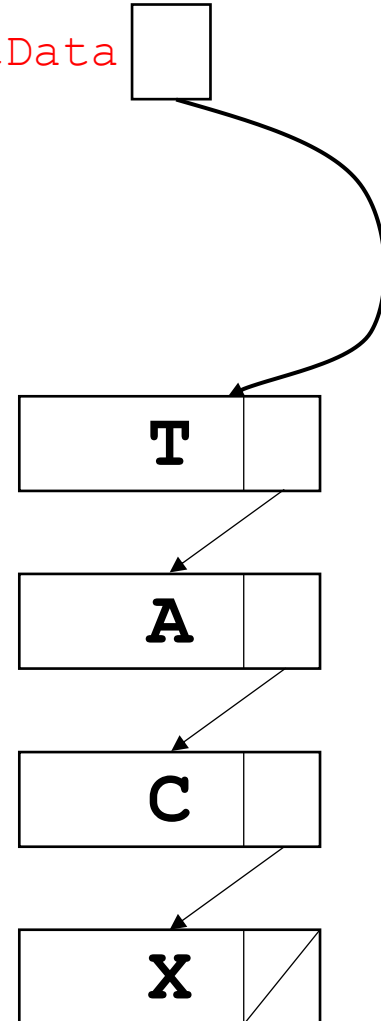
length

5

location

tempLocation

listData



unsortedlinkedlist.cpp

```
template <class ItemType>
void UnsortedType<ItemType>::DeleteItem(ItemType item)
{
    NodeType* location = listData;
    NodeType* tempLocation;
    if (item == listData->info)
    {
        tempLocation = location;
        listData = listData->next;
    }
    else
    {
        while (!(item==(location->next)->info))
            location = location->next;
        tempLocation = location->next;
        location->next = (location->next)->next;
    }
    delete tempLocation;
    length--;
}
```

length

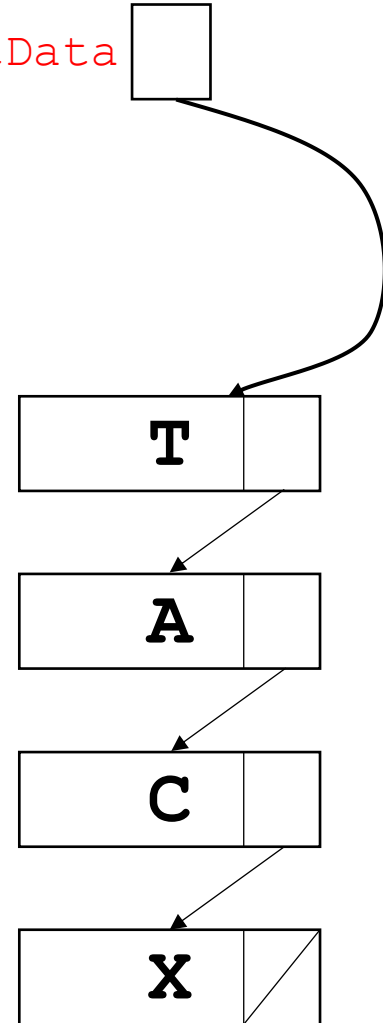
4

DeleteItem('E')

location

tempLocation

listData

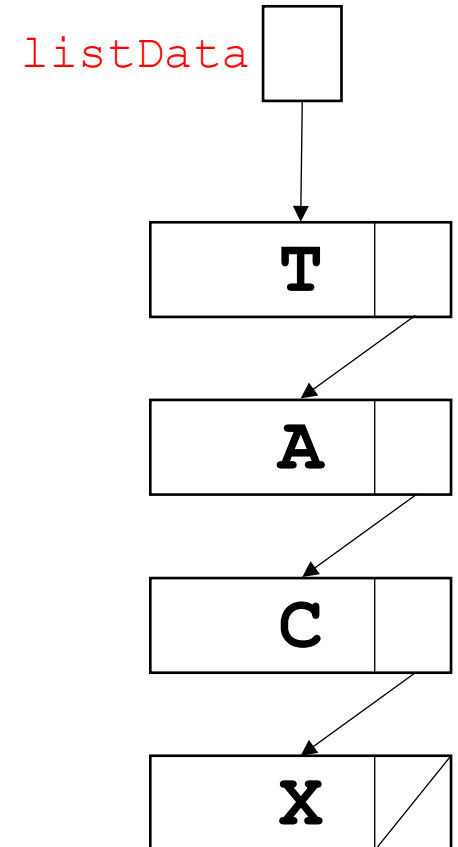


unsortedlinkedlist.cpp

```
template <class ItemType>
void UnsortedType<ItemType>::DeleteItem(ItemType item)
{
    NodeType* location = listData;
    NodeType* tempLocation;
    if (item == listData->info)
    {
        tempLocation = location;
        listData = listData->next;
    }
    else
    {
        while (!(item==(location->next)->info))
            location = location->next;
        tempLocation = location->next;
        location->next = (location->next)->next;
    }
    delete tempLocation;
    length--;
}
```

length

4



unsortedlinkedlist.cpp

```
template <class ItemType>
void UnsortedType<ItemType>::DeleteItem(ItemType item)
{
    NodeType* location = listData;
    NodeType* tempLocation;
    if (item == listData->info)
    {
        tempLocation = location;
        listData = listData->next;
    }
    else
    {
        while (!(item==(location->next)->info))
        {
            location = location->next;
            tempLocation = location->next;
            location->next = (location->next)->next;
        }
        delete tempLocation;
        length--;
    }
}
```

O(N)

unsortedlinkedlist.cpp

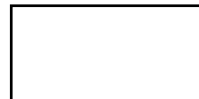
```
template <class ItemType>
void UnsortedType<ItemType>::RetrieveItem(ItemType& item, bool& found)
{
    NodeType* location = listData;
    bool moreToSearch = (location != NULL);
    found = false;
    while (moreToSearch && !found)
    {
        if (item == location->info)
        {
            found = true;
        }
        else
        {
            location = location->next;
            moreToSearch = (location != NULL);
        }
    }
}
```

unsortedlinkedlist.cpp

```
template <class ItemType>
void UnsortedType<ItemType>::RetrieveItem(ItemType& item, bool& found)
{
    NodeType* location = listData;
    bool moreToSearch = (location != NULL);
    found = false;
    while (moreToSearch && !found)
    {
        if (item == location->info)
        {
            found = true;
        }
        else
        {
            location = location->next;
            moreToSearch = (location != NULL);
        }
    }
}
```

RetrieveItem(it, fnd)

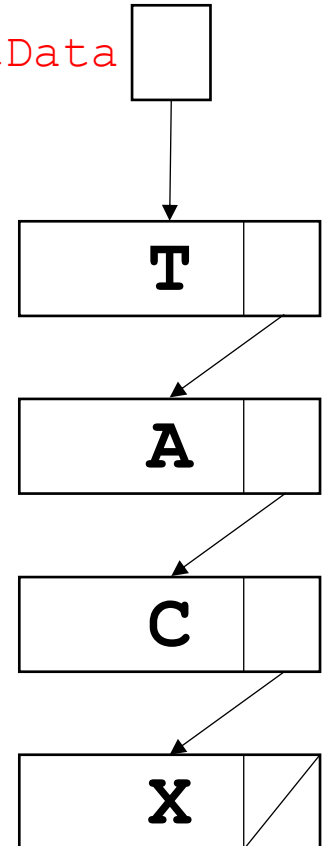
fnd



it



listData

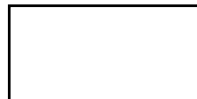


unsortedlinkedlist.cpp

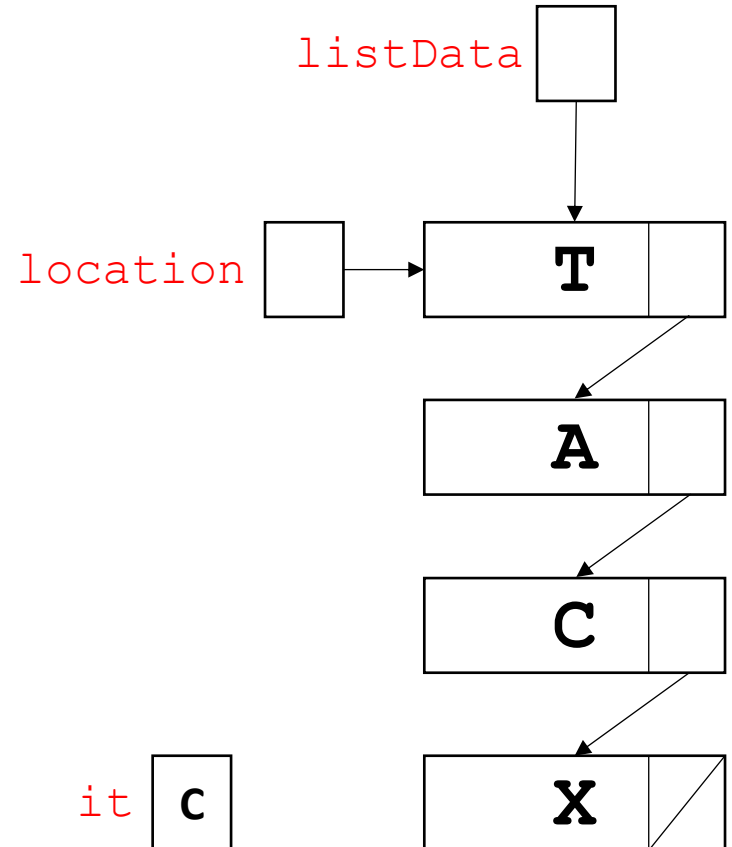
```
template <class ItemType>
void UnsortedType<ItemType>::RetrieveItem(ItemType& item, bool& found)
{
    NodeType* location = listData;
    bool moreToSearch = (location != NULL);
    found = false;
    while (moreToSearch && !found)
    {
        if (item == location->info)
        {
            found = true;
        }
        else
        {
            location = location->next;
            moreToSearch = (location != NULL);
        }
    }
}
```

RetrieveItem(it, fnd)

fnd



it



unsortedlinkedlist.cpp

```
template <class ItemType>
void UnsortedType<ItemType>::RetrieveItem(ItemType& item, bool& found)
{
    NodeType* location = listData;
    bool moreToSearch = (location != NULL);
    found = false;
    while (moreToSearch && !found)
    {
        if (item == location->info)
        {
            found = true;
        }
        else
        {
            location = location->next;
            moreToSearch = (location != NULL);
        }
    }
}
```

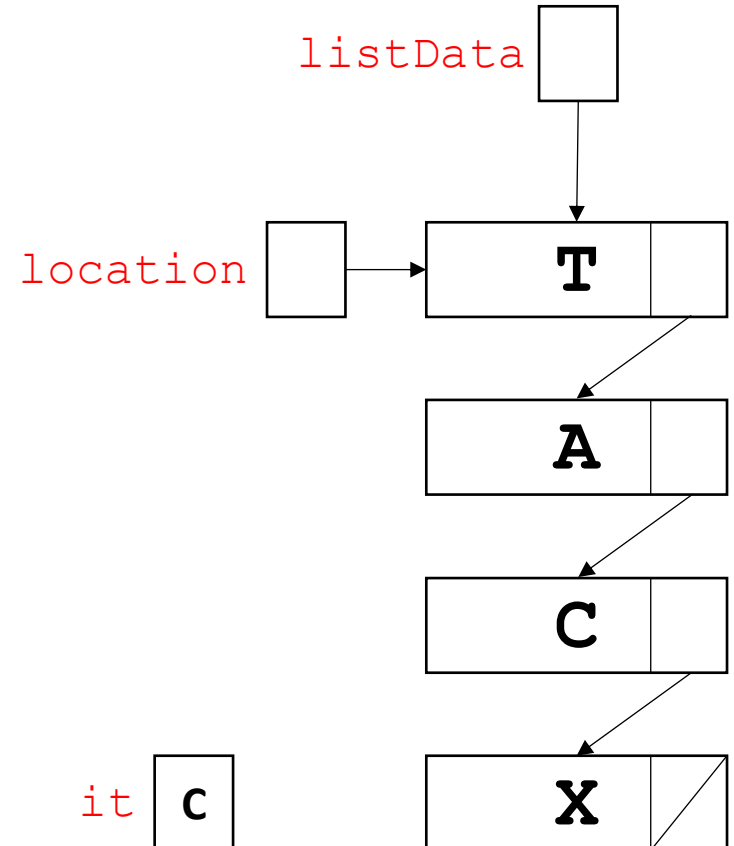
RetrieveItem(it, fnd)

moreToSearch

true

found

--

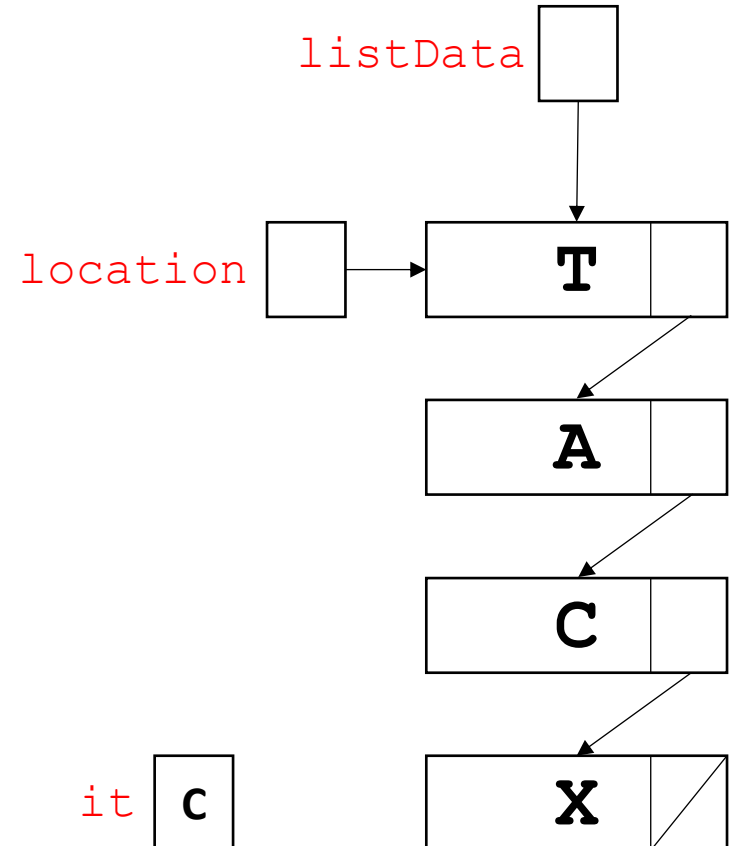


unsortedlinkedlist.cpp

```
template <class ItemType>
void UnsortedType<ItemType>::RetrieveItem(ItemType& item, bool& found)
{
    NodeType* location = listData;
    bool moreToSearch = (location != NULL);
    found = false;
    while (moreToSearch && !found)
    {
        if (item == location->info)
        {
            found = true;
        }
        else
        {
            location = location->next;
            moreToSearch = (location != NULL);
        }
    }
}
```

RetrieveItem(it, fnd)

moreToSearch	true
fnd	false

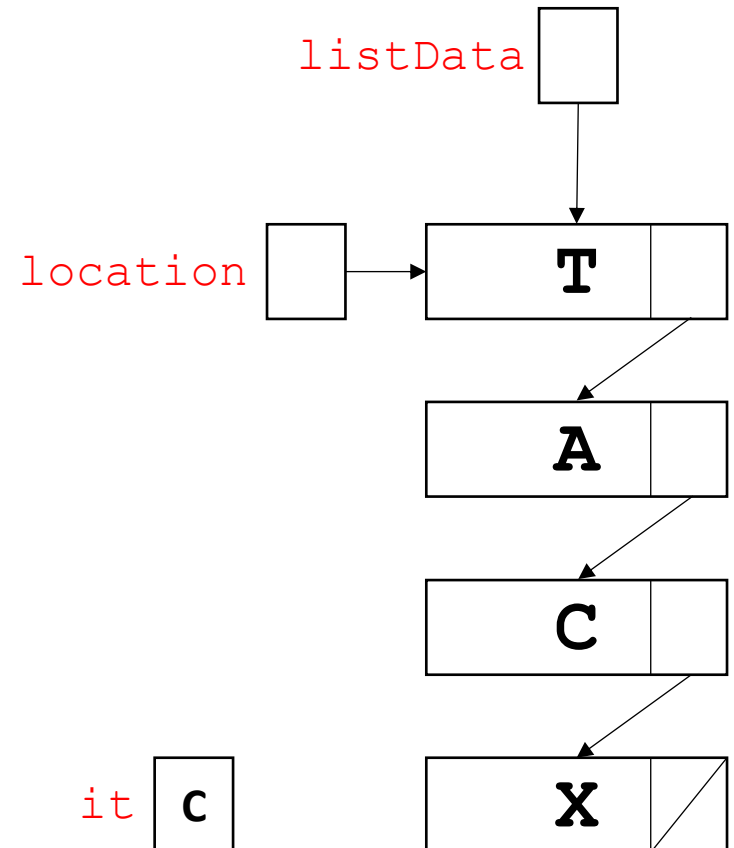


unsortedlinkedlist.cpp

```
template <class ItemType>
void UnsortedType<ItemType>::RetrieveItem(ItemType& item, bool& found)
{
    NodeType* location = listData;
    bool moreToSearch = (location != NULL);
    found = false;
    while (moreToSearch && !found)
    {
        if (item == location->info)
        {
            found = true;
        }
        else
        {
            location = location->next;
            moreToSearch = (location != NULL);
        }
    }
}
```

RetrieveItem(it, fnd)

moreToSearch	true
fnd	false



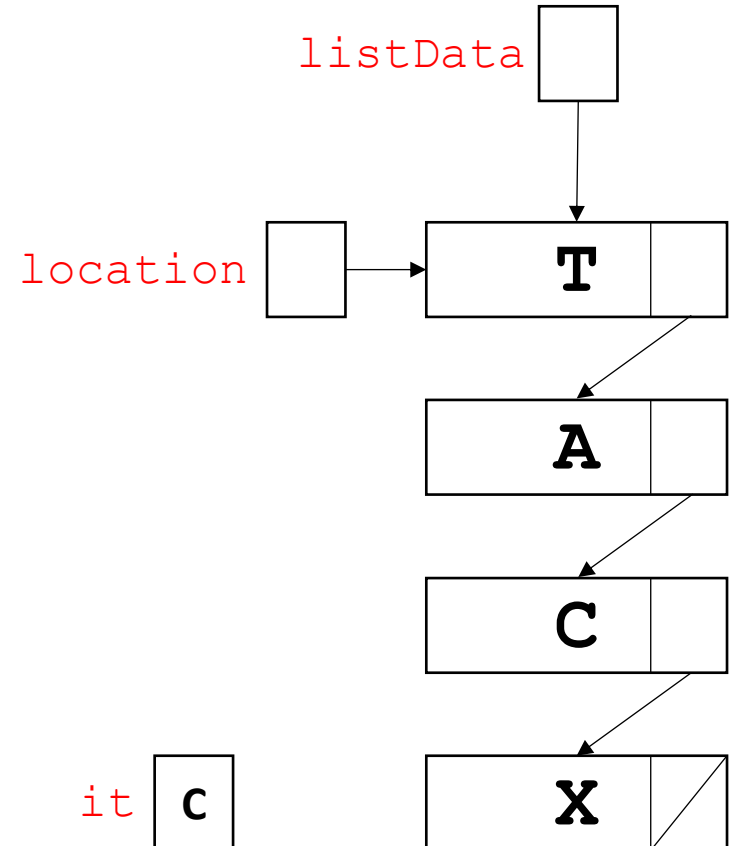
unsortedlinkedlist.cpp

```
template <class ItemType>
void UnsortedType<ItemType>::RetrieveItem(ItemType& item, bool& found)
{
    NodeType* location = listData;
    bool moreToSearch = (location != NULL);
    found = false;
    while (moreToSearch && !found)
    {
        if (item == location->info)
        {
            found = true;
        }
        else
        {
            location = location->next;
            moreToSearch = (location != NULL);
        }
    }
}
```

RetrieveItem(it, fnd)

moreToSearch **true**

fnd **false**



unsortedlinkedlist.cpp

```
template <class ItemType>
void UnsortedType<ItemType>::RetrieveItem(ItemType& item, bool& found)
{
    NodeType* location = listData;
    bool moreToSearch = (location != NULL);
    found = false;
    while (moreToSearch && !found)
    {
        if (item == location->info)
        {
            found = true;
        }
        else
        {
            location = location->next;
            moreToSearch = (location != NULL);
        }
    }
}
```

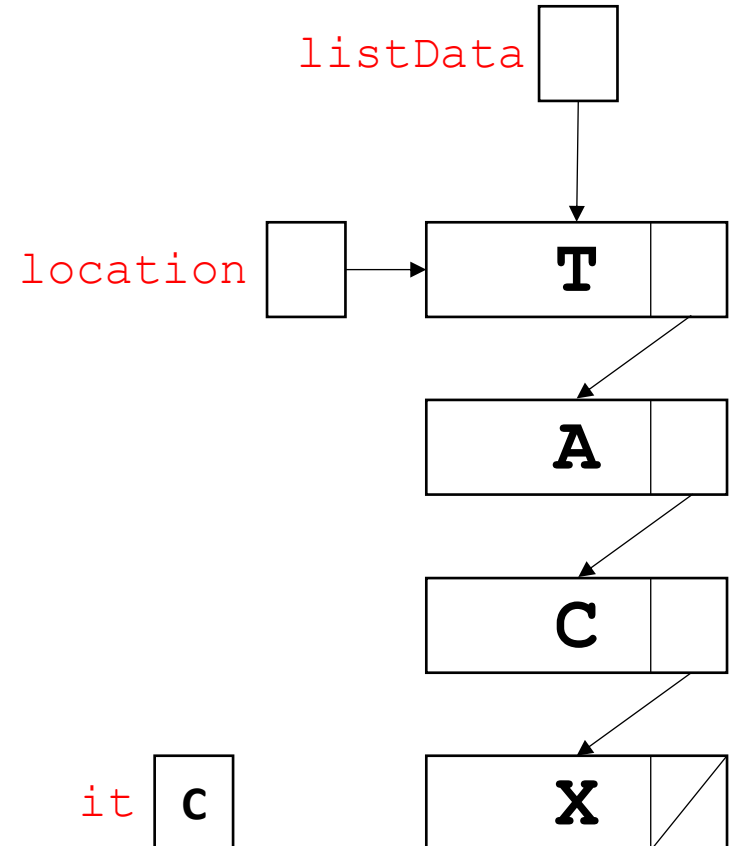
RetrieveItem(it, fnd)

moreToSearch

true

fnd

false

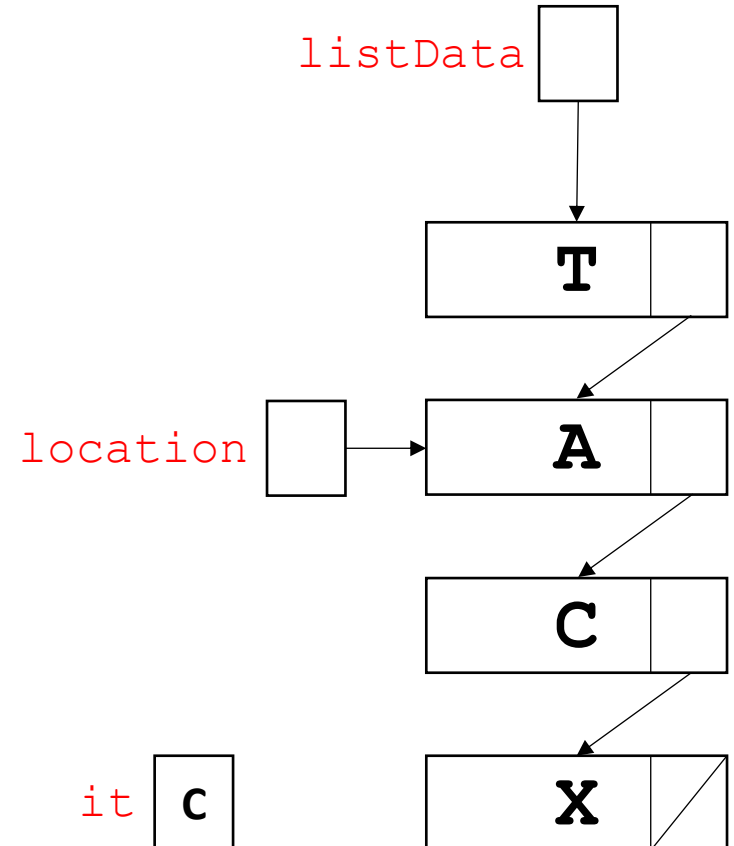


unsortedlinkedlist.cpp

```
template <class ItemType>
void UnsortedType<ItemType>::RetrieveItem(ItemType& item, bool& found)
{
    NodeType* location = listData;
    bool moreToSearch = (location != NULL);
    found = false;
    while (moreToSearch && !found)
    {
        if (item == location->info)
        {
            found = true;
        }
        else
        {
            location = location->next;
            moreToSearch = (location != NULL);
        }
    }
}
```

RetrieveItem(it, fnd)

moreToSearch	true
fnd	false

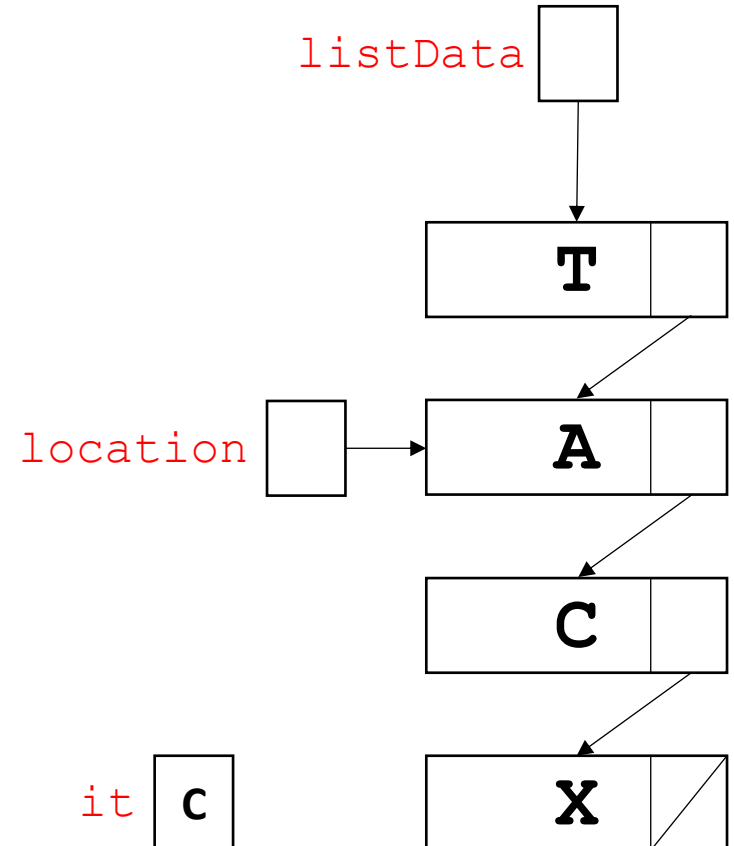


unsortedlinkedlist.cpp

```
template <class ItemType>
void UnsortedType<ItemType>::RetrieveItem(ItemType& item, bool& found)
{
    NodeType* location = listData;
    bool moreToSearch = (location != NULL);
    found = false;
    while (moreToSearch && !found)
    {
        if (item == location->info)
        {
            found = true;
        }
        else
        {
            location = location->next;
            moreToSearch = (location != NULL);
        }
    }
}
```

RetrieveItem(it, fnd)

moreToSearch	true
fnd	false

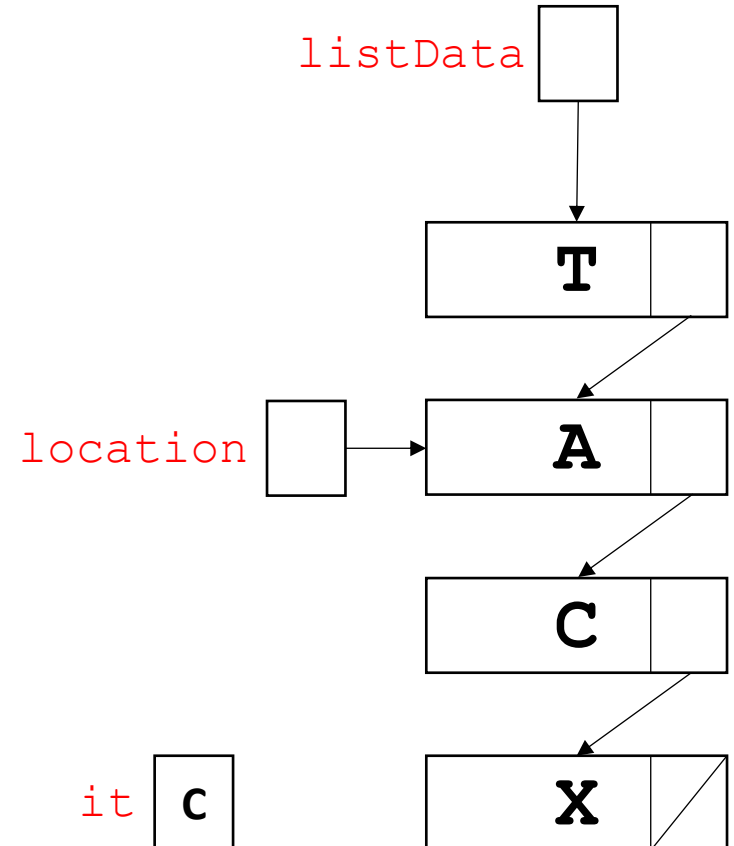


unsortedlinkedlist.cpp

```
template <class ItemType>
void UnsortedType<ItemType>::RetrieveItem(ItemType& item, bool& found)
{
    NodeType* location = listData;
    bool moreToSearch = (location != NULL);
    found = false;
    while (moreToSearch && !found)
    {
        if (item == location->info)
        {
            found = true;
        }
        else
        {
            location = location->next;
            moreToSearch = (location != NULL);
        }
    }
}
```

RetrieveItem(it, fnd)

moreToSearch	true
fnd	false

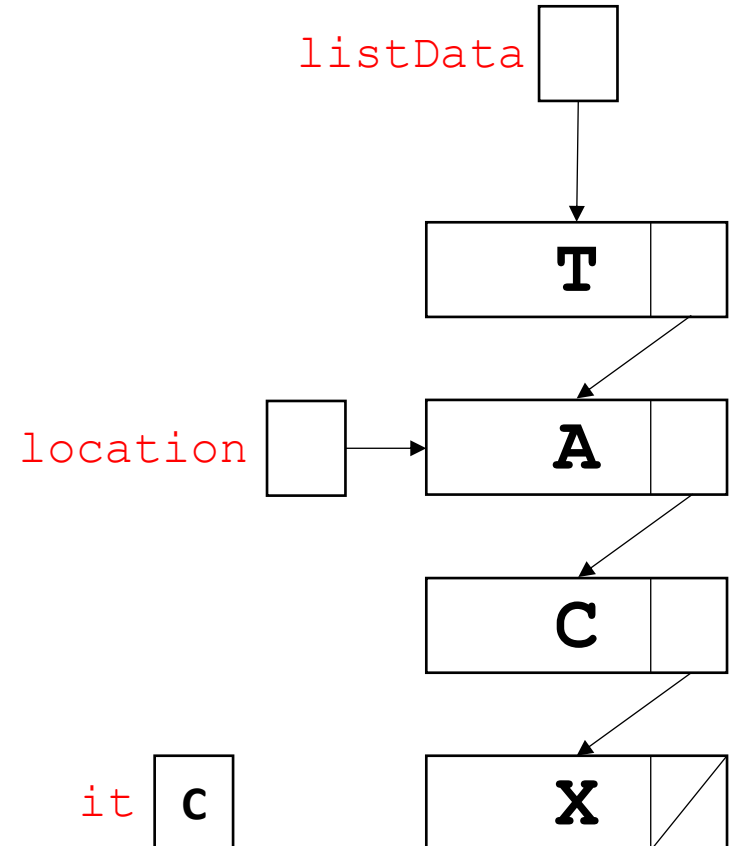


unsortedlinkedlist.cpp

```
template <class ItemType>
void UnsortedType<ItemType>::RetrieveItem(ItemType& item, bool& found)
{
    NodeType* location = listData;
    bool moreToSearch = (location != NULL);
    found = false;
    while (moreToSearch && !found)
    {
        if (item == location->info)
        {
            found = true;
        }
        else
        {
            location = location->next;
            moreToSearch = (location != NULL);
        }
    }
}
```

RetrieveItem(it, fnd)

moreToSearch	true
fnd	false

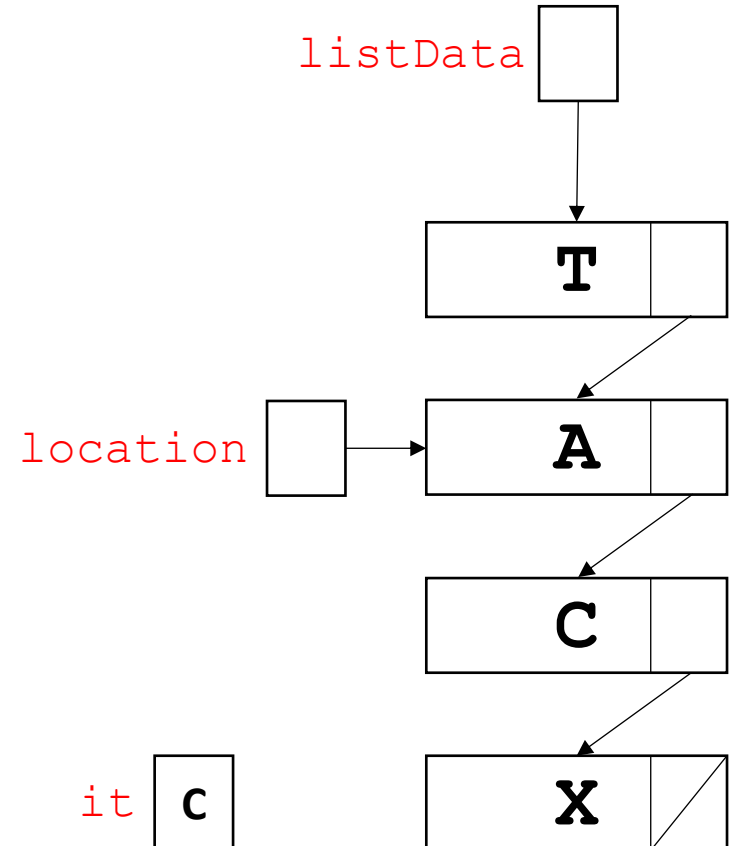


unsortedlinkedlist.cpp

```
template <class ItemType>
void UnsortedType<ItemType>::RetrieveItem(ItemType& item, bool& found)
{
    NodeType* location = listData;
    bool moreToSearch = (location != NULL);
    found = false;
    while (moreToSearch && !found)
    {
        if (item == location->info)
        {
            found = true;
        }
        else
        {
            location = location->next;
            moreToSearch = (location != NULL);
        }
    }
}
```

RetrieveItem(it, fnd)

moreToSearch	true
fnd	false



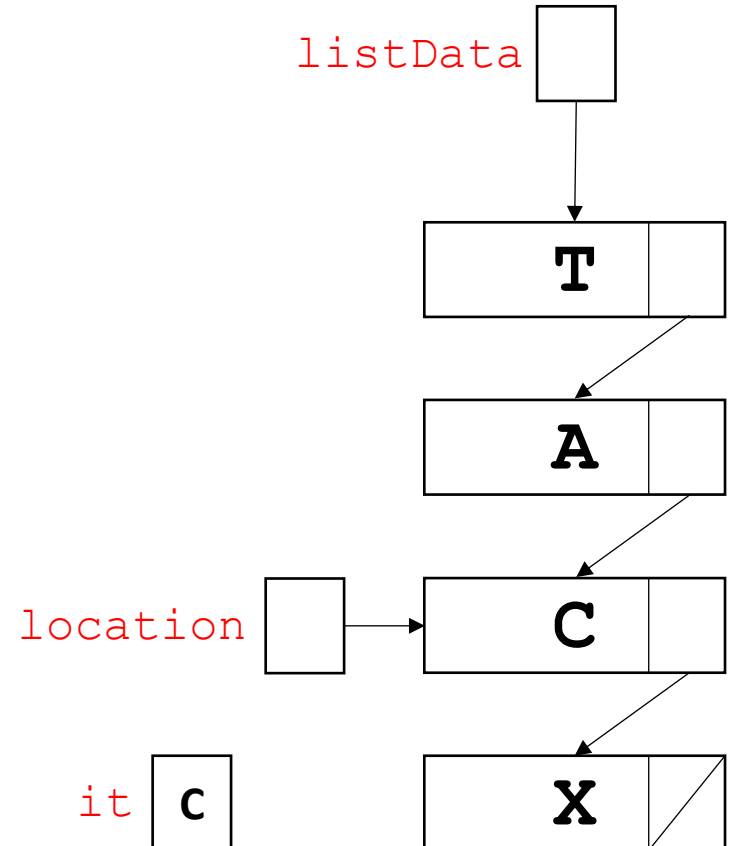
unsortedlinkedlist.cpp

```
template <class ItemType>
void UnsortedType<ItemType>::RetrieveItem(ItemType& item, bool& found)
{
    NodeType* location = listData;
    bool moreToSearch = (location != NULL);
    found = false;
    while (moreToSearch && !found)
    {
        if (item == location->info)
        {
            found = true;
        }
        else
        {
            location = location->next;
            moreToSearch = (location != NULL);
        }
    }
}
```

RetrieveItem(it, fnd)

moreToSearch **true**

fnd **false**



unsortedlinkedlist.cpp

```
template <class ItemType>
void UnsortedType<ItemType>::RetrieveItem(ItemType& item, bool& found)
{
    NodeType* location = listData;
    bool moreToSearch = (location != NULL);
    found = false;
    while (moreToSearch && !found)
    {
        if (item == location->info)
        {
            found = true;
        }
        else
        {
            location = location->next;
            moreToSearch = (location != NULL);
        }
    }
}
```

RetrieveItem(it, fnd)

moreToSearch

true

fnd

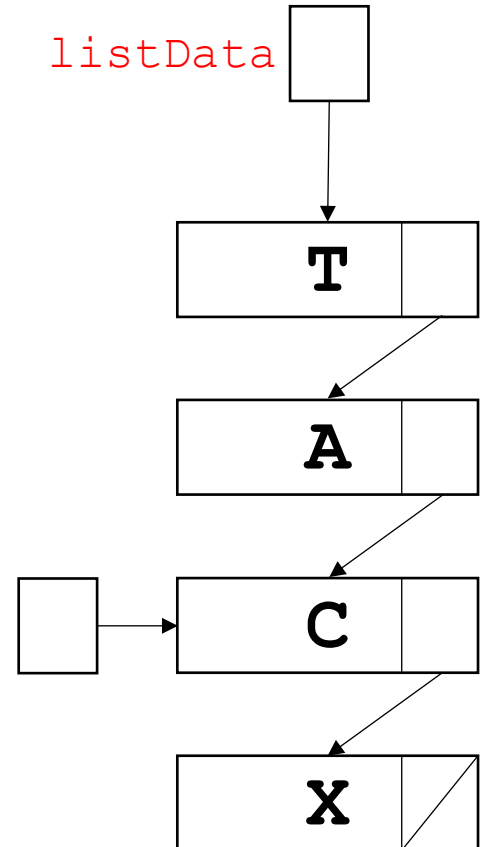
false

location

it

C

listData



unsortedlinkedlist.cpp

```
template <class ItemType>
void UnsortedType<ItemType>::RetrieveItem(ItemType& item, bool& found)
{
    NodeType* location = listData;
    bool moreToSearch = (location != NULL);
    found = false;
    while (moreToSearch && !found)
    {
        if (item == location->info)
        {
            found = true;
        }
        else
        {
            location = location->next;
            moreToSearch = (location != NULL);
        }
    }
}
```

RetrieveItem(it, fnd)

moreToSearch

true

fnd

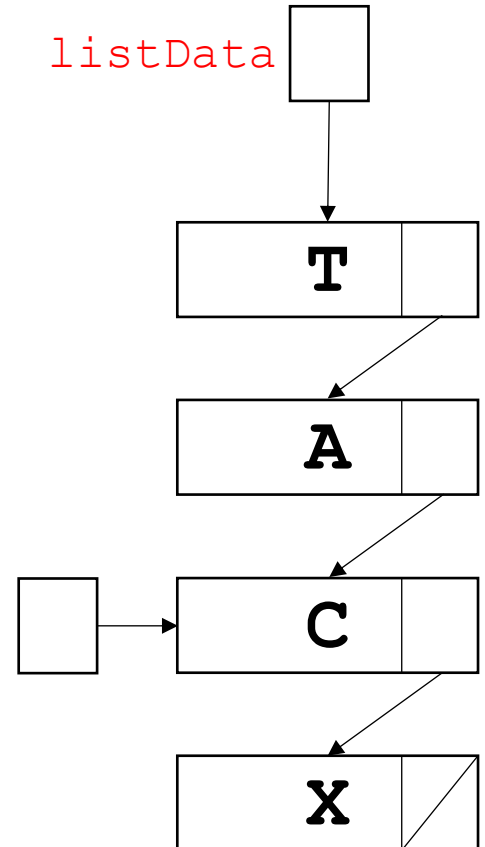
false

location

it

C

listData



unsortedlinkedlist.cpp

```
template <class ItemType>
void UnsortedType<ItemType>::RetrieveItem(ItemType& item, bool& found)
{
    NodeType* location = listData;
    bool moreToSearch = (location != NULL);
    found = false;
    while (moreToSearch && !found)
    {
        if (item == location->info)
        {
            found = true;
        }
        else
        {
            location = location->next;
            moreToSearch = (location != NULL);
        }
    }
}
```

RetrieveItem(it, fnd)

moreToSearch

true

fnd

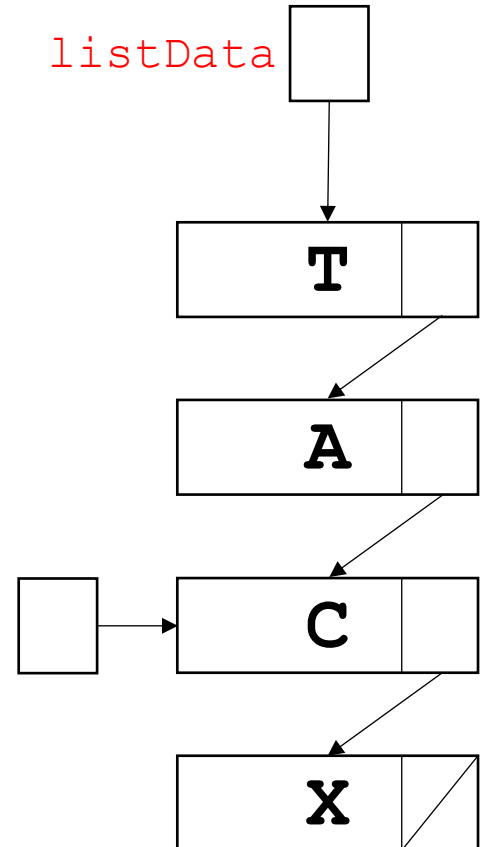
false

location

it

C

listData



unsortedlinkedlist.cpp

```
template <class ItemType>
void UnsortedType<ItemType>::RetrieveItem(ItemType& item, bool& found)
{
    NodeType* location = listData;
    bool moreToSearch = (location != NULL);
    found = false;
    while (moreToSearch && !found)
    {
        if (item == location->info)
        {
            found = true;
        }
        else
        {
            location = location->next;
            moreToSearch = (location != NULL);
        }
    }
}
```

RetrieveItem(it, fnd)

moreToSearch

true

fnd

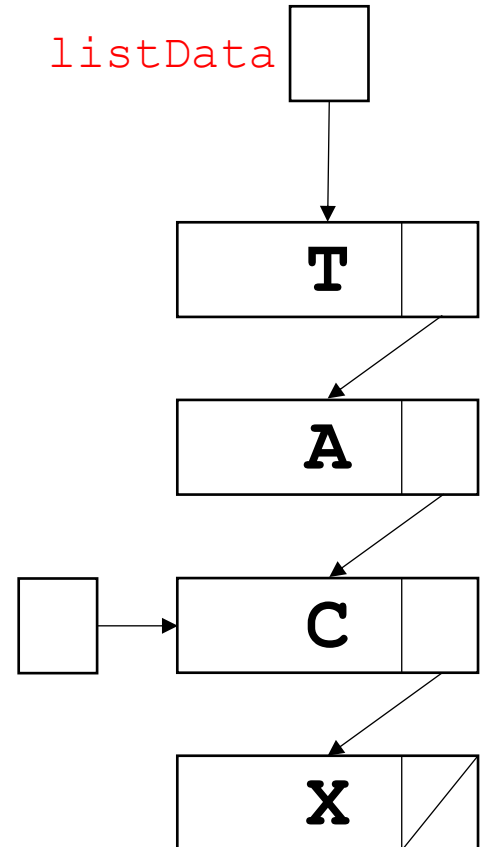
true

location

it

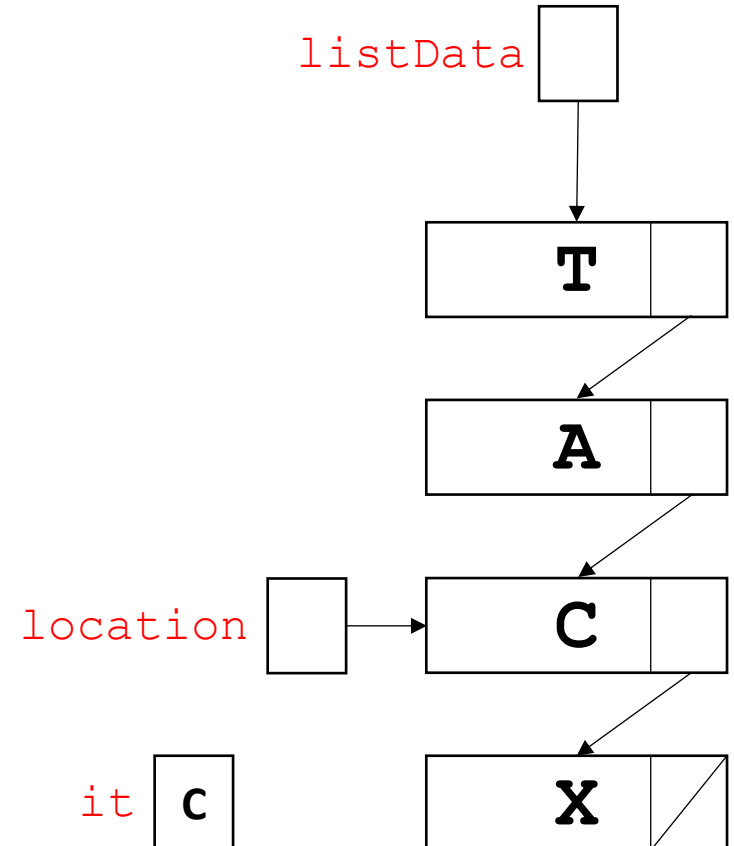
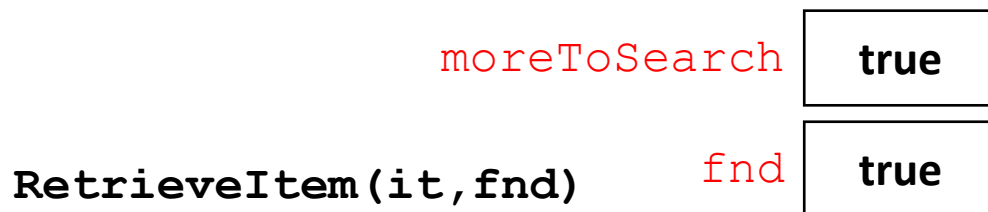
C

listData



unsortedlinkedlist.cpp

```
template <class ItemType>
void UnsortedType<ItemType>::RetrieveItem(ItemType& item, bool& found)
{
    NodeType* location = listData;
    bool moreToSearch = (location != NULL);
    found = false;
    while (moreToSearch && !found)
    {
        if (item == location->info)
        {
            found = true;
        }
        else
        {
            location = location->next;
            moreToSearch = (location != NULL);
        }
    }
}
```

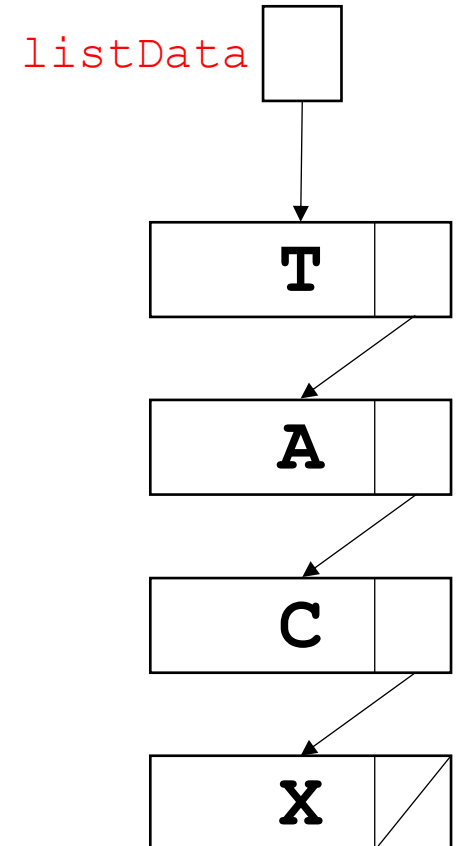


unsortedlinkedlist.cpp

```
template <class ItemType>
void UnsortedType<ItemType>::RetrieveItem(ItemType& item, bool& found)
{
    NodeType* location = listData;
    bool moreToSearch = (location != NULL);
    found = false;
    while (moreToSearch && !found)
    {
        if (item == location->info)
        {
            found = true;
        }
        else
        {
            location = location->next;
            moreToSearch = (location != NULL);
        }
    }
}
```

found **true**

it **C**



unsortedlinkedlist.cpp

```
template <class ItemType>
void UnsortedType<ItemType>::RetrieveItem(ItemType& item, bool& found)
{
    NodeType* location = listData;
    bool moreToSearch = (location != NULL);
    found = false;
    while (moreToSearch && !found)
    {
        if (item == location->info)
        {
            found = true;
        }
        else
        {
            location = location->next;
            moreToSearch = (location != NULL);
        }
    }
}
```

O(N)

unsortedlinkedlist.cpp

```
template <class ItemType>
void UnsortedType<ItemType>::MakeEmpty()
{
    NodeType* tempPtr;

    while (listData != NULL)
    {
        tempPtr = listData;
        listData = listData->next;
        delete tempPtr;
    }
    length = 0;
}

template <class ItemType>
UnsortedType<ItemType>::~~UnsortedType()
{
    MakeEmpty();
}
```

unsortedlinkedlist.cpp

```
template <class ItemType>
void UnsortedType<ItemType>::MakeEmpty()
{
    NodeType* tempPtr;

    while (listData != NULL)
    {
        tempPtr = listData;
        listData = listData->next;
        delete tempPtr;
    }
    length = 0;
}
```

O(N)

```
template <class ItemType>
UnsortedType<ItemType>::~~UnsortedType()
{
    MakeEmpty();
}
```

O(N)

unsortedlinkedlist.cpp

```
template <class ItemType>
void UnsortedType<ItemType>::ResetList()
{
    currentPos = NULL;
}

template <class ItemType>
void UnsortedType<ItemType>::GetNextItem(ItemType& item)
{
    if (currentPos == NULL)
        currentPos = listData;
    else
        currentPos = currentPos->next;
    item = currentPos->info;
}
```

unsortedlinkedlist.cpp

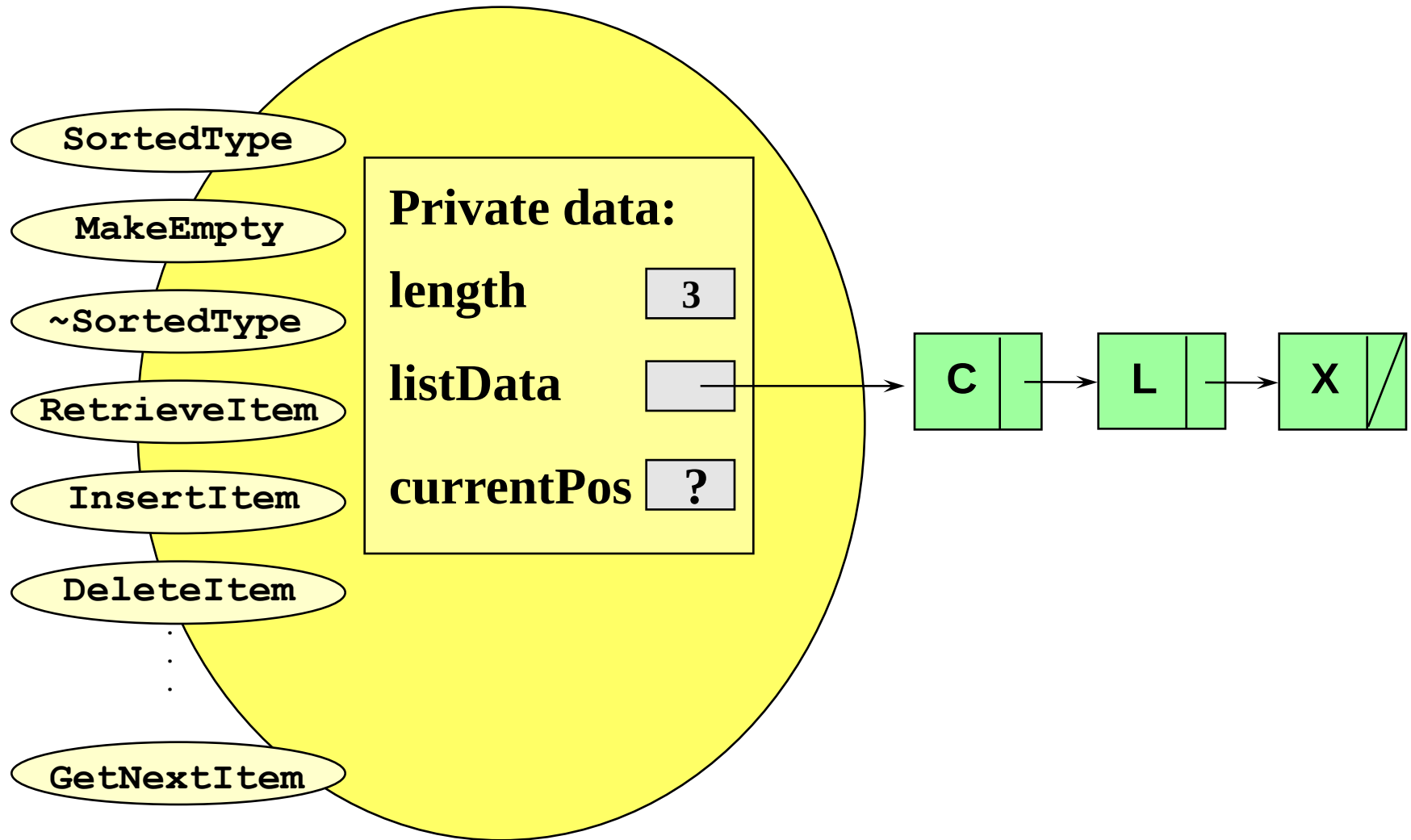
```
template <class ItemType>
void UnsortedType<ItemType>::ResetList()
{
    currentPos = NULL;
}
```

O(1)

```
template <class ItemType>
void UnsortedType<ItemType>::GetNextItem(ItemType& item)
{
    if (currentPos == NULL)
        currentPos = listData;
    else
        currentPos = currentPos->next;
    item = currentPos->info;
}
```

O(1)

class SortedType<char>



sortedlinkedlist.h

```
#ifndef SORTEDLINKEDLIST_H_INCLUDED
#define SORTEDLINKEDLIST_H_INCLUDED
```

```
template <class ItemType>
class SortedType
{
```

```
    struct NodeType
    {
        ItemType info;
        NodeType* next;
    };
```

```
public:
```

```
    SortedType();
    ~SortedType();
    bool IsFull();
    int LengthIs();
    void MakeEmpty();
    void RetrieveItem(ItemType& item, bool& found);
```

```
    void InsertItem(ItemType item);
    void DeleteItem(ItemType item);
    void ResetList();
    void GetNextItem(ItemType& item);
```

```
private:
```

```
    NodeType* listData;
    int length;
    NodeType* currentPos;
};
```

```
#endif // SORTEDLINKEDLIST_H_INCLUDED
```

sortedlinkedlist.cpp

```
#include "sortedlinkedlist.h"
#include<cstddef>
#include<new>

template <class ItemType>
SortedType<ItemType>::SortedType()
{
    length = 0;
    listData = NULL;
    currentPos = NULL;
}

template <class ItemType>
int SortedType<ItemType>::LengthIs()
{
    return length;
}
```

```
template<class ItemType>
bool SortedType<ItemType>::IsFull()
{
    NodeType* location;
    try
    {
        location = new NodeType;
        delete location;
        return false;
    }
    catch(std::bad_alloc& exception)
    {
        return true;
    }
}
```

sortedlinkedlist.cpp

```
#include "sortedlinkedlist.h"
#include<cstddef>
#include<new>

template <class ItemType>
SortedType<ItemType>::SortedType()
{
    length = 0;
    listData = NULL;
    currentPos = NULL;
}

template <class ItemType>
int SortedType<ItemType>::LengthIs()
{
    return length;
}
```

O(1)

O(1)

```
template<class ItemType>
bool SortedType<ItemType>::IsFull()
{
    NodeType* location;
    try
    {
        location = new NodeType;
        delete location;
        return false;
    }
    catch(std::bad_alloc& exception)
    {
        return true;
    }
}
```

O(1)

sortedlinkedlist.cpp

```
template <class ItemType>
void SortedType<ItemType>::MakeEmpty()
{
    NodeType* tempPtr;

    while (listData != NULL)
    {
        tempPtr = listData;
        listData = listData->next;
        delete tempPtr;
    }
    length = 0;
}

template <class ItemType>
SortedType<ItemType>::~~SortedType()
{
    MakeEmpty();
}
```

sortedlinkedlist.cpp

```
template <class ItemType>
void SortedType<ItemType>::MakeEmpty()
{
    NodeType* tempPtr;

    while (listData != NULL)
    {
        tempPtr = listData;
        listData = listData->next;
        delete tempPtr;
    }
    length = 0;
}
```

O(N)

```
template <class ItemType>
SortedType<ItemType>::~~SortedType()
{
    MakeEmpty();
}
```

O(N)

sortedlinkedlist.cpp

```
template <class ItemType>
void SortedType<ItemType>::ResetList()
{
    currentPos = NULL;
}

template <class ItemType>
void SortedType<ItemType>::GetNextItem(ItemType& item)
{
    if (currentPos == NULL)
        currentPos = listData;
    else
        currentPos = currentPos->next;
    item = currentPos->info;
}
```

sortedlinkedlist.cpp

```
template <class ItemType>
void SortedType<ItemType>::ResetList()
{
    currentPos = NULL;
}
```

O(1)

```
template <class ItemType>
void SortedType<ItemType>::GetNextItem(ItemType& item)
{
    if (currentPos == NULL)
        currentPos = listData;
    else
        currentPos = currentPos->next;
    item = currentPos->info;
}
```

O(1)

sortedlinkedlist.cpp

```
template <class ItemType>
void SortedType<ItemType>::InsertItem(ItemType item)
{
    NodeType* newNode;
    NodeType* predLoc;
    NodeType* location;
    bool moreToSearch;

    location = listData;
    predLoc = NULL;
    moreToSearch = (location != NULL);
    while (moreToSearch)
    {
        if (location->info < item)
        {
            predLoc = location;
            location = location->next;
            moreToSearch = (location != NULL);
        }
        else moreToSearch = false;
    }
}
```

```
newNode = new NodeType;
newNode->info = item;

if (predLoc == NULL)
{
    newNode->next = listData;
    listData = newNode;
}
else
{
    newNode->next = location;
    predLoc->next = newNode;
}
length++;
}
```

InsertItem('L')

length 0 listData

InsertItem('L')

```
template <class ItemType>
void SortedType<ItemType>::InsertItem(ItemType item)
{
    NodeType* newNode;
    NodeType* predLoc;
    NodeType* location;
    bool moreToSearch;

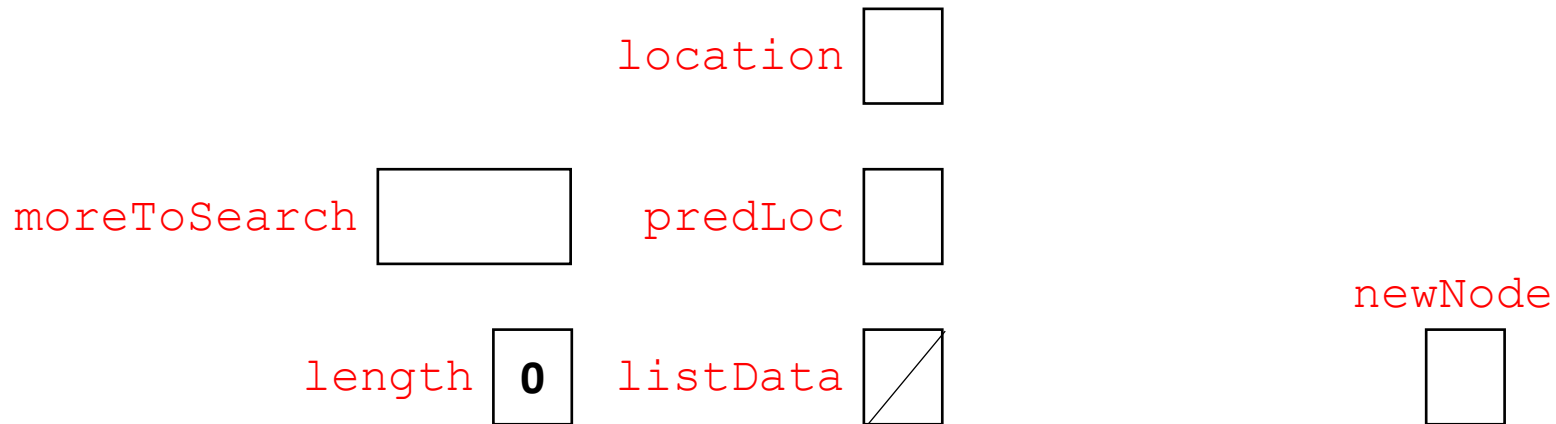
    location = listData;
    predLoc = NULL;
    moreToSearch = (location != NULL);
```

length 0 listData

InsertItem('L')

```
template <class ItemType>
void SortedType<ItemType>::InsertItem(ItemType item)
{
    NodeType* newNode;
    NodeType* predLoc;
    NodeType* location;
    bool moreToSearch;

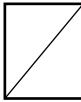
    location = listData;
    predLoc = NULL;
    moreToSearch = (location != NULL);
```

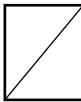


InsertItem('L')

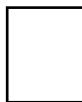
```
template <class ItemType>
void SortedType<ItemType>::InsertItem(ItemType item)
{
    NodeType* newNode;
    NodeType* predLoc;
    NodeType* location;
    bool moreToSearch;

    location = listData;
    predLoc = NULL;
    moreToSearch = (location != NULL);
```

location 

moreToSearch **false** predLoc 

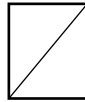
length **0** listData 

newNode 

InsertItem('L')

```
while (moreToSearch)
{
    if (location->info < item)
    {
        predLoc = location;
        location = location->next;
        moreToSearch = (location != NULL);
    }
    else moreToSearch = false;
}
```

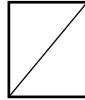
location



moreToSearch

false

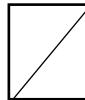
predLoc



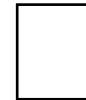
length

0

listData



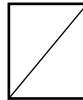
newNode



InsertItem('L')

```
while (moreToSearch)
{
    if (location->info < item)
    {
        predLoc = location;
        location = location->next;
        moreToSearch = (location != NULL);
    }
    else moreToSearch = false;
}
```

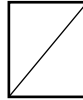
location



moreToSearch

false

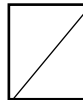
predLoc



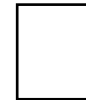
length

0

listData



newNode



InsertItem('L')

```
newNode = new NodeType;  
newNode->info = item;
```

```
if (predLoc == NULL)  
{  
    newNode->next = listData;  
    listData = newNode;  
}  
else  
{  
    newNode->next = location;  
    predLoc->next = newNode;  
}  
length++;  
}
```

location 

moreToSearch **false** predLoc 

length **0** listData 

newNode


InsertItem('L')

```
newNode = new NodeType;
```

```
newNode->info = item;
```

```
if (predLoc == NULL)
```

```
{
```

```
    newNode->next = listData;
```

```
    listData = newNode;
```

```
}
```

```
else
```

```
{
```

```
    newNode->next = location;
```

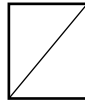
```
    predLoc->next = newNode;
```

```
}
```

```
length++;
```

```
}
```

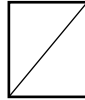
location



moreToSearch

false

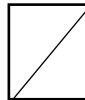
predLoc



length

0

listData



newNode



InsertItem('L')

```
newNode = new NodeType;
```

```
newNode->info = item;
```

```
if (predLoc == NULL)
```

```
{
```

```
    newNode->next = listData;
```

```
    listData = newNode;
```

```
}
```

```
else
```

```
{
```

```
    newNode->next = location;
```

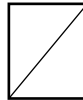
```
    predLoc->next = newNode;
```

```
}
```

```
length++;
```

```
}
```

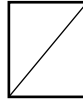
location



moreToSearch

false

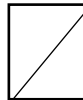
predLoc



length

0

listData



newNode



InsertItem('L')

```
newNode = new NodeType;  
newNode->info = item;
```

```
if (predLoc == NULL)
```

```
{  
    newNode->next = listData;  
    listData = newNode;  
}
```

```
else
```

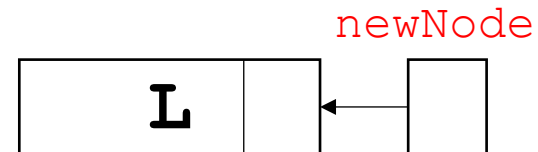
```
{  
    newNode->next = location;  
    predLoc->next = newNode;  
}
```

```
length++;
```

location 

moreToSearch **false** predLoc 

length **0** listData 



InsertItem('L')

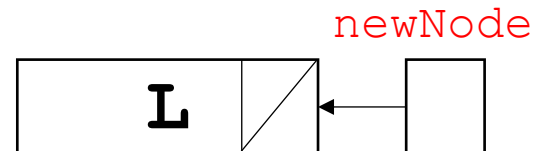
```
newNode = new NodeType;  
newNode->info = item;
```

```
if (predLoc == NULL)  
{  
    newNode->next = listData;  
    listData = newNode;  
}  
else  
{  
    newNode->next = location;  
    predLoc->next = newNode;  
}  
length++;  
}
```

location 

moreToSearch **false** predLoc 

length **0** listData 



InsertItem('L')

```
newNode = new NodeType;  
newNode->info = item;
```

```
if (predLoc == NULL)  
{  
    newNode->next = listData;  
    listData = newNode;  
}  
else  
{  
    newNode->next = location;  
    predLoc->next = newNode;  
}  
length++;  
}
```

location 

moreToSearch **false** predLoc 

length **0** listData  →

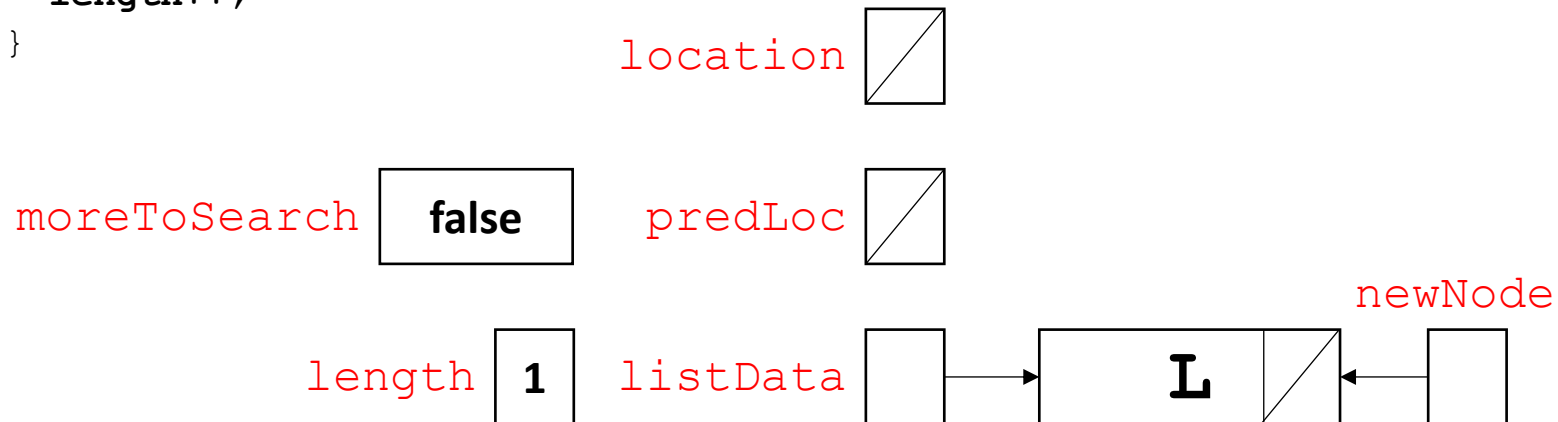
L	
----------	---

 ←  newNode

InsertItem('L')

```
newNode = new NodeType;  
newNode->info = item;
```

```
if (predLoc == NULL)  
{  
    newNode->next = listData;  
    listData = newNode;  
}  
else  
{  
    newNode->next = location;  
    predLoc->next = newNode;  
}  
length++;
```





InsertItem('A')



InsertItem('A')

```
template <class ItemType>
void SortedType<ItemType>::InsertItem(ItemType item)
{
    NodeType* newNode;
    NodeType* predLoc;
    NodeType* location;
    bool moreToSearch;

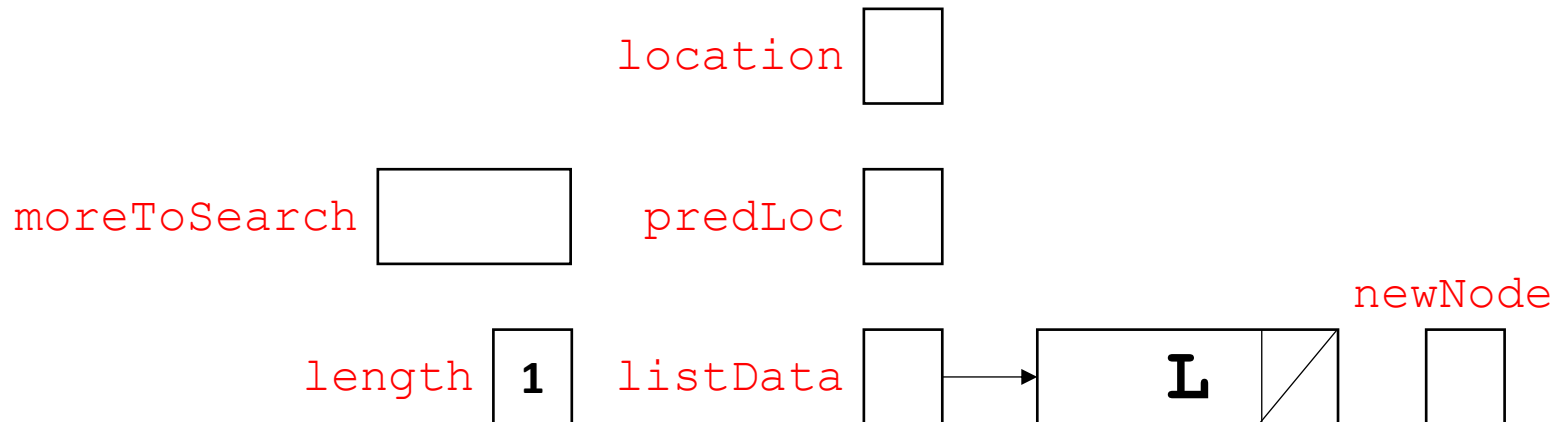
    location = listData;
    predLoc = NULL;
    moreToSearch = (location != NULL);
```



InsertItem('A')

```
template <class ItemType>
void SortedType<ItemType>::InsertItem(ItemType item)
{
    NodeType* newNode;
    NodeType* predLoc;
    NodeType* location;
    bool moreToSearch;

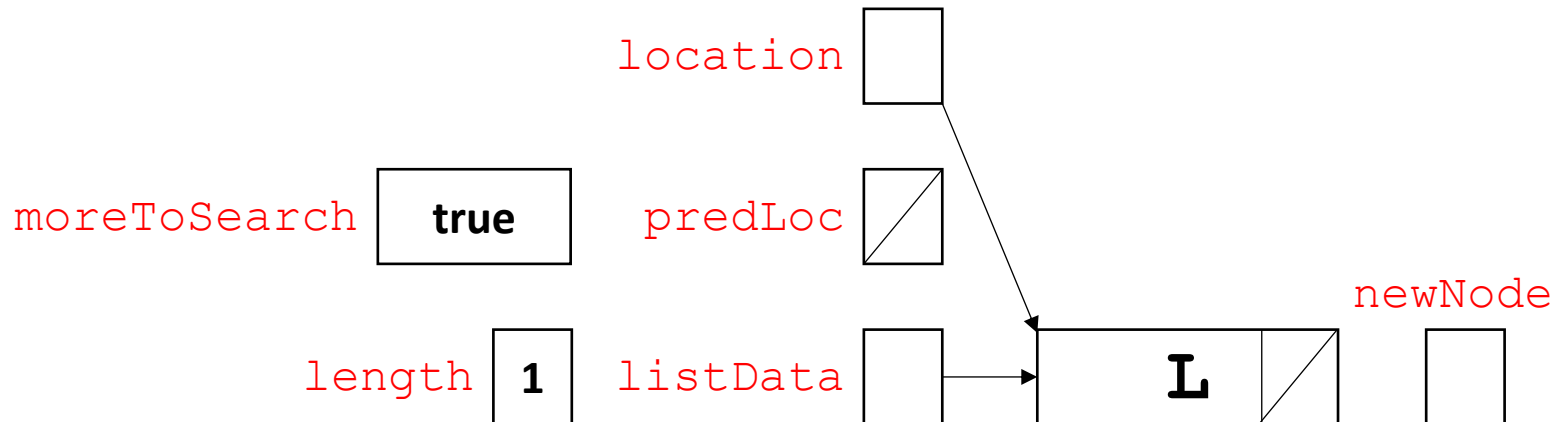
    location = listData;
    predLoc = NULL;
    moreToSearch = (location != NULL);
```



InsertItem('A')

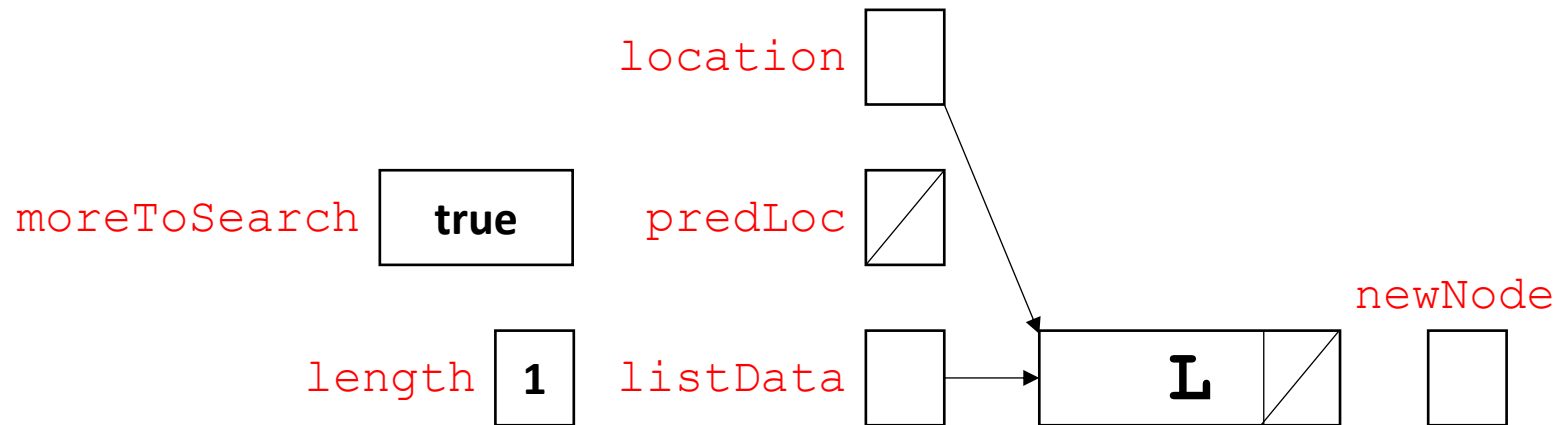
```
template <class ItemType>
void SortedType<ItemType>::InsertItem(ItemType item)
{
    NodeType* newNode;
    NodeType* predLoc;
    NodeType* location;
    bool moreToSearch;

    location = listData;
    predLoc = NULL;
    moreToSearch = (location != NULL);
```



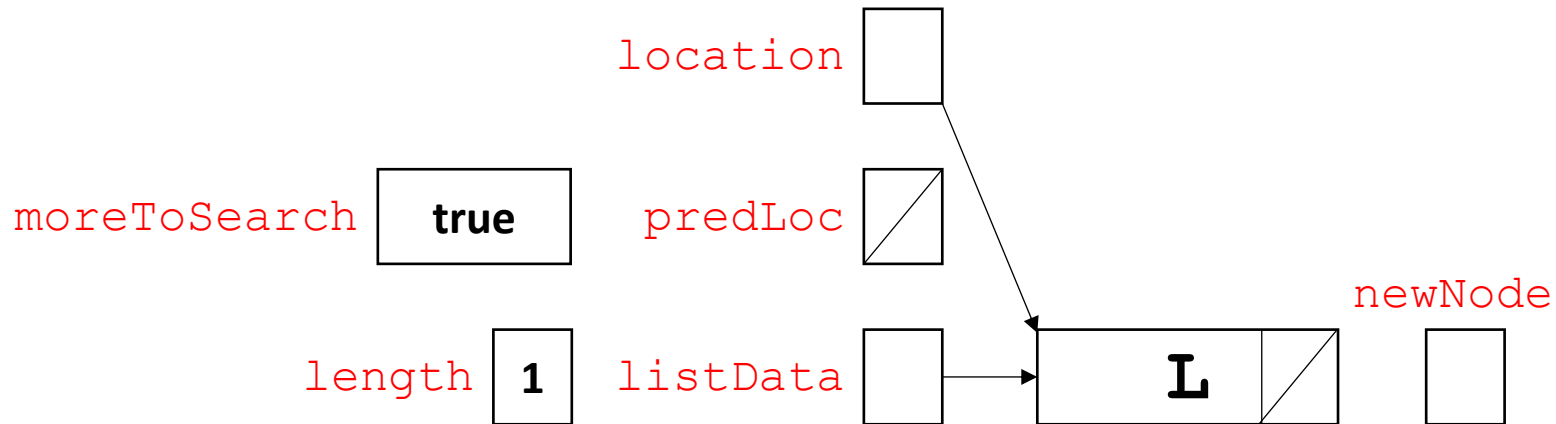
InsertItem('A')

```
while (moreToSearch)
{
    if (location->info < item)
    {
        predLoc = location;
        location = location->next;
        moreToSearch = (location != NULL);
    }
    else moreToSearch = false;
}
```



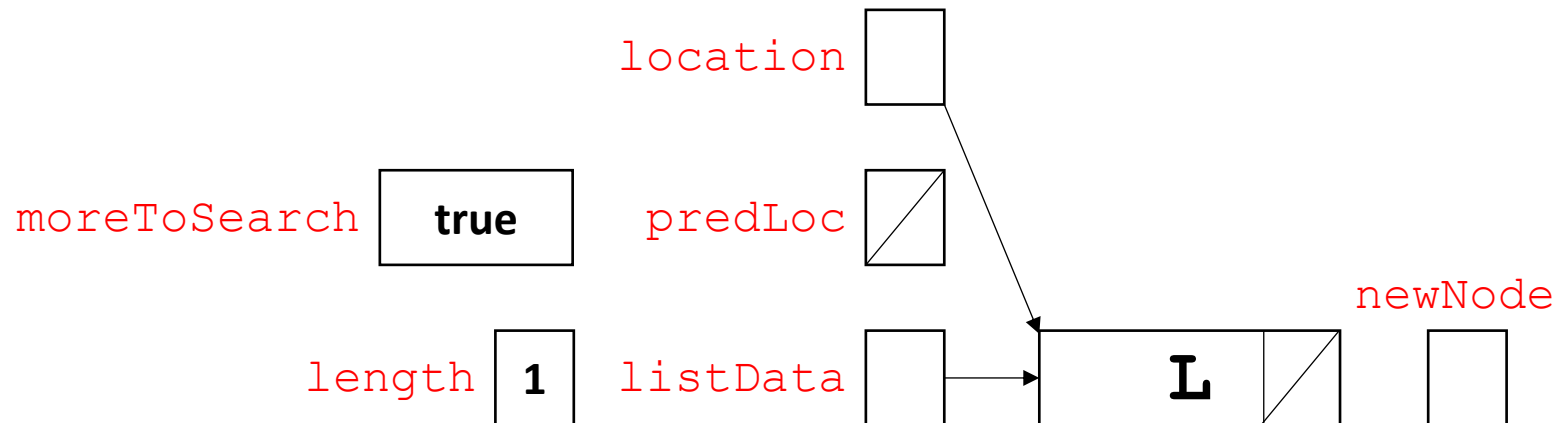
InsertItem('A')

```
while (moreToSearch)
{
    if (location->info < item)
    {
        predLoc = location;
        location = location->next;
        moreToSearch = (location != NULL);
    }
    else moreToSearch = false;
}
```



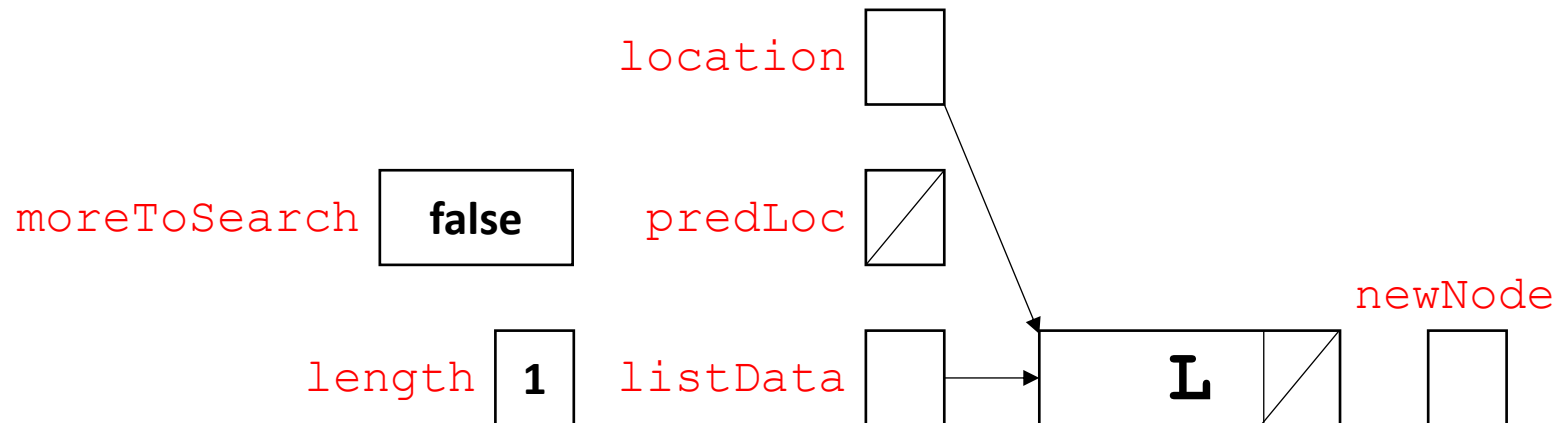
InsertItem('A')

```
while (moreToSearch)
{
    if (location->info < item)
    {
        predLoc = location;
        location = location->next;
        moreToSearch = (location != NULL);
    }
    else moreToSearch = false;
}
```



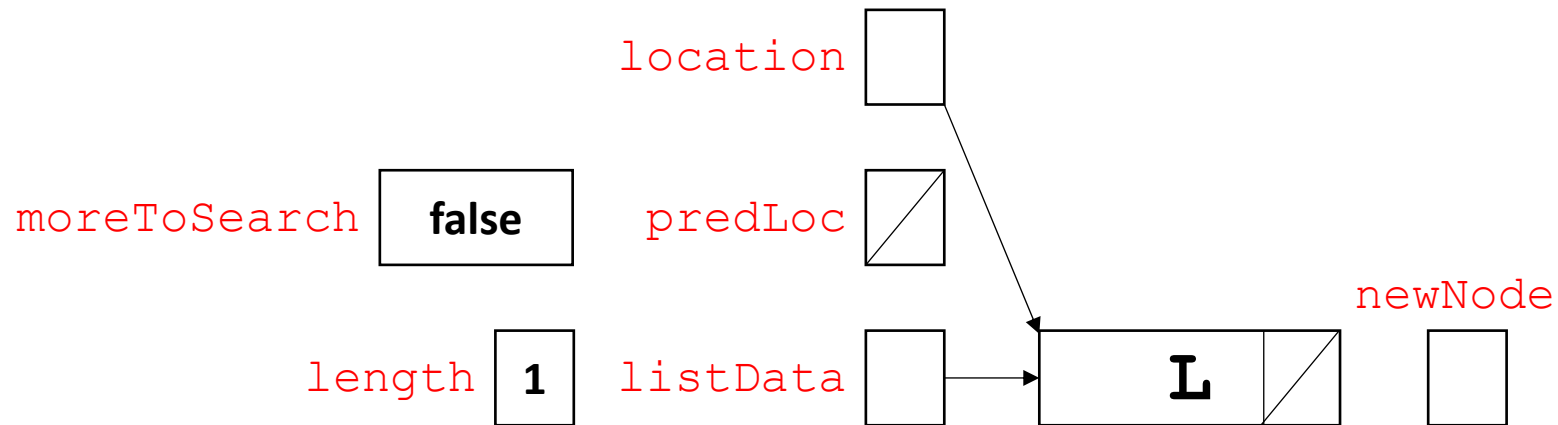
InsertItem('A')

```
while (moreToSearch)
{
    if (location->info < item)
    {
        predLoc = location;
        location = location->next;
        moreToSearch = (location != NULL);
    }
    else moreToSearch = false;
}
```



InsertItem('A')

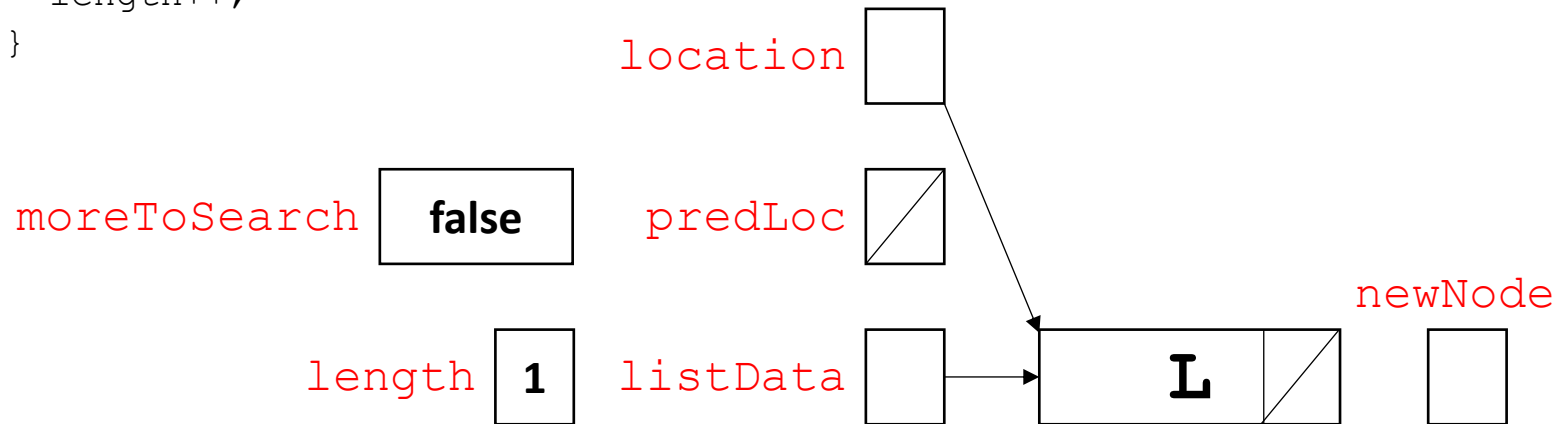
```
while (moreToSearch)
{
    if (location->info < item)
    {
        predLoc = location;
        location = location->next;
        moreToSearch = (location != NULL);
    }
    else moreToSearch = false;
}
```



InsertItem('A')

```
newNode = new NodeType;  
newNode->info = item;
```

```
if (predLoc == NULL)  
{  
    newNode->next = listData;  
    listData = newNode;  
}  
else  
{  
    newNode->next = location;  
    predLoc->next = newNode;  
}  
length++;
```



InsertItem('A')

```
newNode = new NodeType;
```

```
newNode->info = item;
```

```
if (predLoc == NULL)
```

```
{
```

```
    newNode->next = listData;
```

```
    listData = newNode;
```

```
}
```

```
else
```

```
{
```

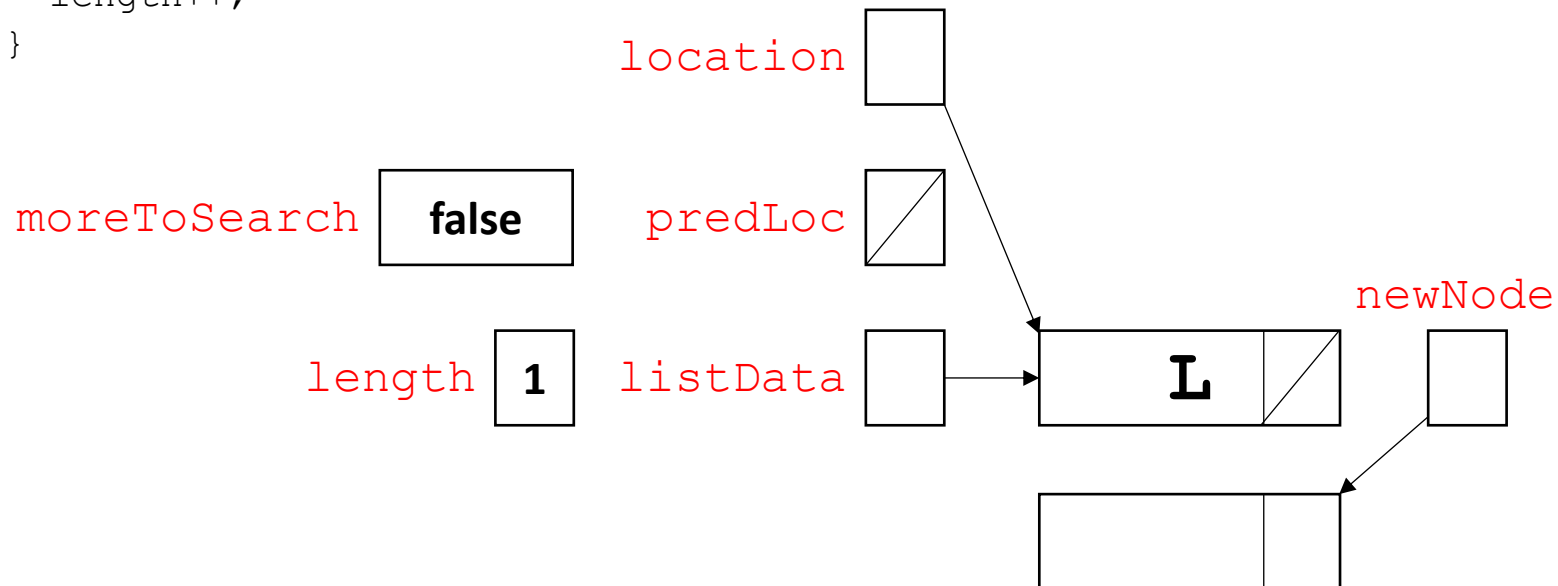
```
    newNode->next = location;
```

```
    predLoc->next = newNode;
```

```
}
```

```
length++;
```

```
}
```



InsertItem('A')

```
newNode = new NodeType;
```

```
newNode->info = item;
```

```
if (predLoc == NULL)
```

```
{
```

```
    newNode->next = listData;
```

```
    listData = newNode;
```

```
}
```

```
else
```

```
{
```

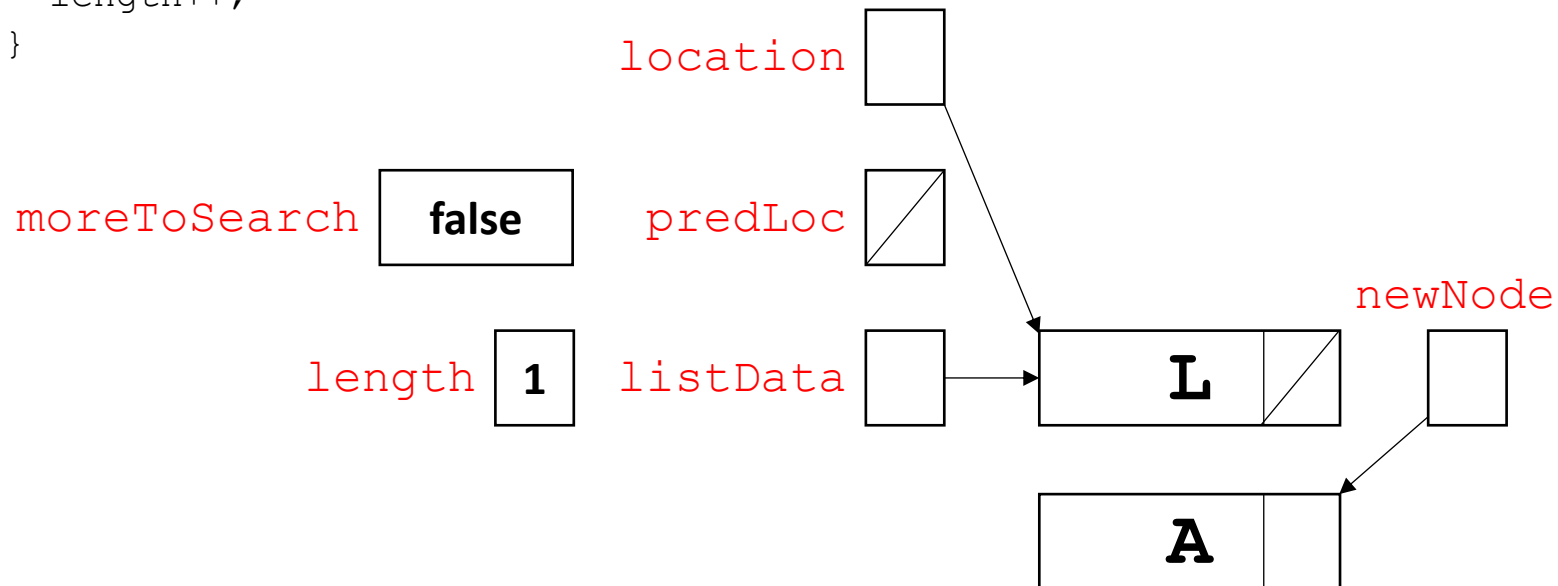
```
    newNode->next = location;
```

```
    predLoc->next = newNode;
```

```
}
```

```
length++;
```

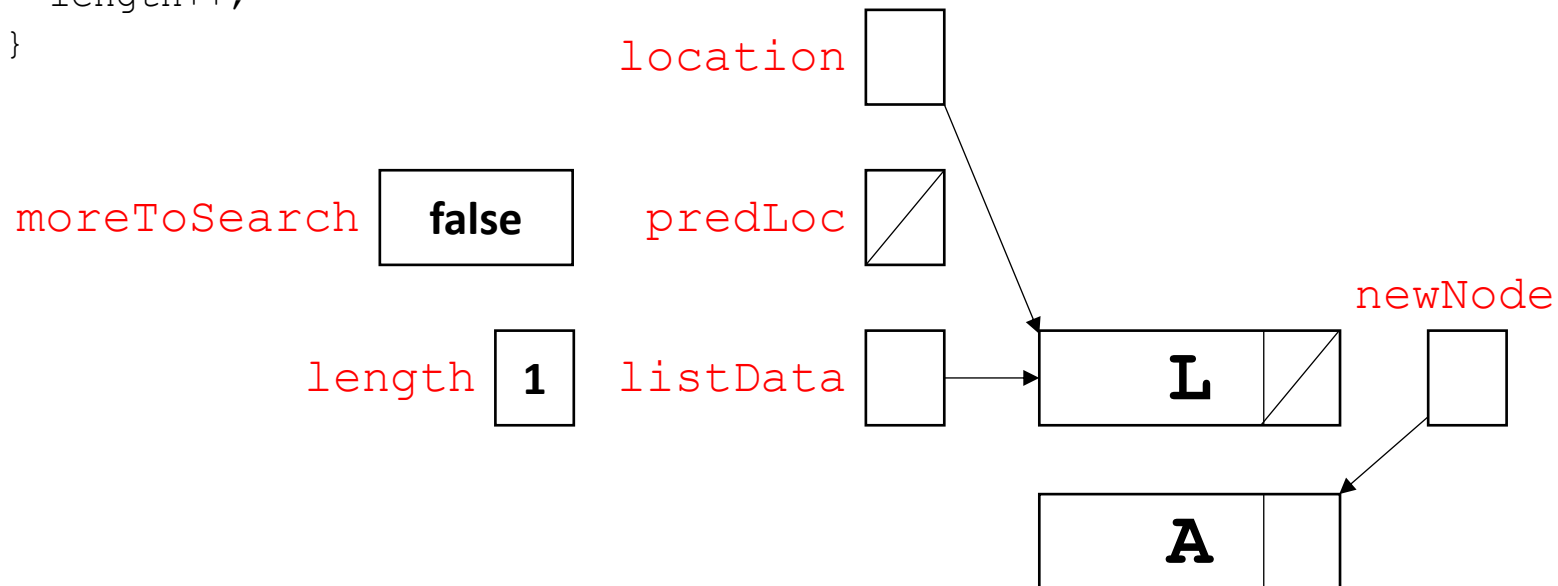
```
}
```



InsertItem('A')

```
newNode = new NodeType;  
newNode->info = item;
```

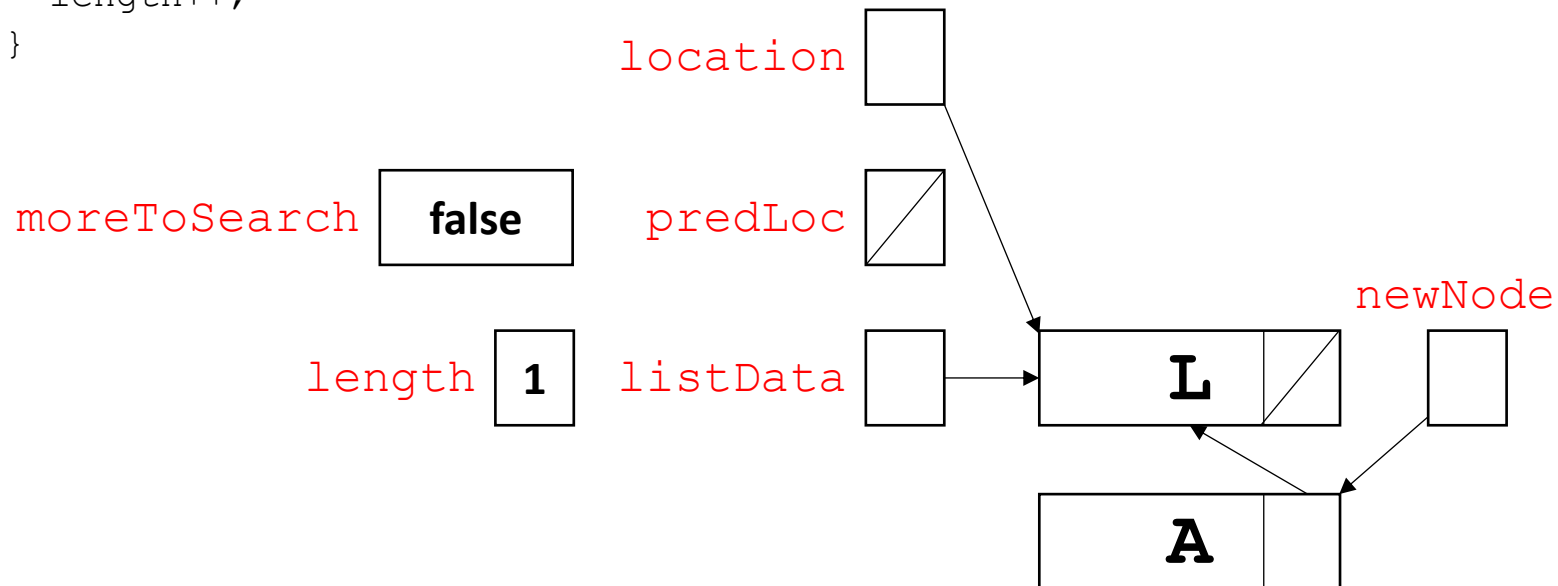
```
if (predLoc == NULL)  
{  
    newNode->next = listData;  
    listData = newNode;  
}  
else  
{  
    newNode->next = location;  
    predLoc->next = newNode;  
}  
length++;  
}
```



InsertItem('A')

```
newNode = new NodeType;  
newNode->info = item;
```

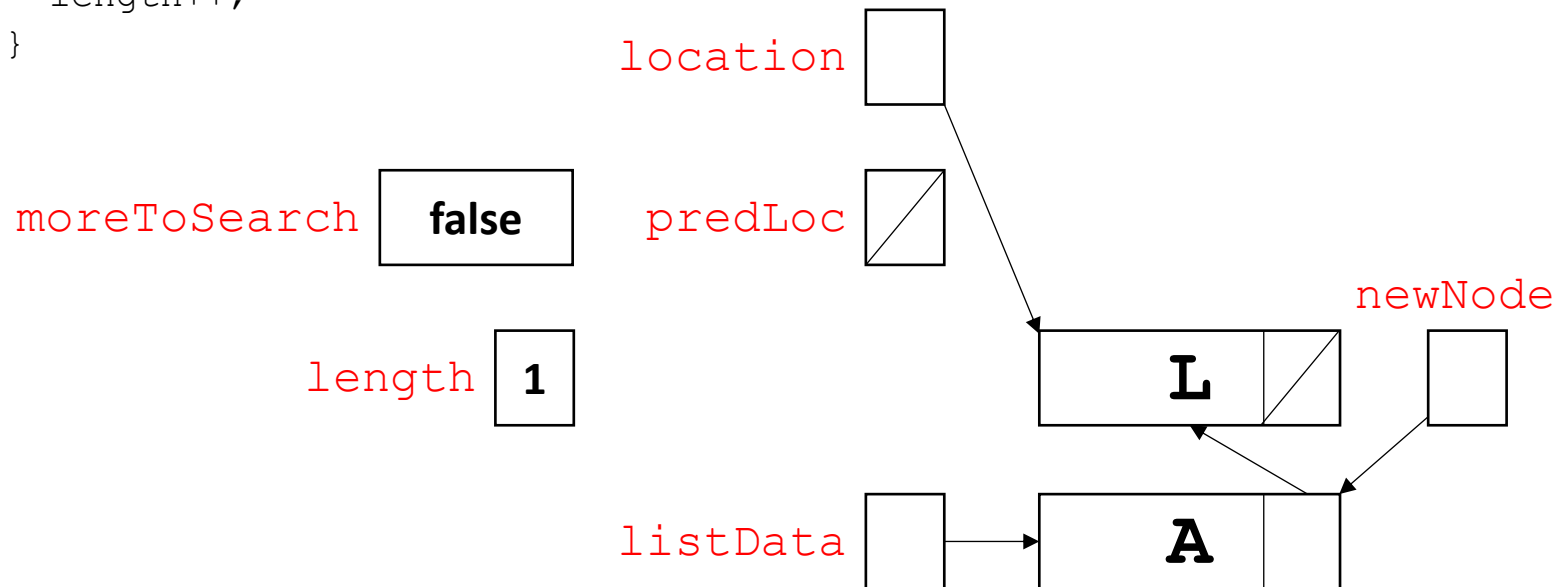
```
if (predLoc == NULL)  
{  
    newNode->next = listData;  
    listData = newNode;  
}  
else  
{  
    newNode->next = location;  
    predLoc->next = newNode;  
}  
length++;  
}
```



InsertItem('A')

```
newNode = new NodeType;  
newNode->info = item;
```

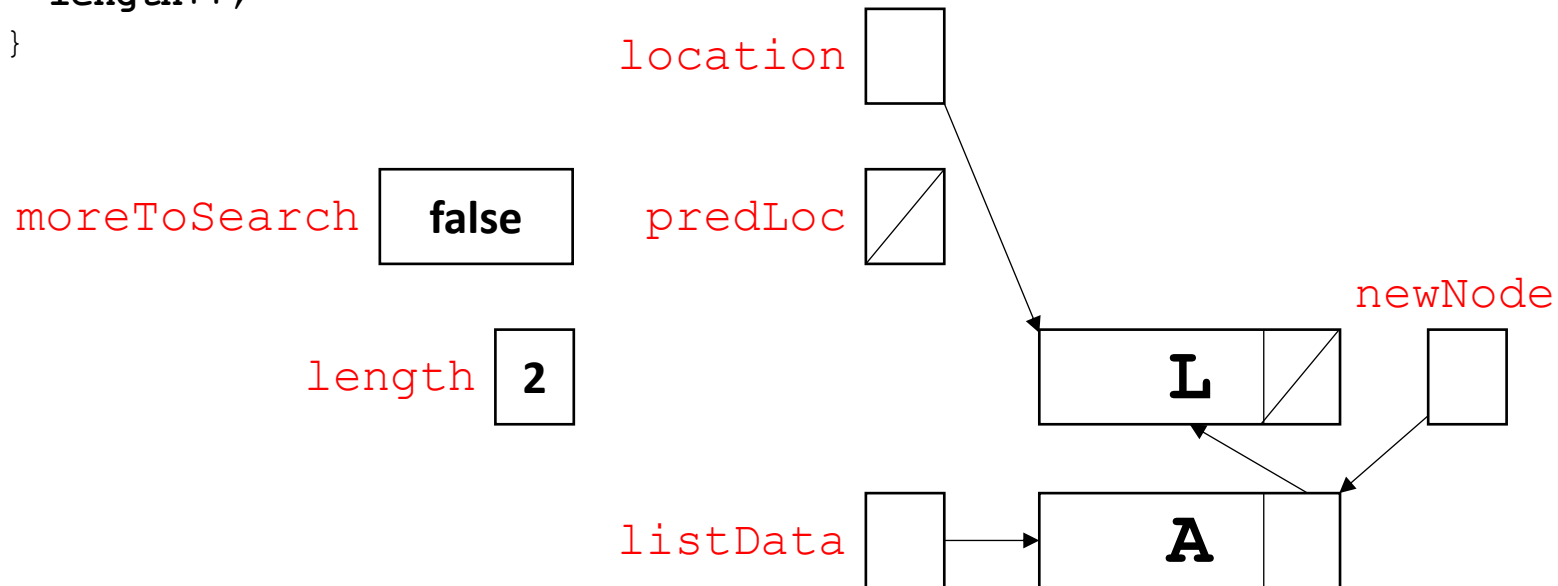
```
if (predLoc == NULL)  
{  
    newNode->next = listData;  
    listData = newNode;  
}  
else  
{  
    newNode->next = location;  
    predLoc->next = newNode;  
}  
length++;  
}
```



InsertItem('A')

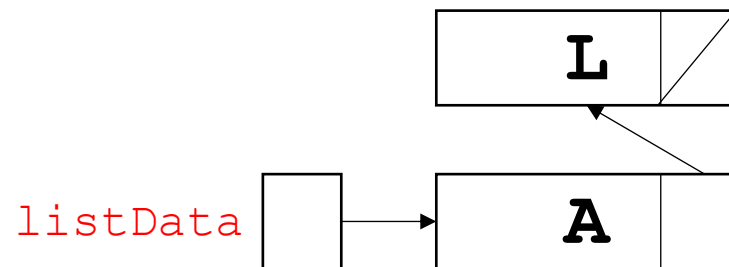
```
newNode = new NodeType;  
newNode->info = item;
```

```
if (predLoc == NULL)  
{  
    newNode->next = listData;  
    listData = newNode;  
}  
else  
{  
    newNode->next = location;  
    predLoc->next = newNode;  
}  
length++;
```



length

2



InsertItem('S')



InsertItem('S')

```
template <class ItemType>
void SortedType<ItemType>::InsertItem(ItemType item)
{
    NodeType* newNode;
    NodeType* predLoc;
    NodeType* location;
    bool moreToSearch;

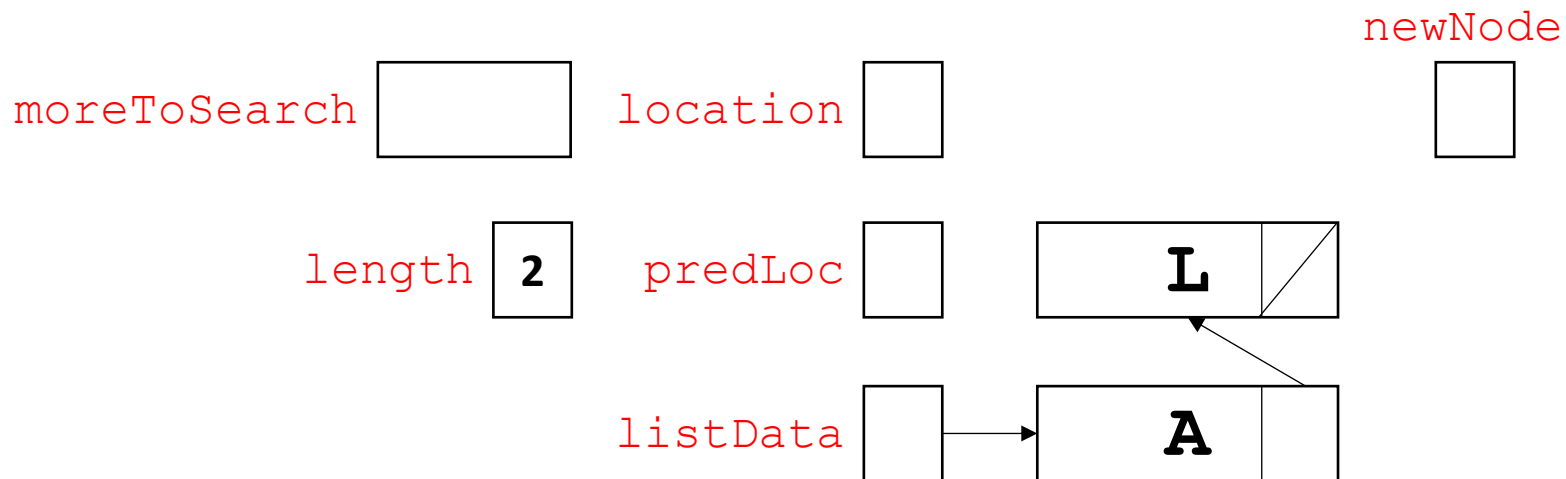
    location = listData;
    predLoc = NULL;
    moreToSearch = (location != NULL);
```



InsertItem('S')

```
template <class ItemType>
void SortedType<ItemType>::InsertItem(ItemType item)
{
    NodeType* newNode;
    NodeType* predLoc;
    NodeType* location;
    bool moreToSearch;

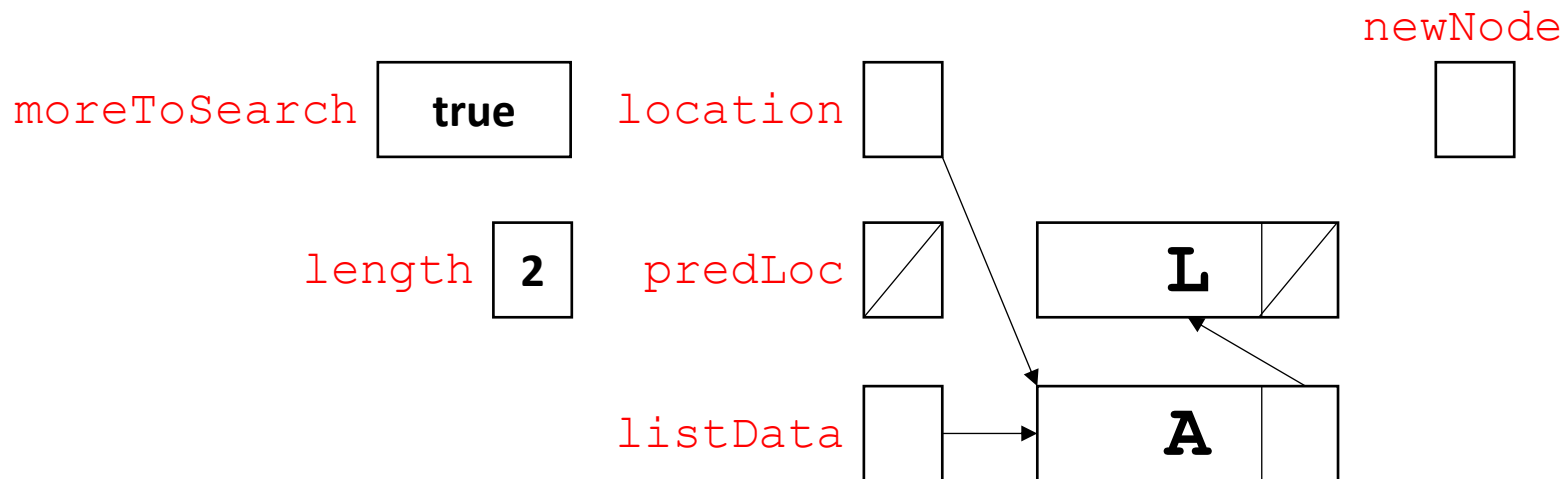
    location = listData;
    predLoc = NULL;
    moreToSearch = (location != NULL);
```



InsertItem('S')

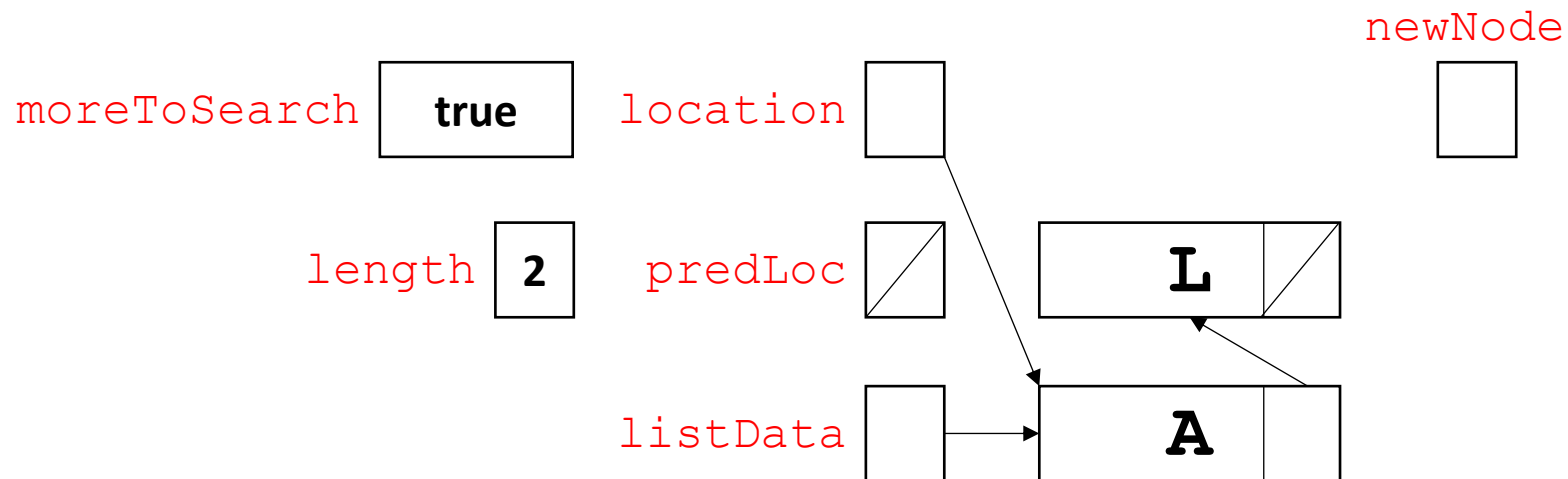
```
template <class ItemType>
void SortedType<ItemType>::InsertItem(ItemType item)
{
    NodeType* newNode;
    NodeType* predLoc;
    NodeType* location;
    bool moreToSearch;

    location = listData;
    predLoc = NULL;
    moreToSearch = (location != NULL);
```



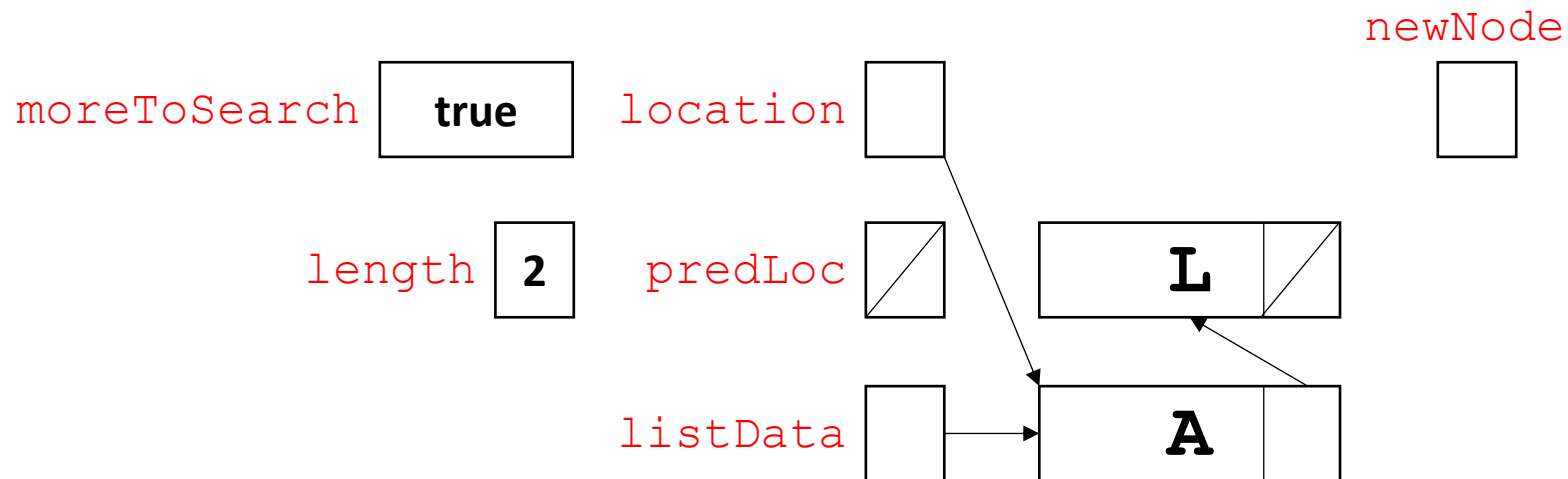
InsertItem('S')

```
while (moreToSearch)
{
    if (location->info < item)
    {
        predLoc = location;
        location = location->next;
        moreToSearch = (location != NULL);
    }
    else moreToSearch = false;
}
```



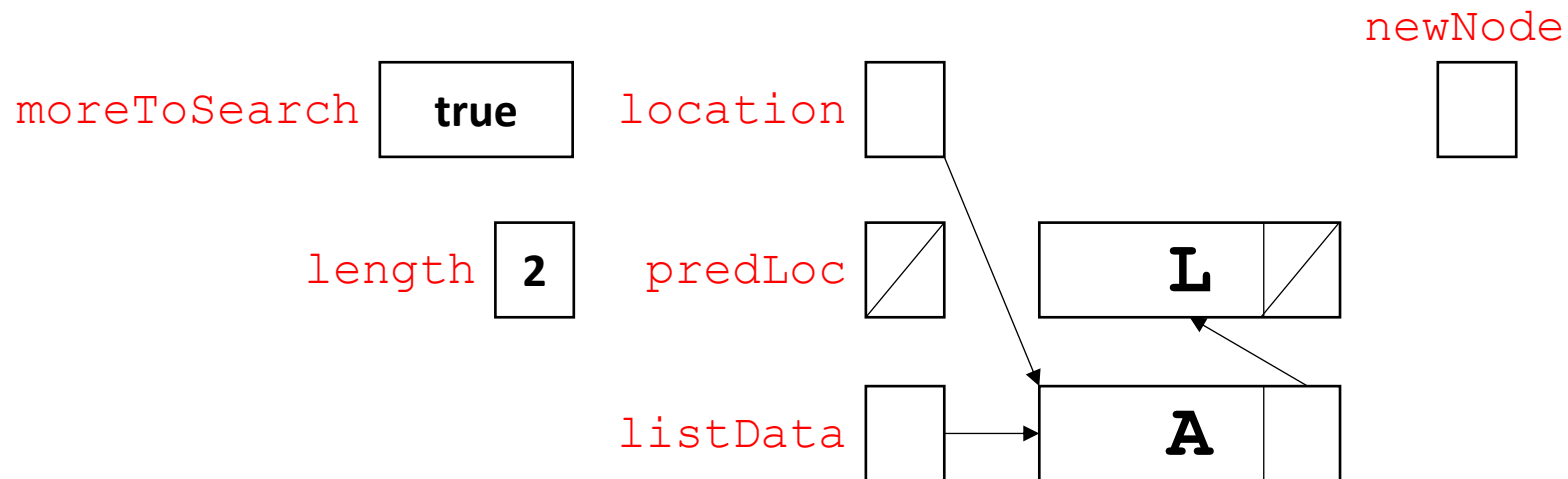
InsertItem('S')

```
while (moreToSearch)
{
    if (location->info < item)
    {
        predLoc = location;
        location = location->next;
        moreToSearch = (location != NULL);
    }
    else moreToSearch = false;
}
```



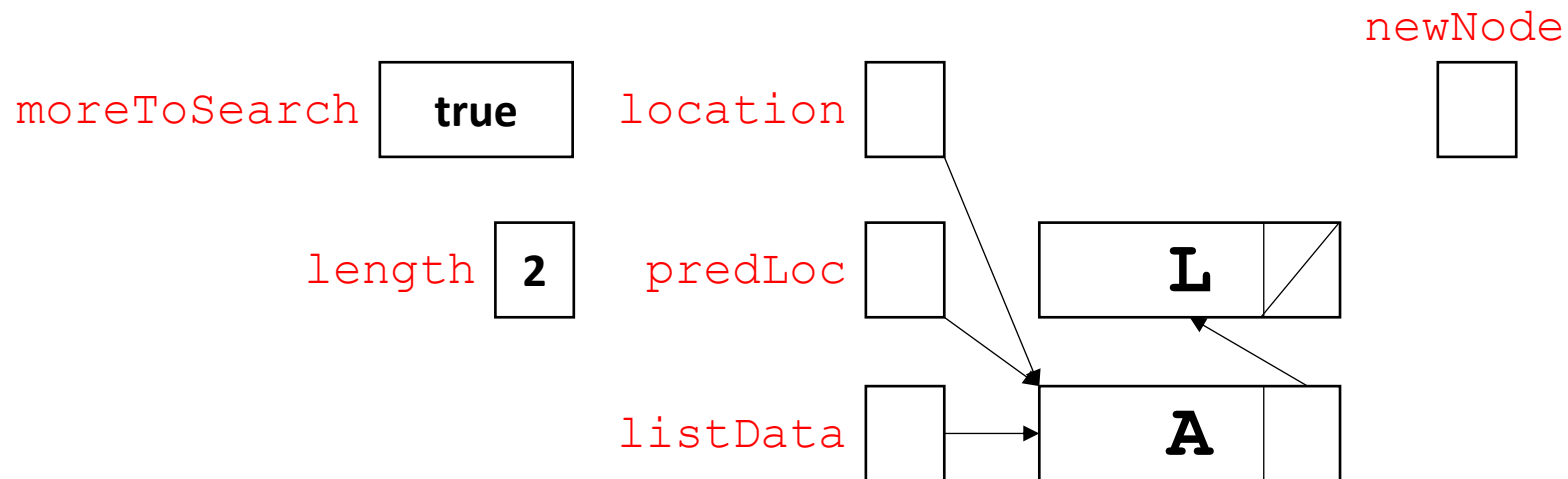
InsertItem('S')

```
while (moreToSearch)
{
    if (location->info < item)
    {
        predLoc = location;
        location = location->next;
        moreToSearch = (location != NULL);
    }
    else moreToSearch = false;
}
```



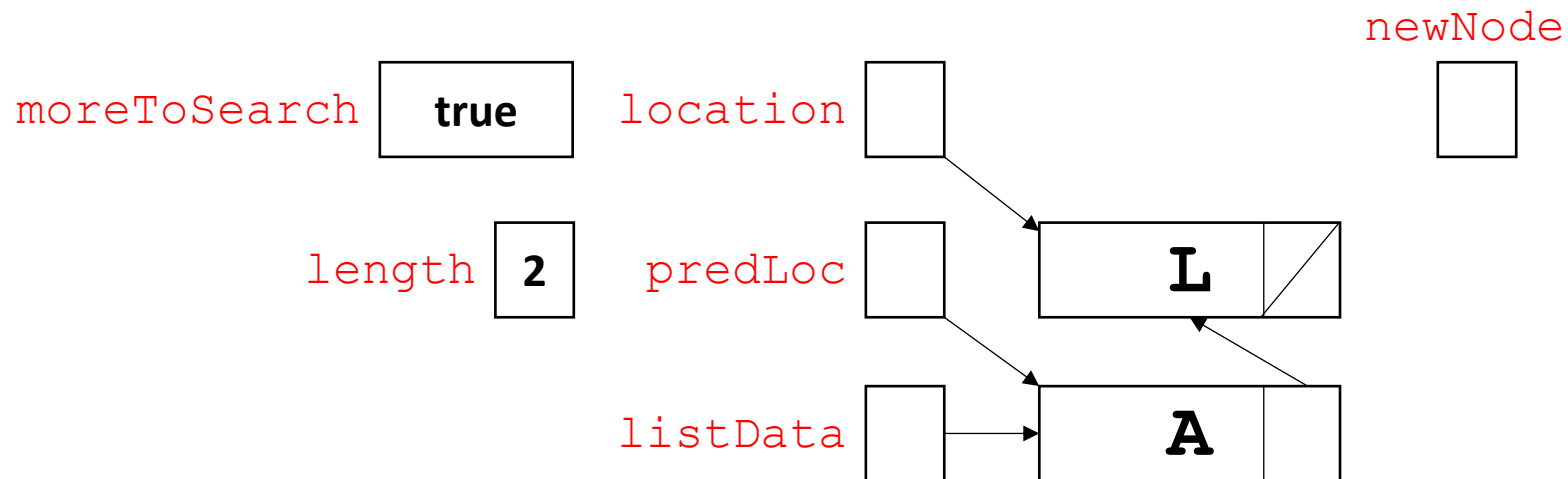
InsertItem('S')

```
while (moreToSearch)
{
    if (location->info < item)
    {
        predLoc = location;
        location = location->next;
        moreToSearch = (location != NULL);
    }
    else moreToSearch = false;
}
```



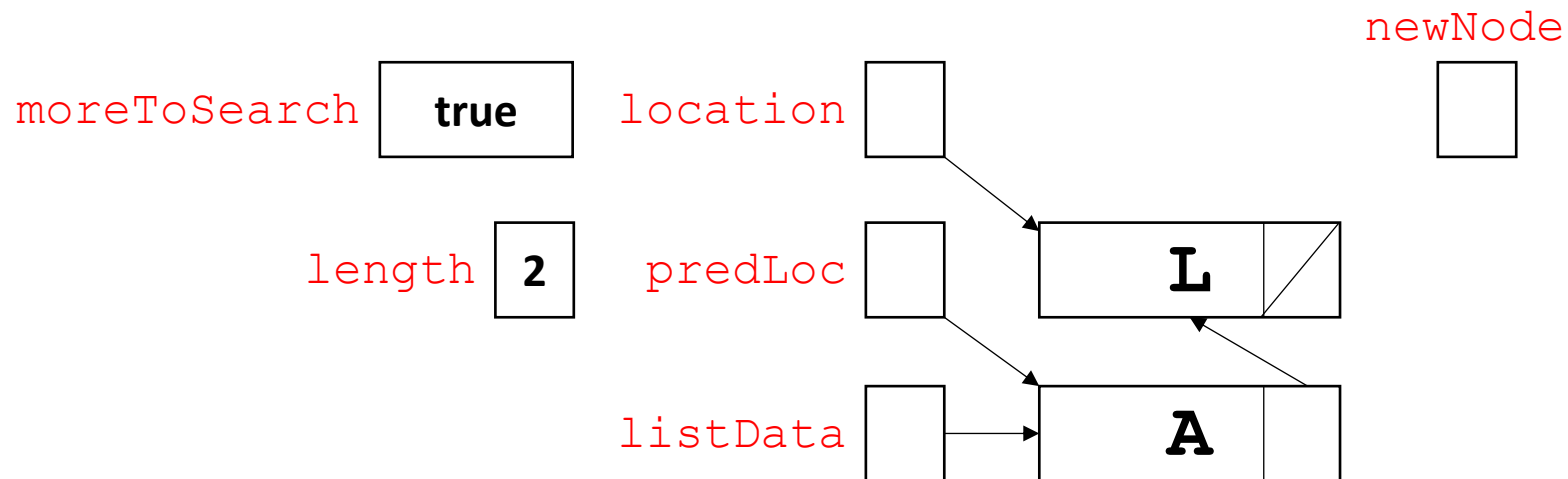
InsertItem('S')

```
while (moreToSearch)
{
    if (location->info < item)
    {
        predLoc = location;
        location = location->next;
        moreToSearch = (location != NULL);
    }
    else moreToSearch = false;
}
```



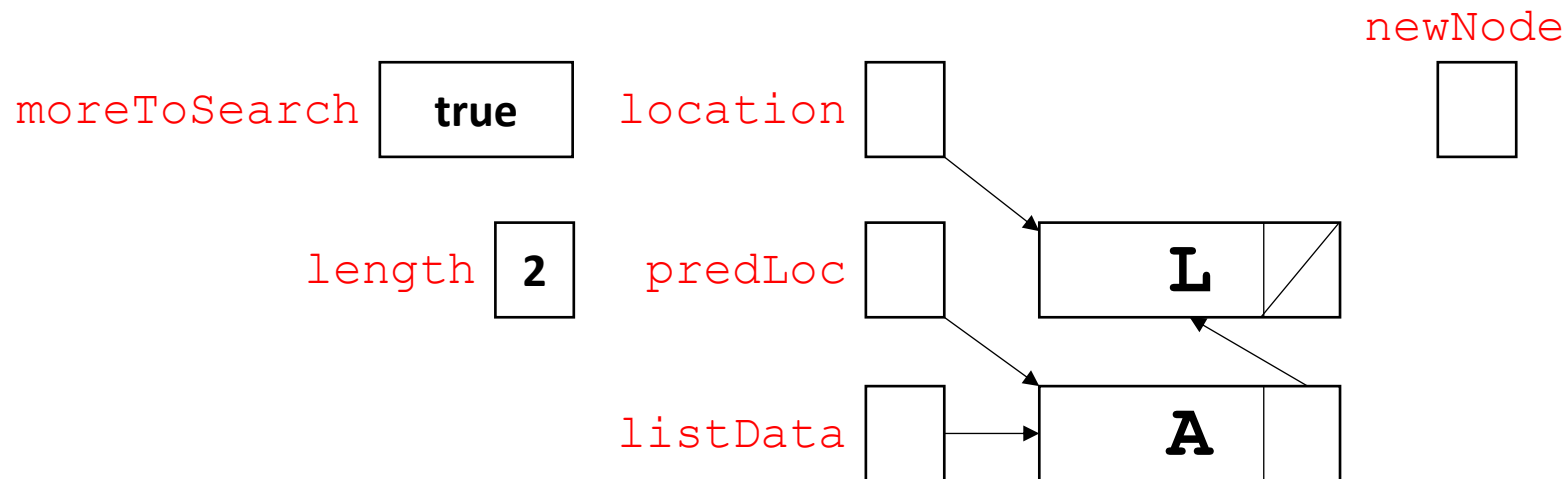
InsertItem('S')

```
while (moreToSearch)
{
    if (location->info < item)
    {
        predLoc = location;
        location = location->next;
        moreToSearch = (location != NULL);
    }
    else moreToSearch = false;
}
```



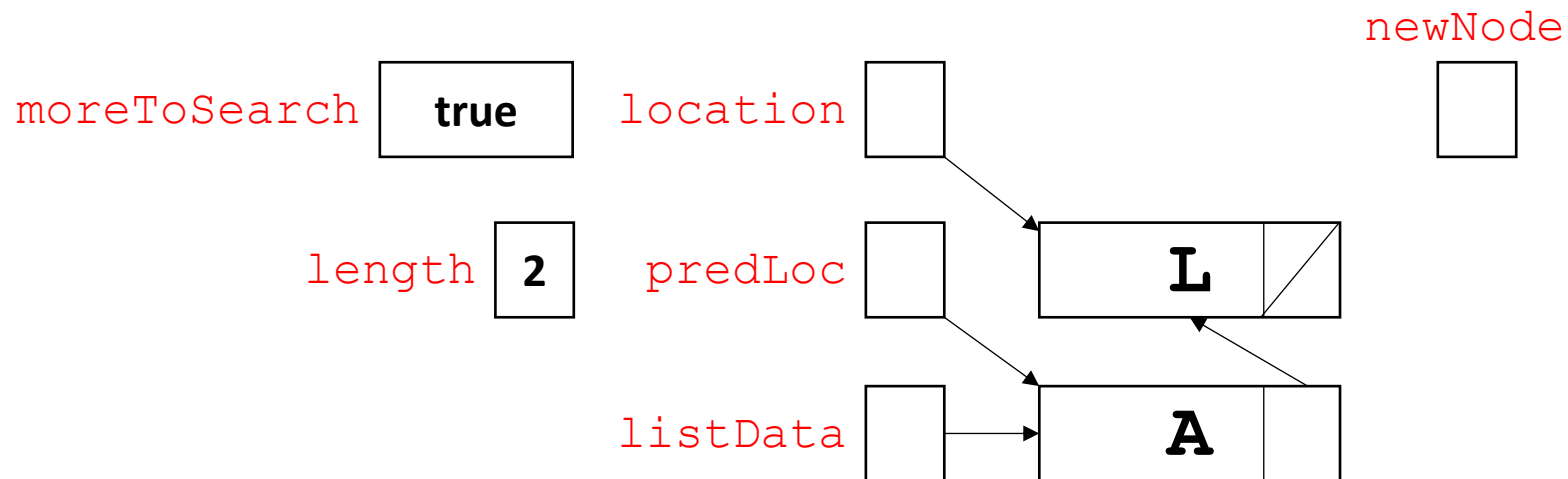
InsertItem('S')

```
while (moreToSearch)
{
    if (location->info < item)
    {
        predLoc = location;
        location = location->next;
        moreToSearch = (location != NULL);
    }
    else moreToSearch = false;
}
```



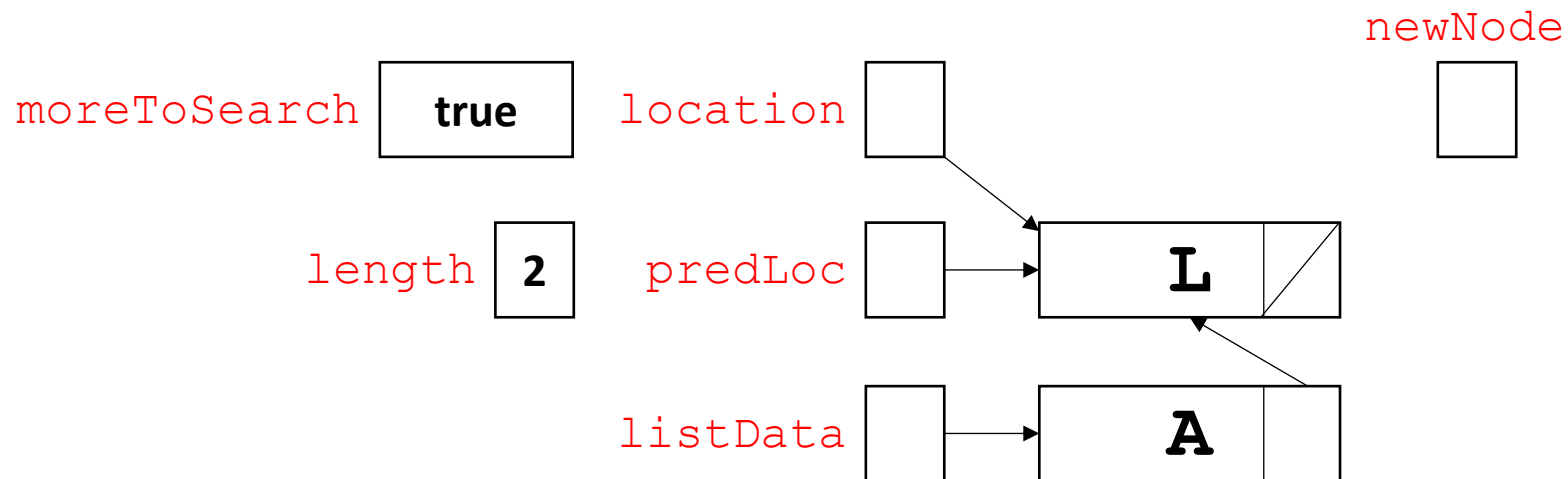
InsertItem('S')

```
while (moreToSearch)
{
    if (location->info < item)
    {
        predLoc = location;
        location = location->next;
        moreToSearch = (location != NULL);
    }
    else moreToSearch = false;
}
```



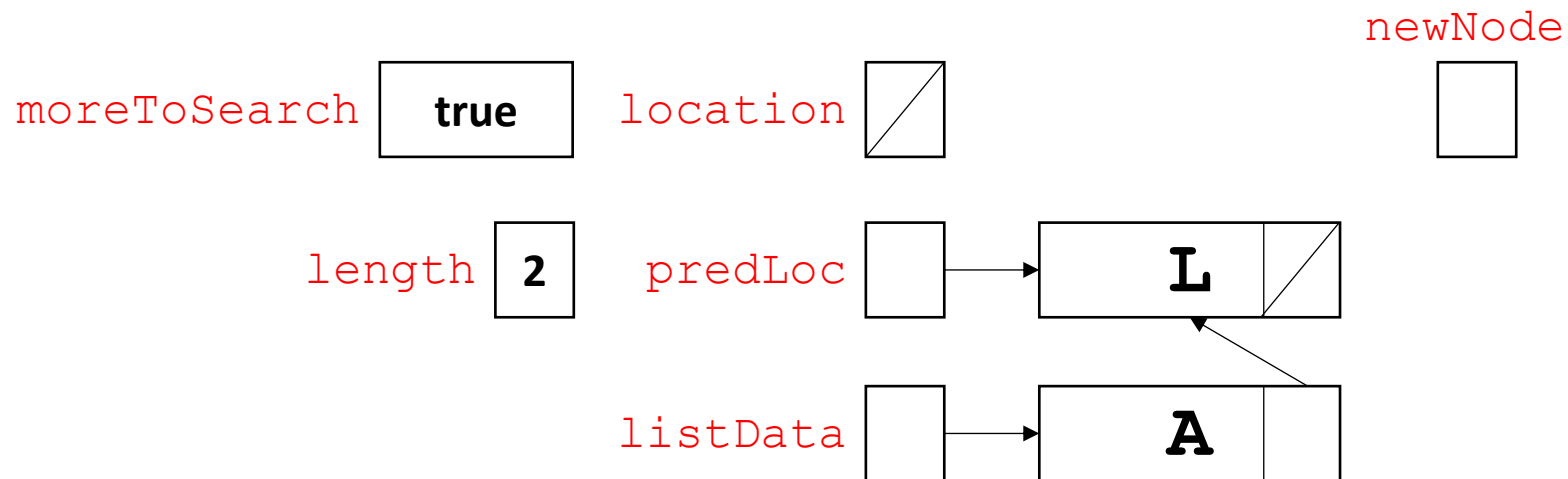
InsertItem('S')

```
while (moreToSearch)
{
    if (location->info < item)
    {
        predLoc = location;
        location = location->next;
        moreToSearch = (location != NULL);
    }
    else moreToSearch = false;
}
```



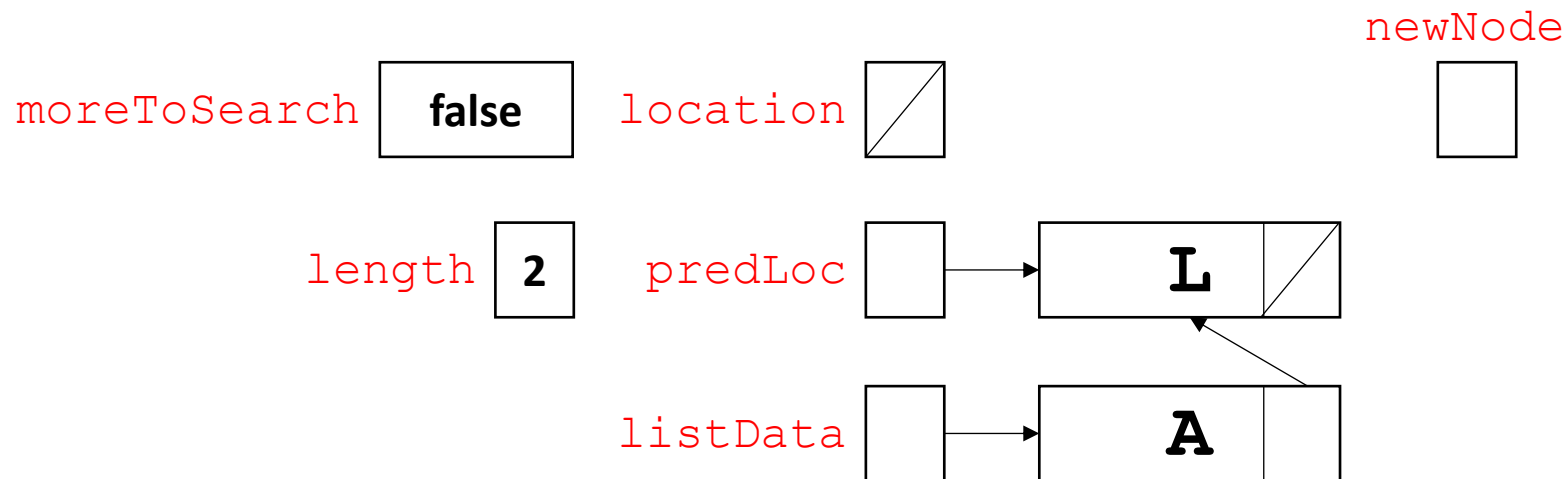
InsertItem('S')

```
while (moreToSearch)
{
    if (location->info < item)
    {
        predLoc = location;
        location = location->next;
        moreToSearch = (location != NULL);
    }
    else moreToSearch = false;
}
```



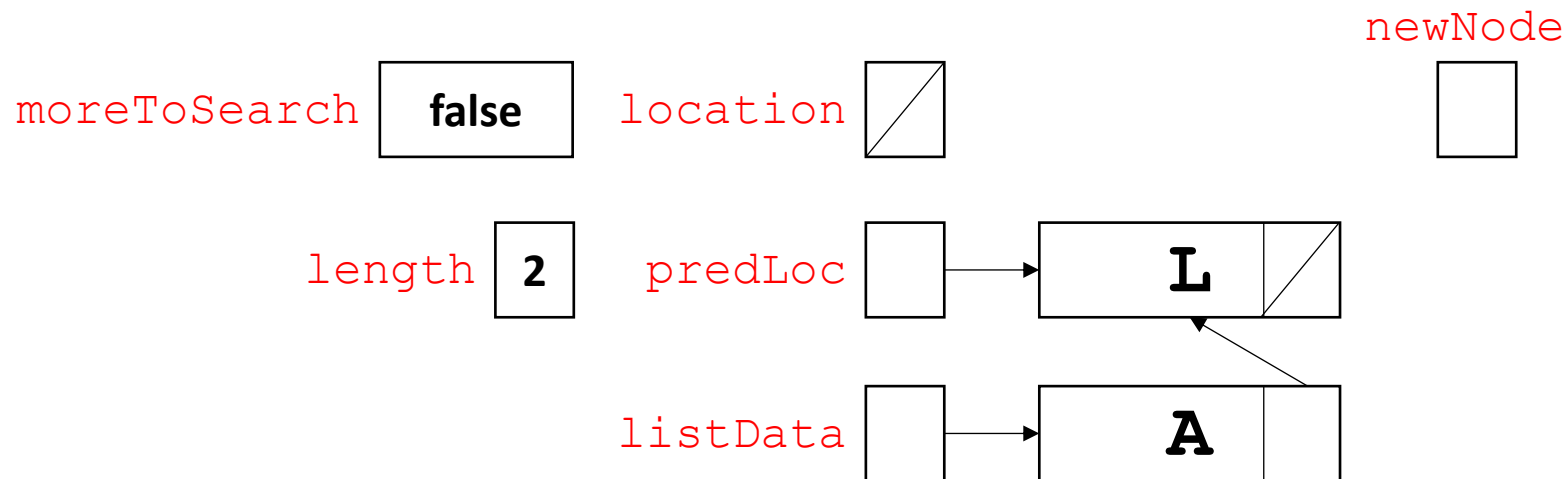
InsertItem('S')

```
while (moreToSearch)
{
    if (location->info < item)
    {
        predLoc = location;
        location = location->next;
        moreToSearch = (location != NULL);
    }
    else moreToSearch = false;
}
```



InsertItem('S')

```
while (moreToSearch)
{
    if (location->info < item)
    {
        predLoc = location;
        location = location->next;
        moreToSearch = (location != NULL);
    }
    else moreToSearch = false;
}
```



InsertItem('S')

```
newNode = new NodeType;
```

```
newNode->info = item;
```

```
if (predLoc == NULL)
```

```
{
```

```
    newNode->next = listData;
```

```
    listData = newNode;
```

```
}
```

```
else
```

```
{
```

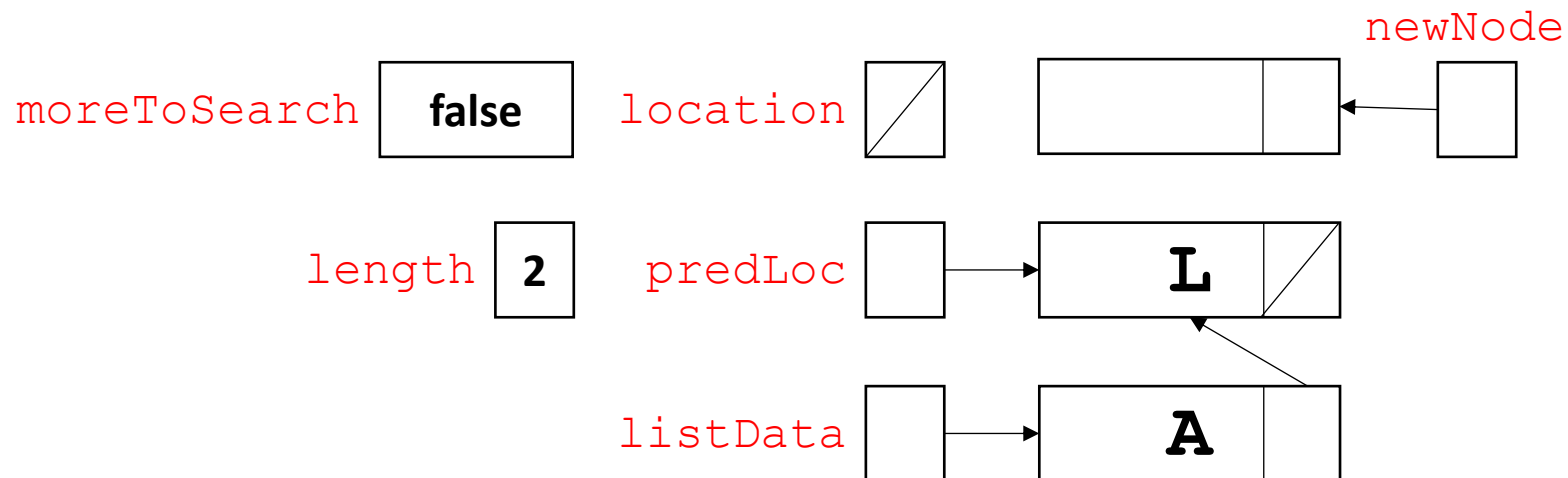
```
    newNode->next = location;
```

```
    predLoc->next = newNode;
```

```
}
```

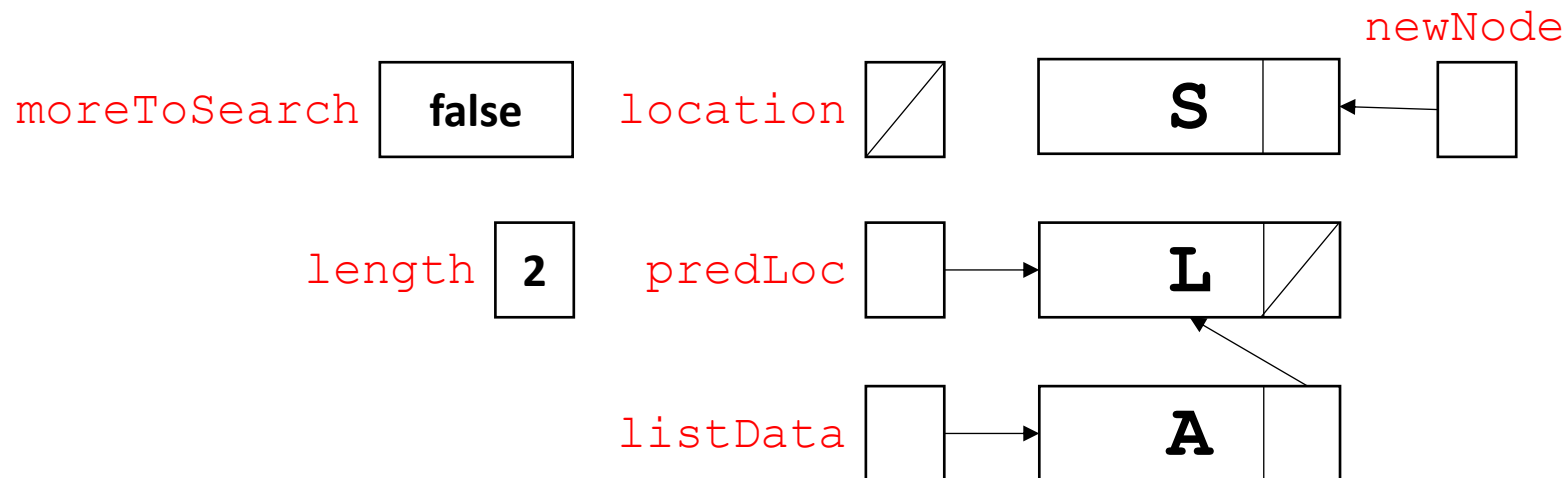
```
length++;
```

```
}
```



InsertItem('S')

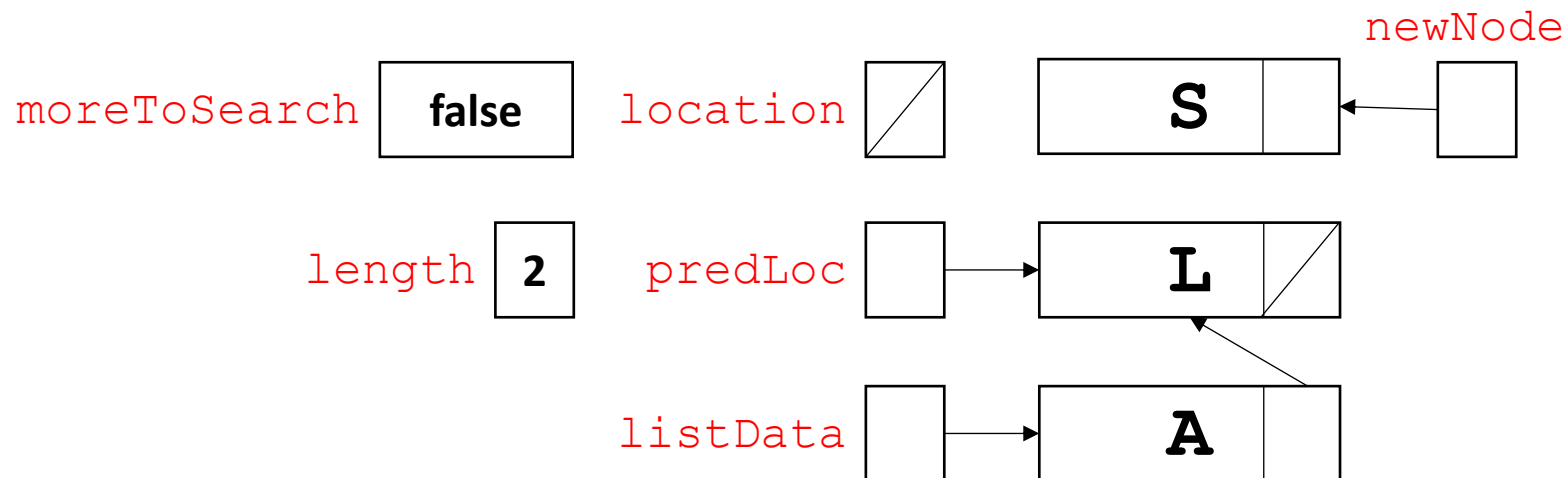
```
newNode = new NodeType;  
newNode->info = item;  
  
if (predLoc == NULL)  
{  
    newNode->next = listData;  
    listData = newNode;  
}  
else  
{  
    newNode->next = location;  
    predLoc->next = newNode;  
}  
length++;  
}
```



InsertItem('S')

```
newNode = new NodeType;  
newNode->info = item;
```

```
if (predLoc == NULL)  
{  
    newNode->next = listData;  
    listData = newNode;  
}  
else  
{  
    newNode->next = location;  
    predLoc->next = newNode;  
}  
length++;  
}
```



InsertItem('S')

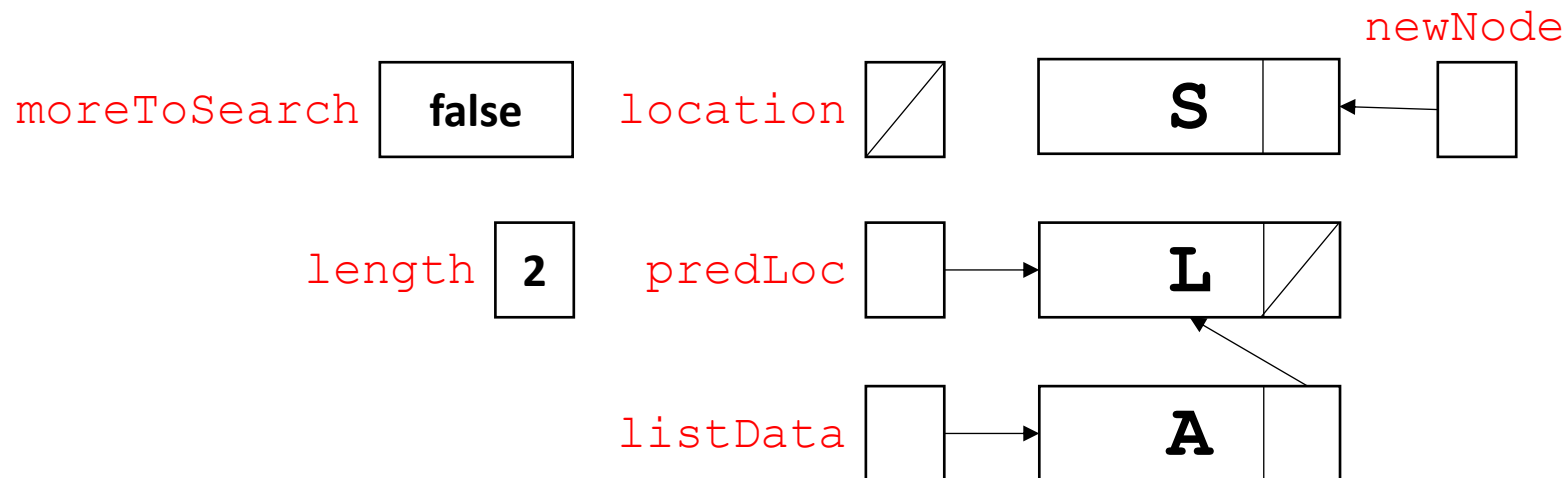
```
newNode = new NodeType;  
newNode->info = item;
```

```
if (predLoc == NULL)  
{  
    newNode->next = listData;  
    listData = newNode;  
}
```

else

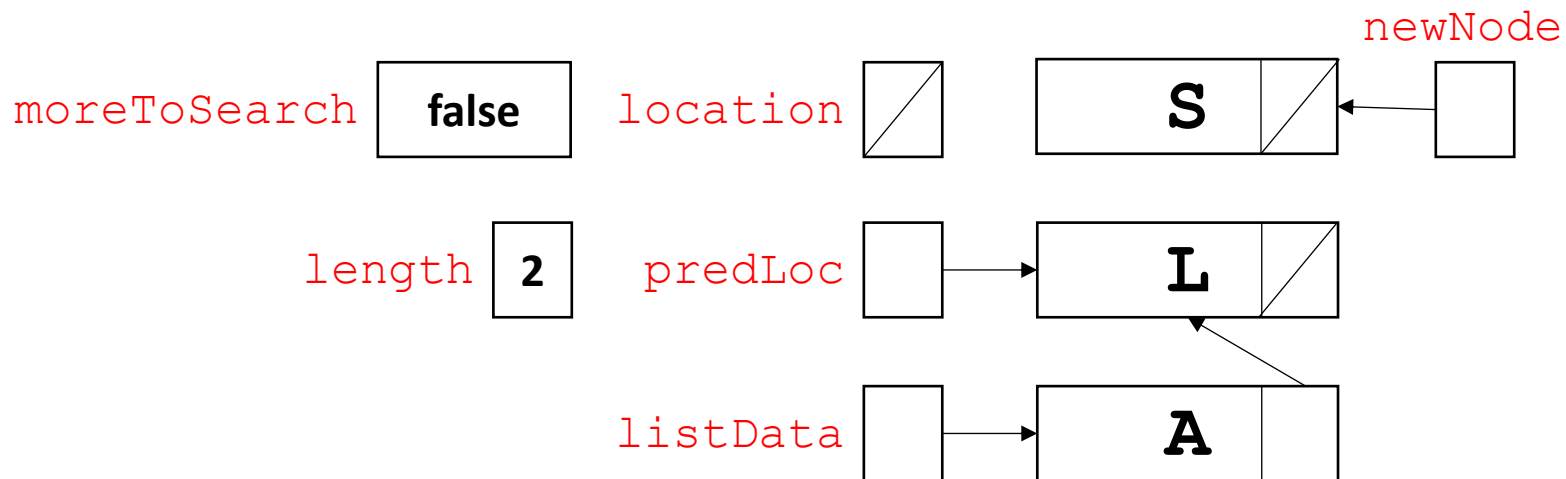
```
{  
    newNode->next = location;  
    predLoc->next = newNode;  
}  
length++;
```

```
}
```



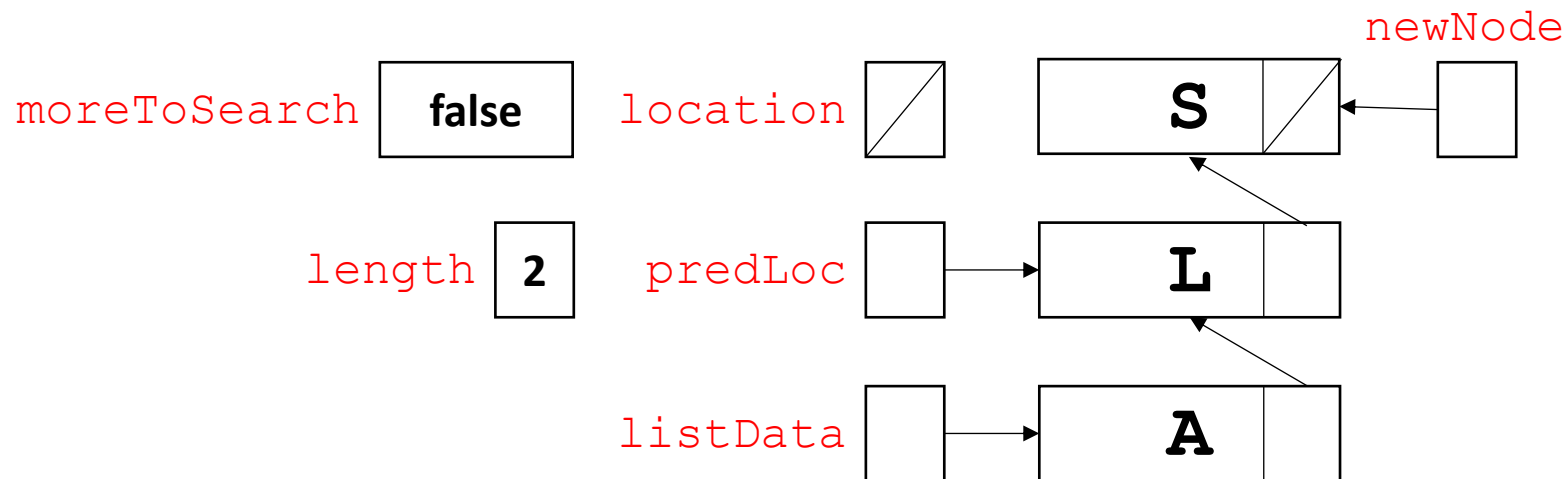
InsertItem('S')

```
newNode = new NodeType;  
newNode->info = item;  
  
if (predLoc == NULL)  
{  
    newNode->next = listData;  
    listData = newNode;  
}  
else  
{  
    newNode->next = location;  
    predLoc->next = newNode;  
}  
length++;  
}
```



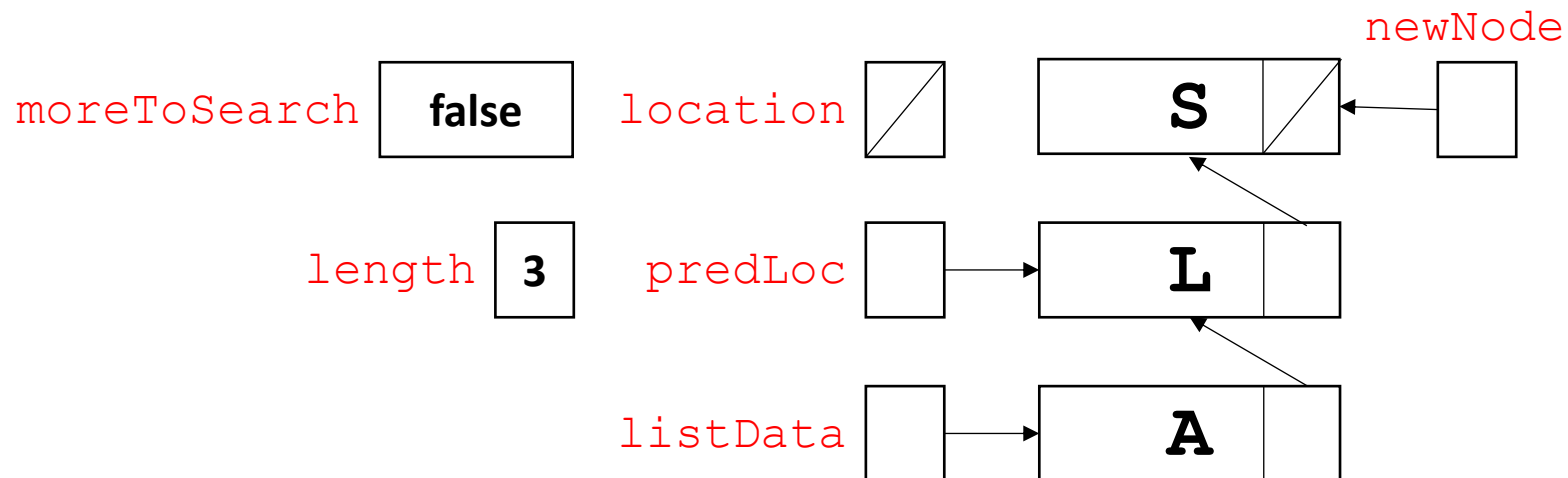
InsertItem('S')

```
newNode = new NodeType;  
newNode->info = item;  
  
if (predLoc == NULL)  
{  
    newNode->next = listData;  
    listData = newNode;  
}  
else  
{  
    newNode->next = location;  
    predLoc->next = newNode;  
}  
length++;  
}
```



InsertItem('S')

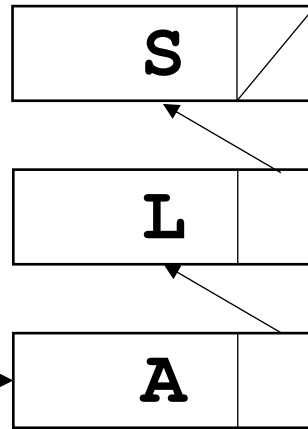
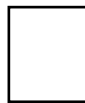
```
newNode = new NodeType;  
newNode->info = item;  
  
if (predLoc == NULL)  
{  
    newNode->next = listData;  
    listData = newNode;  
}  
else  
{  
    newNode->next = location;  
    predLoc->next = newNode;  
}  
length++;  
}
```



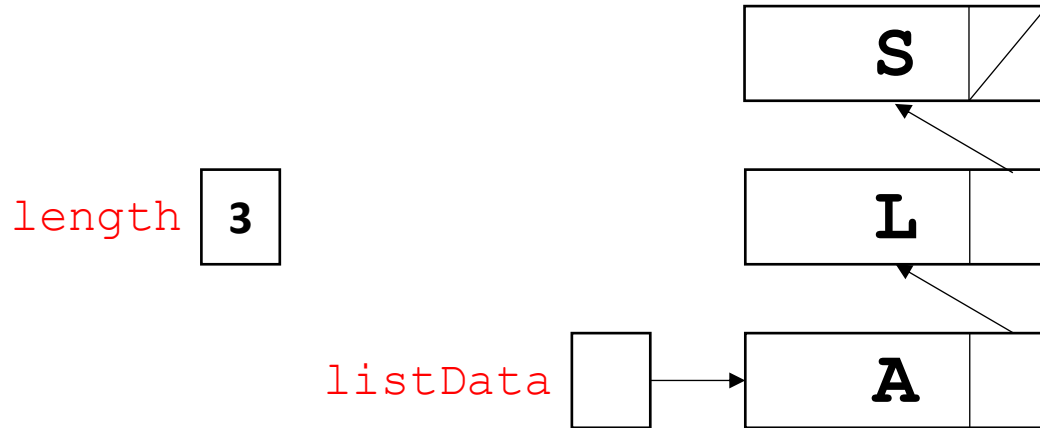
length

3

listData



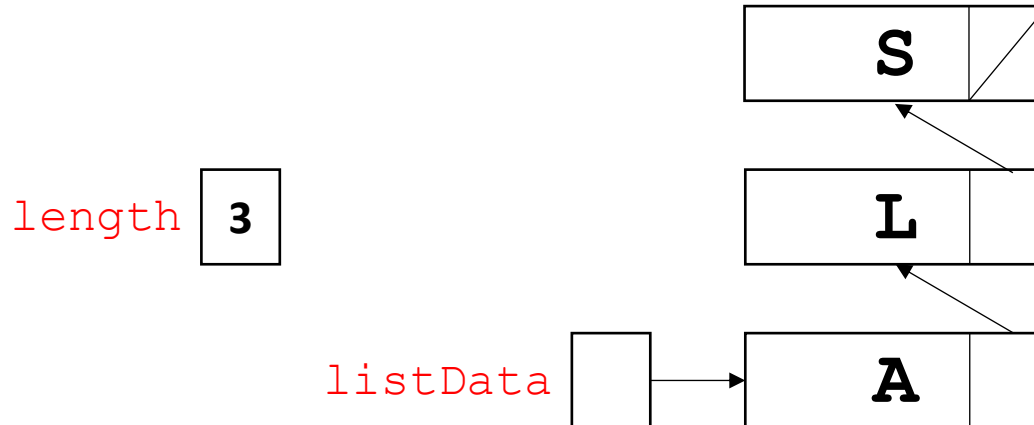
InsertItem('P')



InsertItem('P')

```
template <class ItemType>
void SortedType<ItemType>::InsertItem(ItemType item)
{
    NodeType* newNode;
    NodeType* predLoc;
    NodeType* location;
    bool moreToSearch;

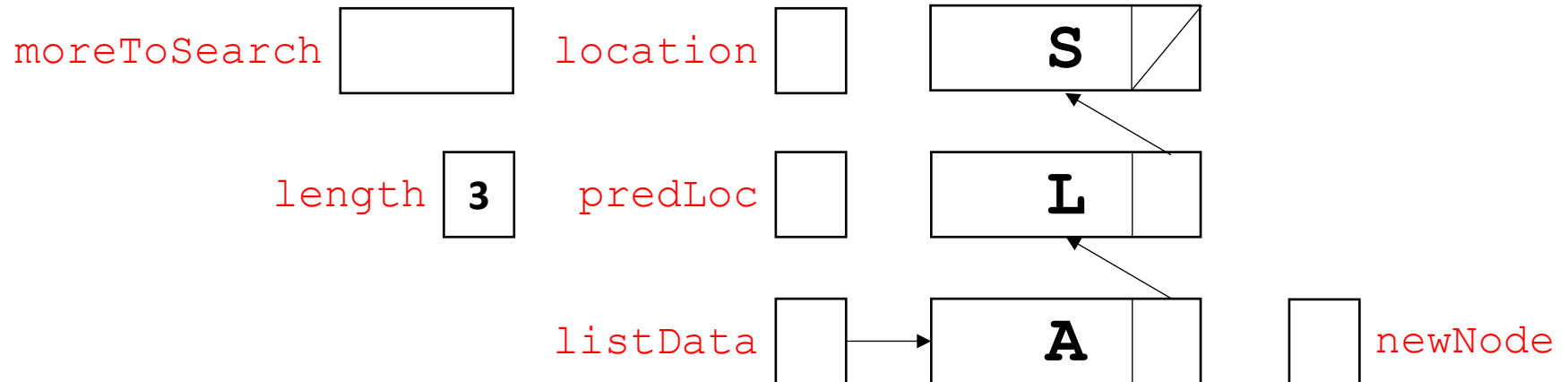
    location = listData;
    predLoc = NULL;
    moreToSearch = (location != NULL);
```



InsertItem('P')

```
template <class ItemType>
void SortedType<ItemType>::InsertItem(ItemType item)
{
    NodeType* newNode;
    NodeType* predLoc;
    NodeType* location;
    bool moreToSearch;

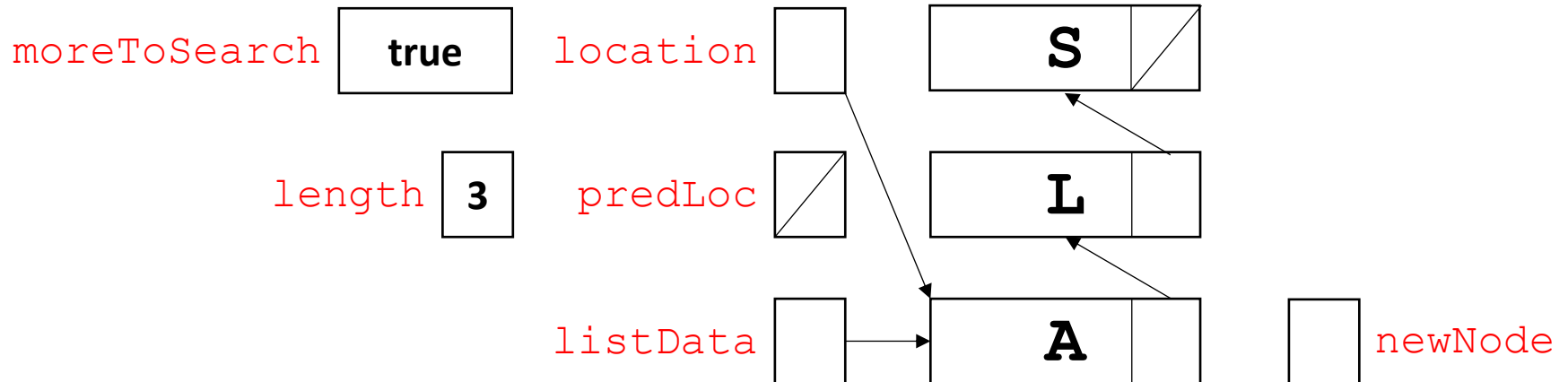
    location = listData;
    predLoc = NULL;
    moreToSearch = (location != NULL);
```



InsertItem('P')

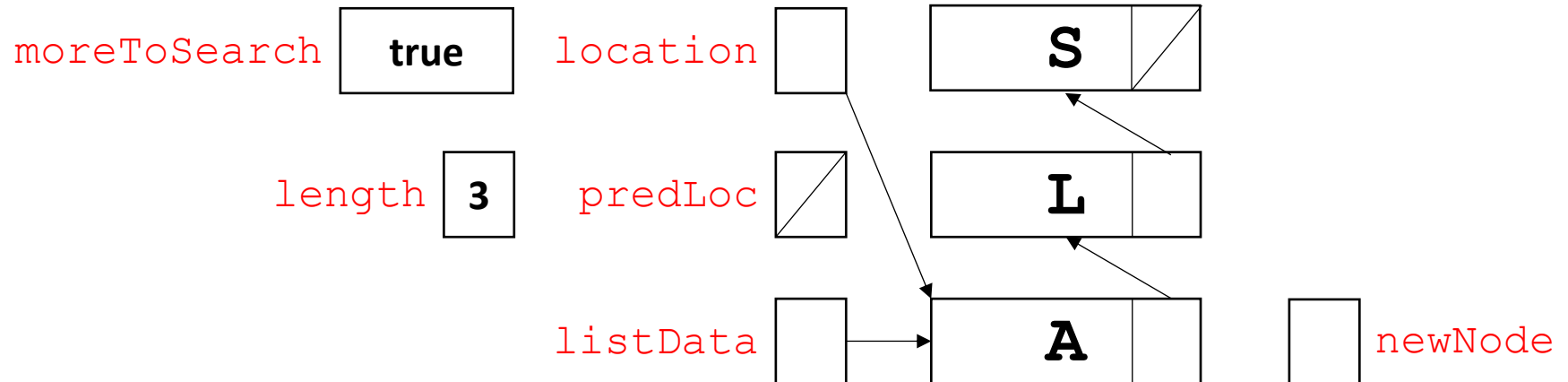
```
template <class ItemType>
void SortedType<ItemType>::InsertItem(ItemType item)
{
    NodeType* newNode;
    NodeType* predLoc;
    NodeType* location;
    bool moreToSearch;

    location = listData;
    predLoc = NULL;
    moreToSearch = (location != NULL);
```



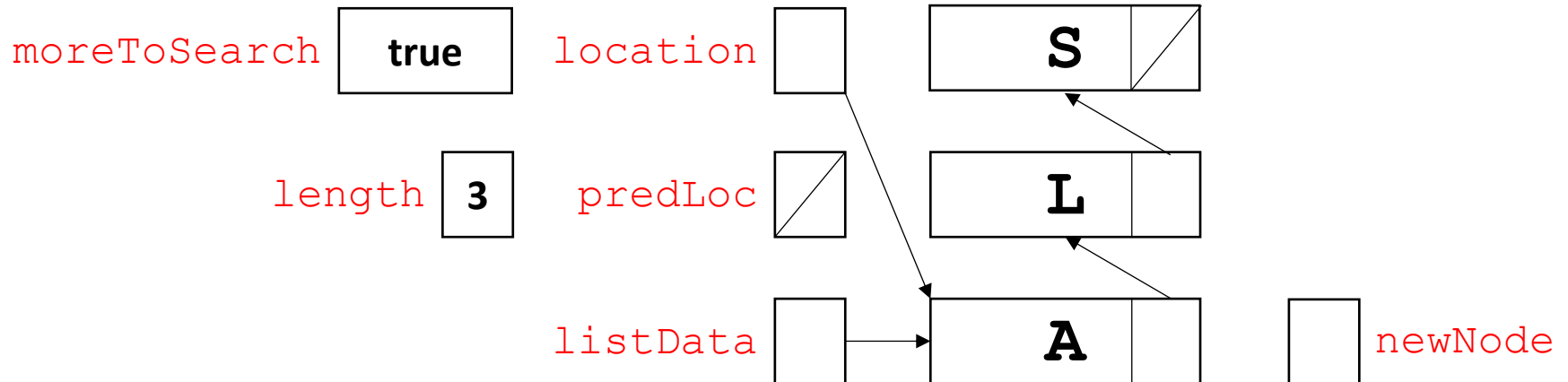
InsertItem('P')

```
while (moreToSearch)
{
    if (location->info < item)
    {
        predLoc = location;
        location = location->next;
        moreToSearch = (location != NULL);
    }
    else moreToSearch = false;
}
```



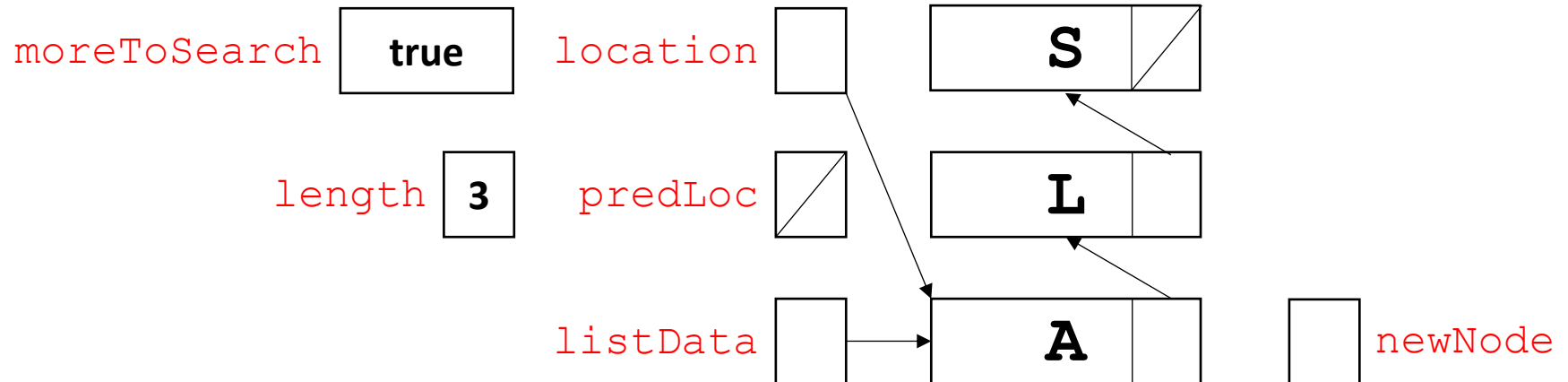
InsertItem('P')

```
while (moreToSearch)
{
    if (location->info < item)
    {
        predLoc = location;
        location = location->next;
        moreToSearch = (location != NULL);
    }
    else moreToSearch = false;
}
```



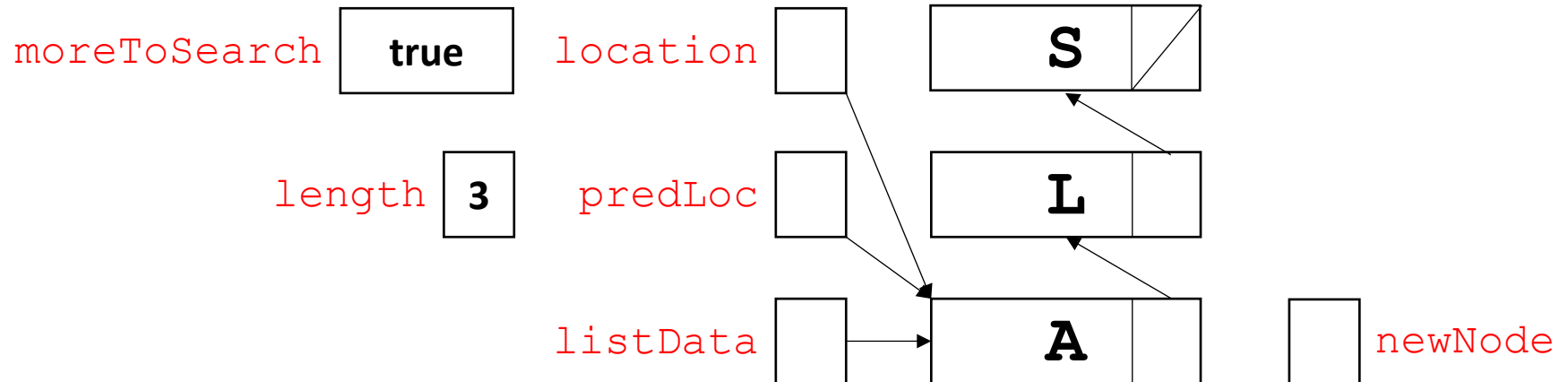
InsertItem('P')

```
while (moreToSearch)
{
    if (location->info < item)
    {
        predLoc = location;
        location = location->next;
        moreToSearch = (location != NULL);
    }
    else moreToSearch = false;
}
```



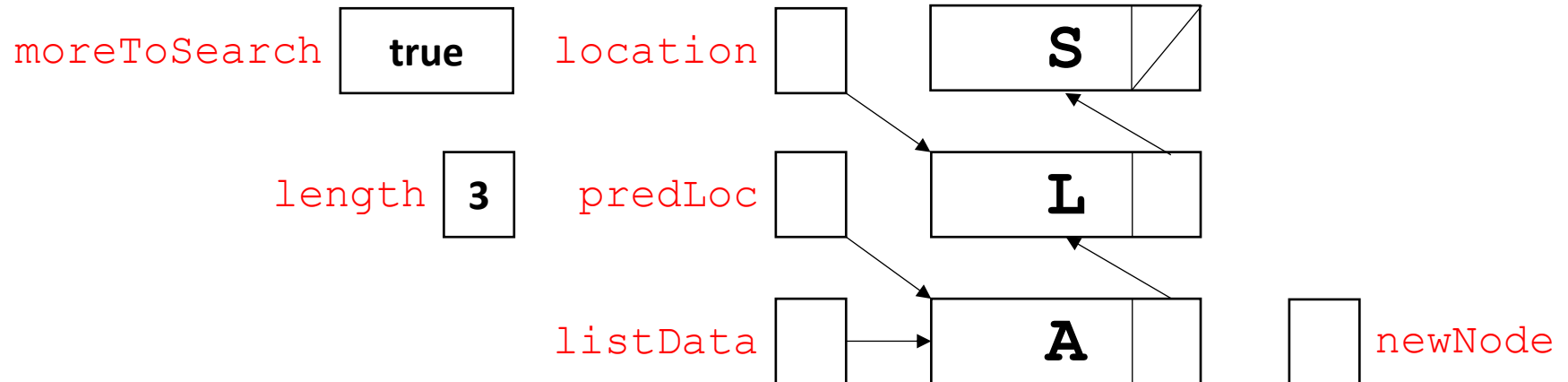
InsertItem('P')

```
while (moreToSearch)
{
    if (location->info < item)
    {
        predLoc = location;
        location = location->next;
        moreToSearch = (location != NULL);
    }
    else moreToSearch = false;
}
```



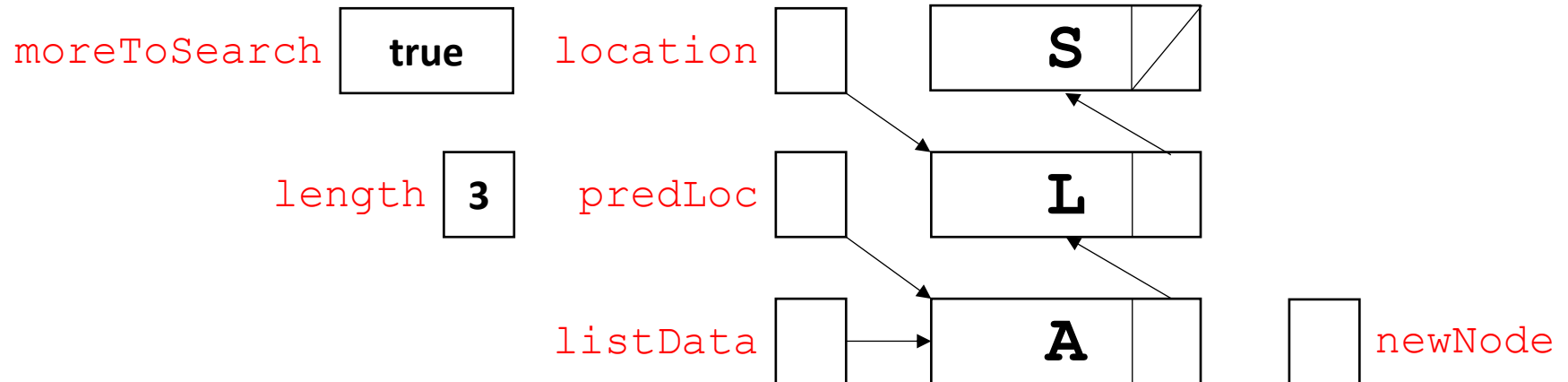
InsertItem('P')

```
while (moreToSearch)
{
    if (location->info < item)
    {
        predLoc = location;
        location = location->next;
        moreToSearch = (location != NULL);
    }
    else moreToSearch = false;
}
```



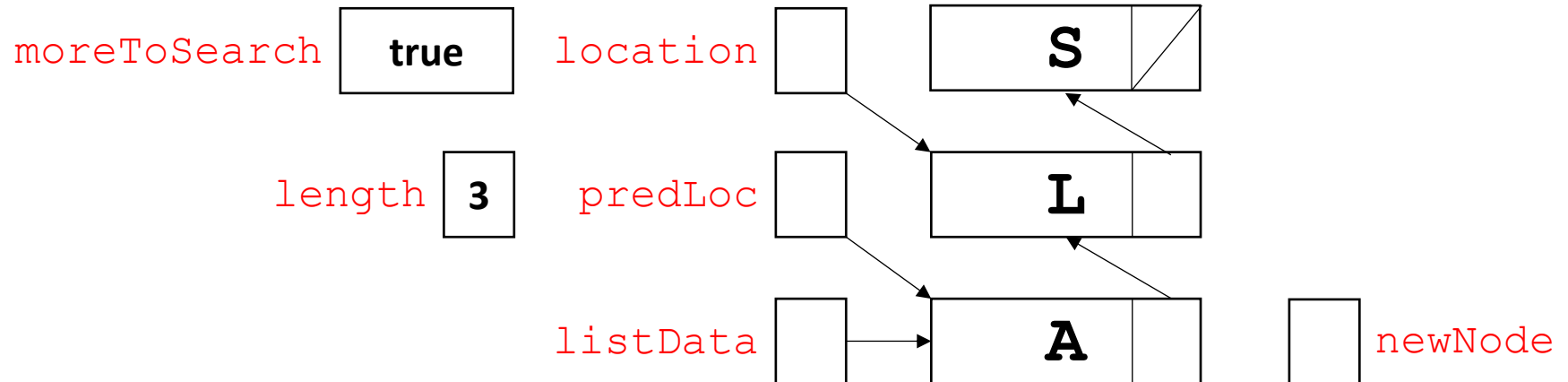
InsertItem('P')

```
while (moreToSearch)
{
    if (location->info < item)
    {
        predLoc = location;
        location = location->next;
        moreToSearch = (location != NULL);
    }
    else moreToSearch = false;
}
```



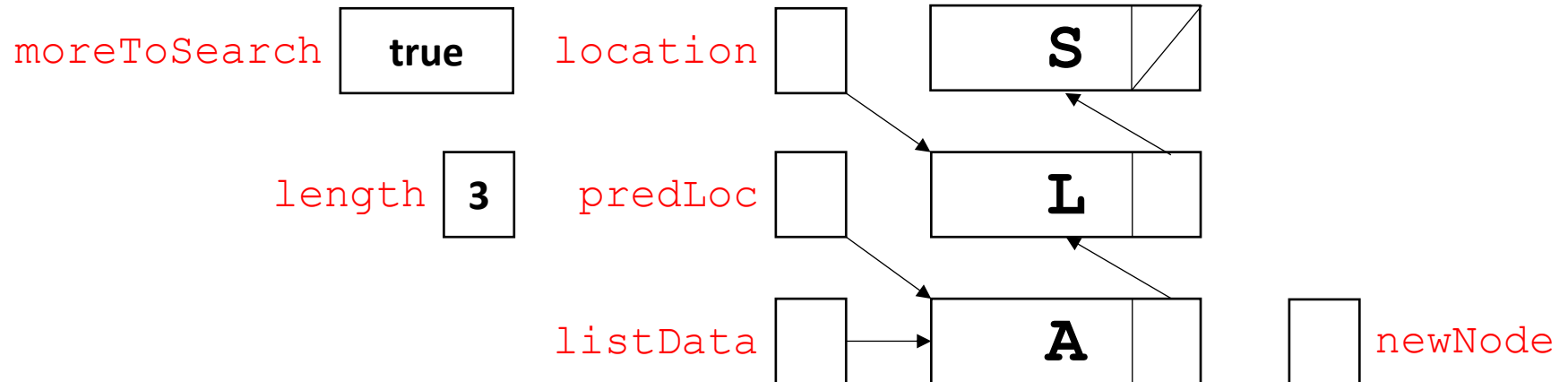
InsertItem('P')

```
while (moreToSearch)
{
    if (location->info < item)
    {
        predLoc = location;
        location = location->next;
        moreToSearch = (location != NULL);
    }
    else moreToSearch = false;
}
```



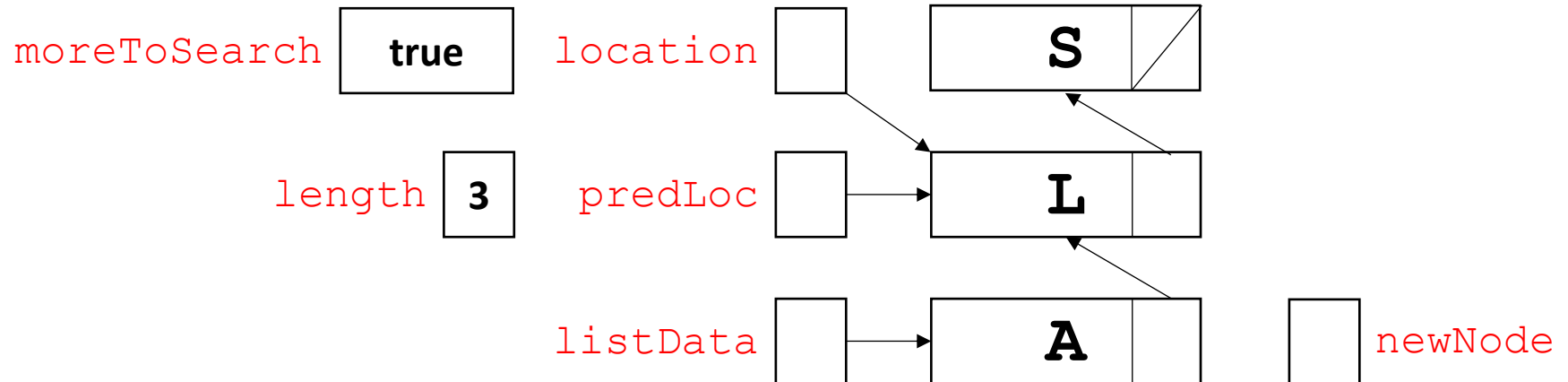
InsertItem('P')

```
while (moreToSearch)
{
    if (location->info < item)
    {
        predLoc = location;
        location = location->next;
        moreToSearch = (location != NULL);
    }
    else moreToSearch = false;
}
```



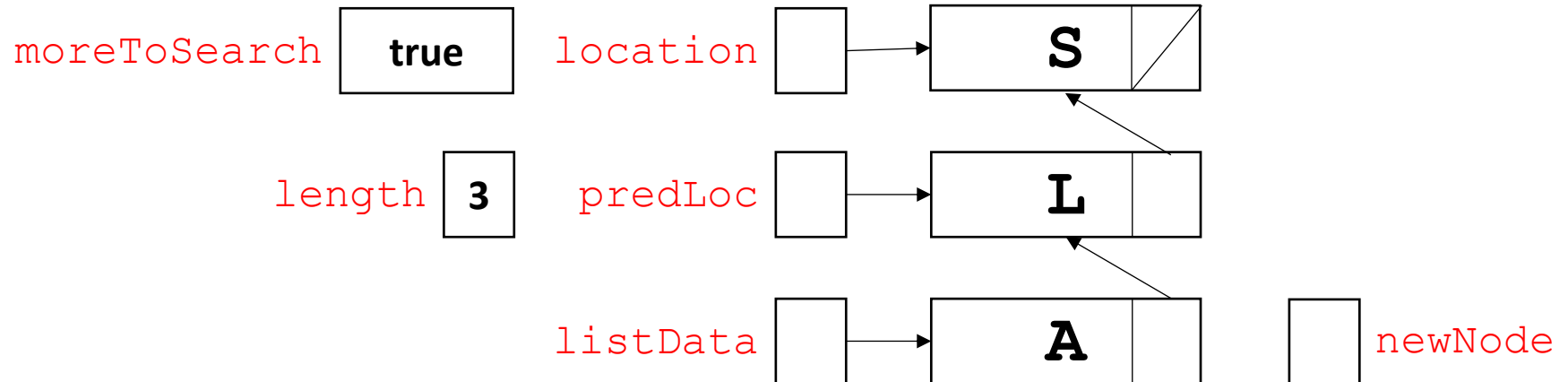
InsertItem('P')

```
while (moreToSearch)
{
    if (location->info < item)
    {
        predLoc = location;
        location = location->next;
        moreToSearch = (location != NULL);
    }
    else moreToSearch = false;
}
```



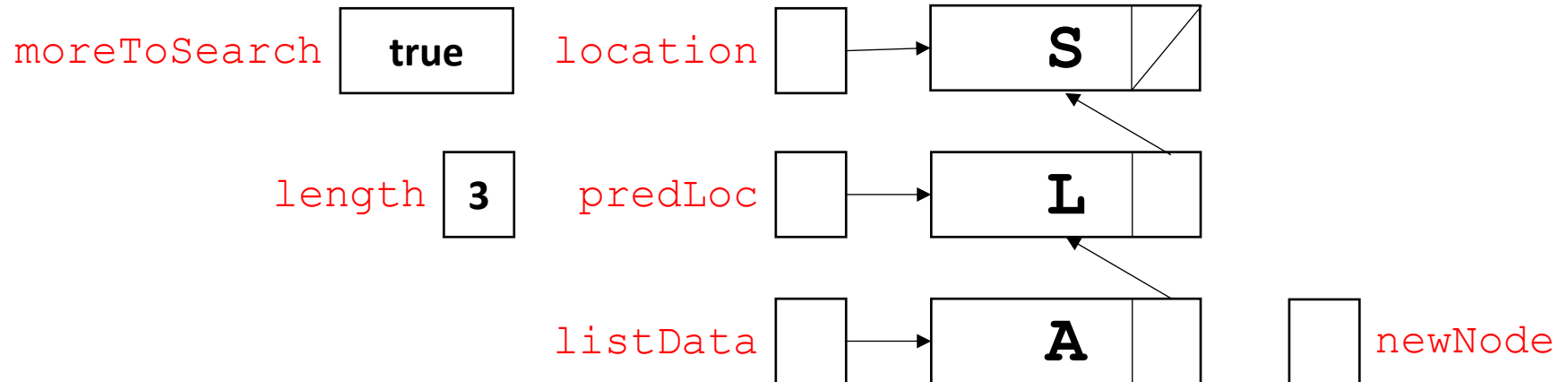
InsertItem('P')

```
while (moreToSearch)
{
    if (location->info < item)
    {
        predLoc = location;
        location = location->next;
        moreToSearch = (location != NULL);
    }
    else moreToSearch = false;
}
```



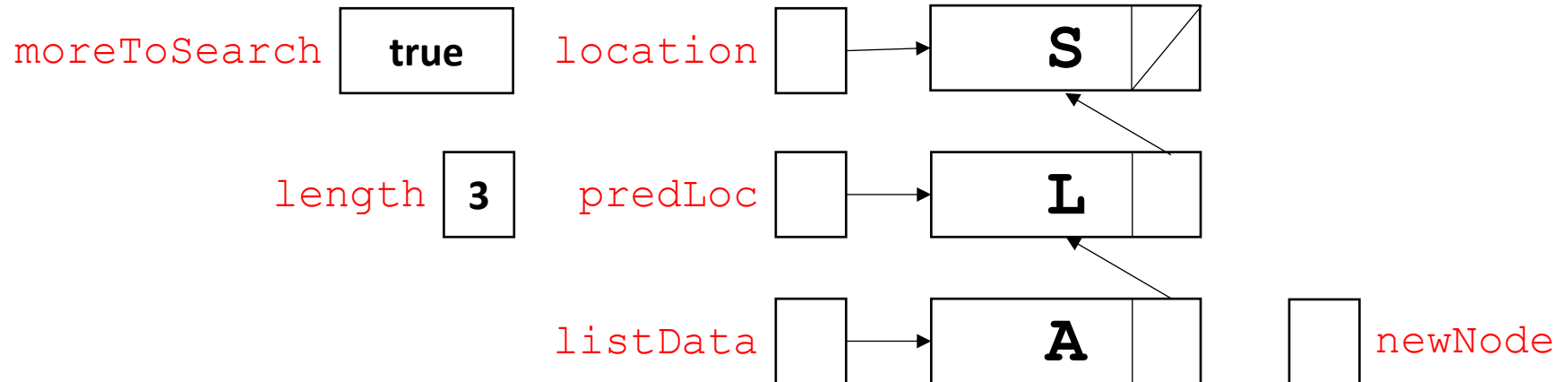
InsertItem('P')

```
while (moreToSearch)
{
    if (location->info < item)
    {
        predLoc = location;
        location = location->next;
        moreToSearch = (location != NULL);
    }
    else moreToSearch = false;
}
```



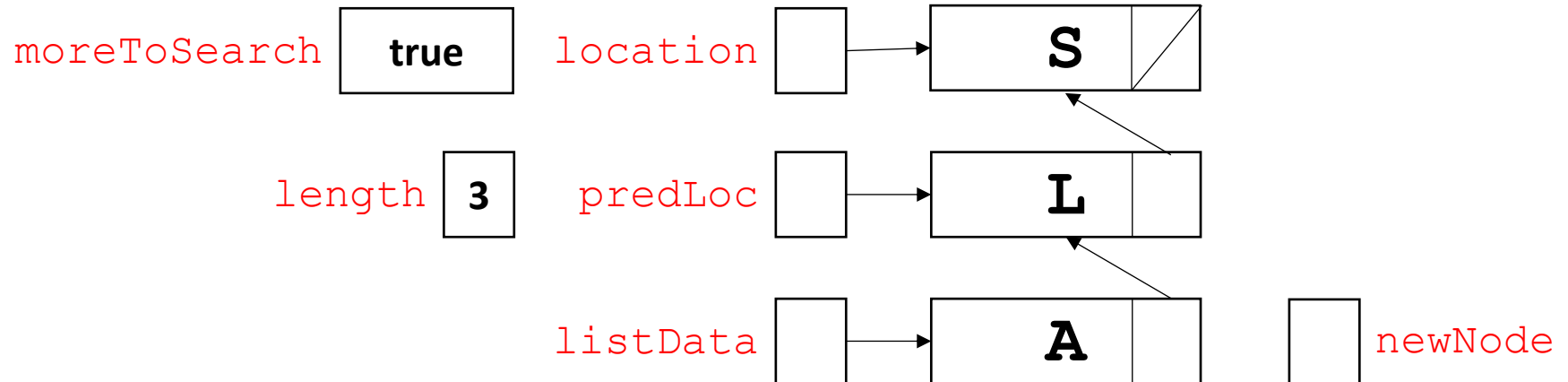
InsertItem('P')

```
while (moreToSearch)
{
    if (location->info < item)
    {
        predLoc = location;
        location = location->next;
        moreToSearch = (location != NULL);
    }
    else moreToSearch = false;
}
```



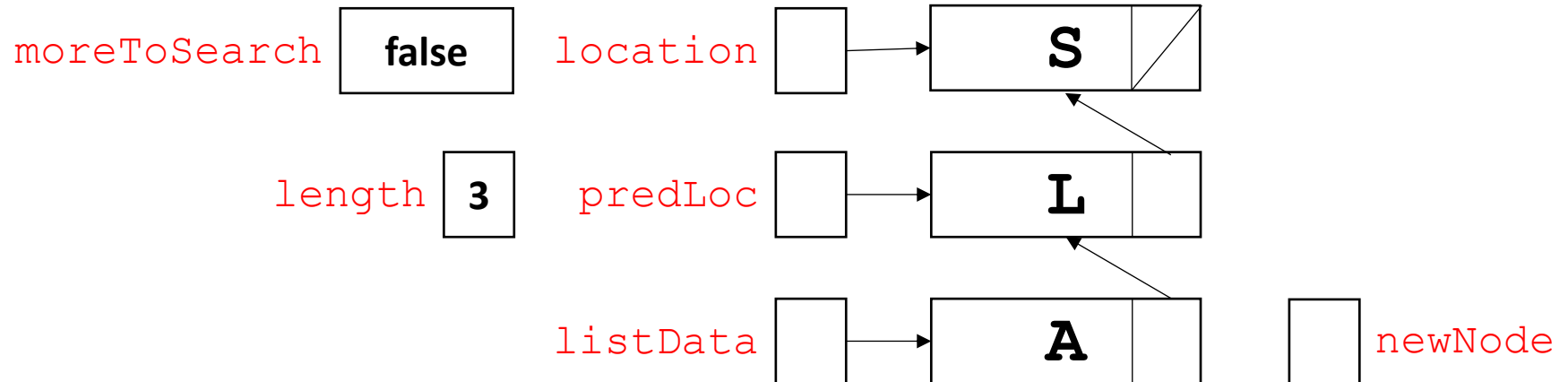
InsertItem('P')

```
while (moreToSearch)
{
    if (location->info < item)
    {
        predLoc = location;
        location = location->next;
        moreToSearch = (location != NULL);
    }
    else moreToSearch = false;
}
```



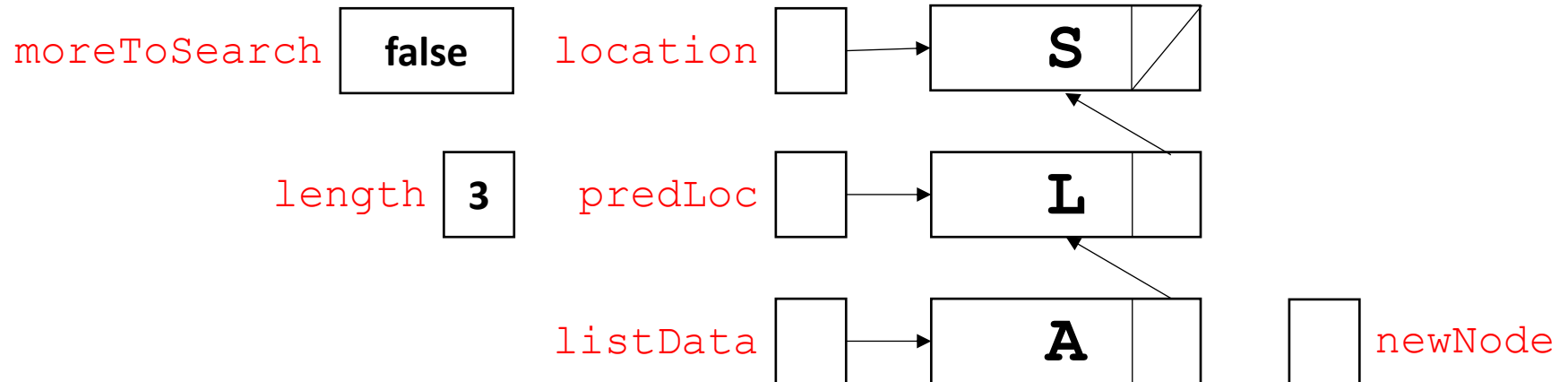
InsertItem('P')

```
while (moreToSearch)
{
    if (location->info < item)
    {
        predLoc = location;
        location = location->next;
        moreToSearch = (location != NULL);
    }
    else moreToSearch = false;
}
```



InsertItem('P')

```
while (moreToSearch)
{
    if (location->info < item)
    {
        predLoc = location;
        location = location->next;
        moreToSearch = (location != NULL);
    }
    else moreToSearch = false;
}
```



InsertItem('P')

```
newNode = new NodeType;
```

```
newNode->info = item;
```

```
if (predLoc == NULL)
```

```
{
```

```
    newNode->next = listData;
```

```
    listData = newNode;
```

```
}
```

```
else
```

```
{
```

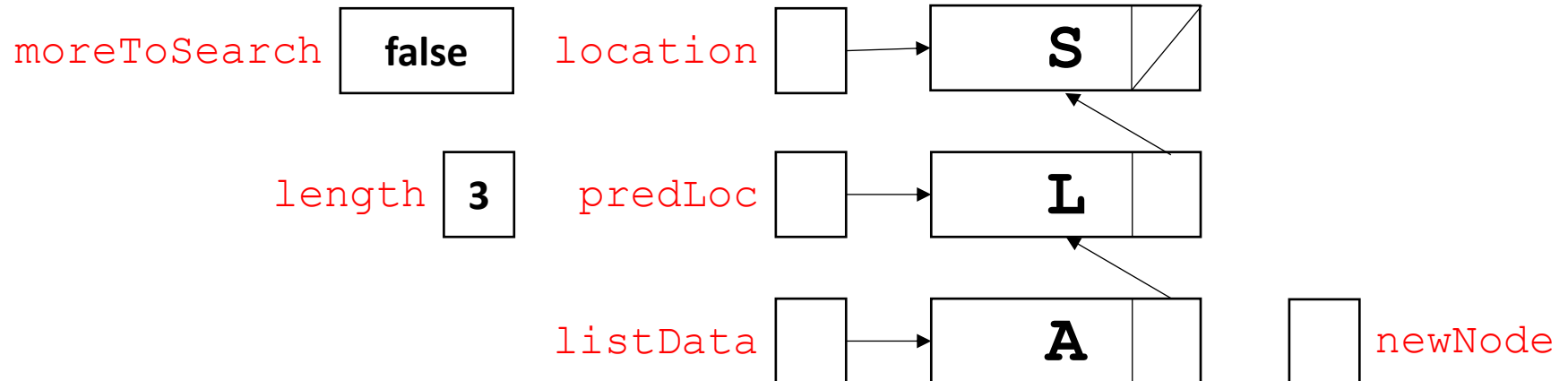
```
    newNode->next = location;
```

```
    predLoc->next = newNode;
```

```
}
```

```
length++;
```

```
}
```



InsertItem('P')

```
newNode = new NodeType;
```

```
newNode->info = item;
```

```
if (predLoc == NULL)
```

```
{
```

```
    newNode->next = listData;
```

```
    listData = newNode;
```

```
}
```

```
else
```

```
{
```

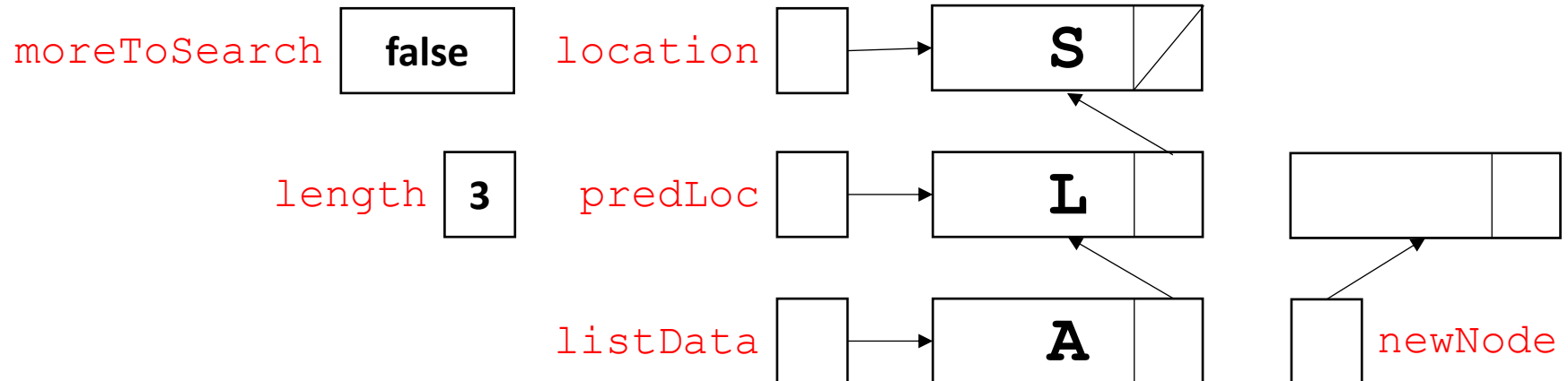
```
    newNode->next = location;
```

```
    predLoc->next = newNode;
```

```
}
```

```
length++;
```

```
}
```



InsertItem('P')

```
newNode = new NodeType;
```

```
newNode->info = item;
```

```
if (predLoc == NULL)
```

```
{
```

```
    newNode->next = listData;
```

```
    listData = newNode;
```

```
}
```

```
else
```

```
{
```

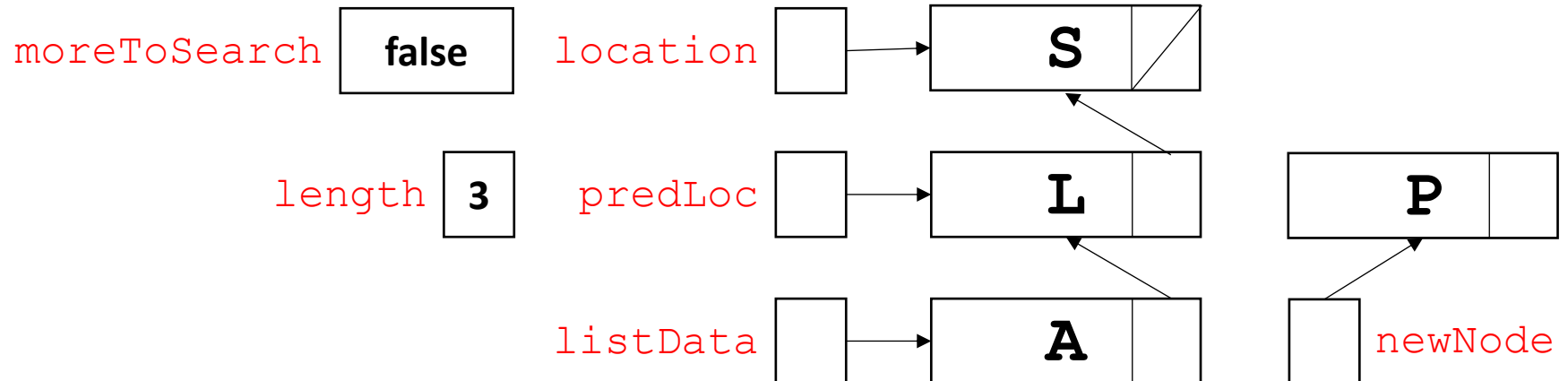
```
    newNode->next = location;
```

```
    predLoc->next = newNode;
```

```
}
```

```
length++;
```

```
}
```



InsertItem('P')

```
newNode = new NodeType;
```

```
newNode->info = item;
```

```
if (predLoc == NULL)
```

```
{
```

```
    newNode->next = listData;
```

```
    listData = newNode;
```

```
}
```

```
else
```

```
{
```

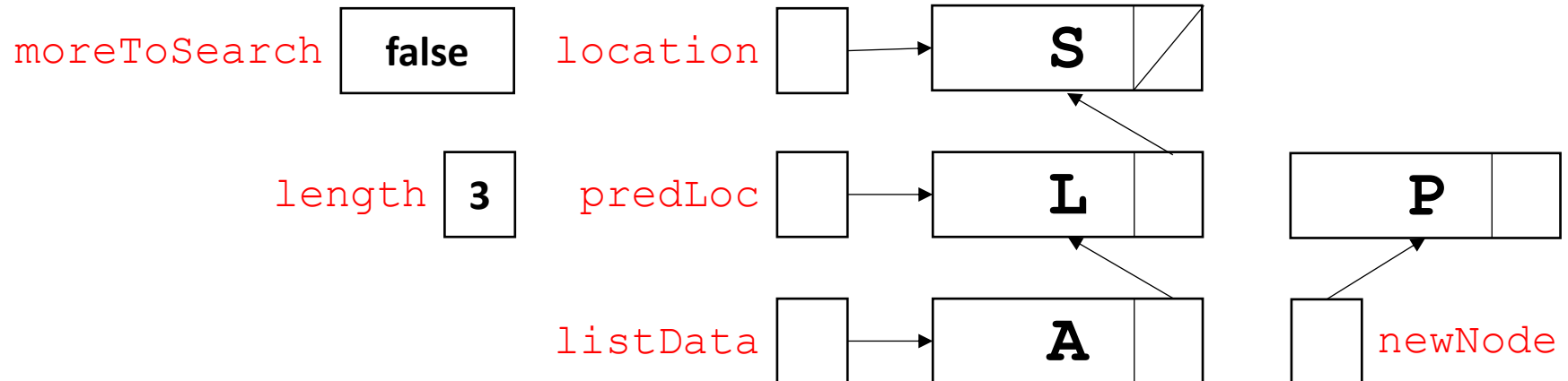
```
    newNode->next = location;
```

```
    predLoc->next = newNode;
```

```
}
```

```
length++;
```

```
}
```



InsertItem('P')

```
newNode = new NodeType;
```

```
newNode->info = item;
```

```
if (predLoc == NULL)
```

```
{
```

```
    newNode->next = listData;
```

```
    listData = newNode;
```

```
}
```

```
else
```

```
{
```

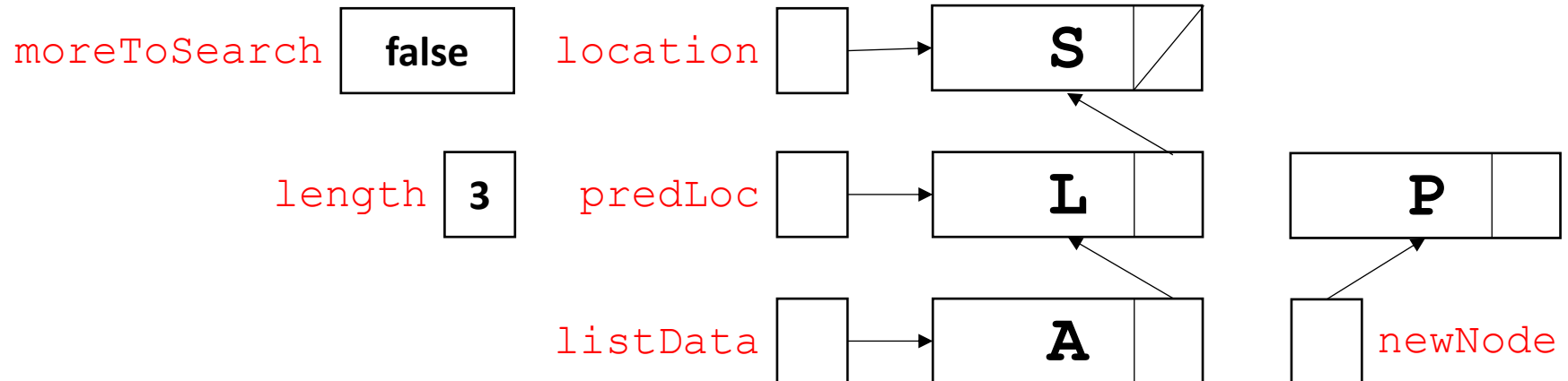
```
    newNode->next = location;
```

```
    predLoc->next = newNode;
```

```
}
```

```
length++;
```

```
}
```



InsertItem('P')

```
newNode = new NodeType;
```

```
newNode->info = item;
```

```
if (predLoc == NULL)
```

```
{
```

```
    newNode->next = listData;
```

```
    listData = newNode;
```

```
}
```

```
else
```

```
{
```

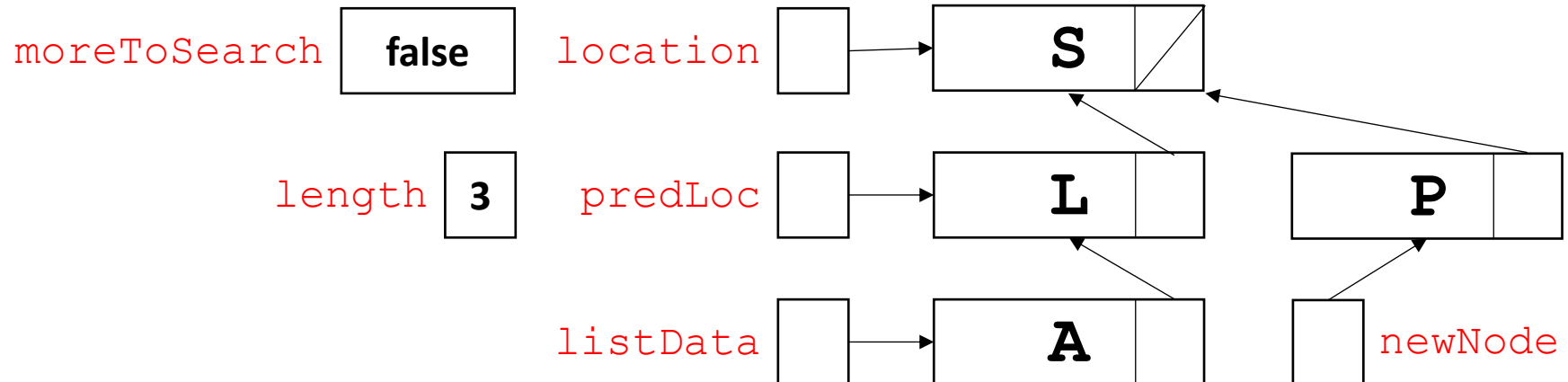
```
    newNode->next = location;
```

```
    predLoc->next = newNode;
```

```
}
```

```
length++;
```

```
}
```



InsertItem('P')

```
newNode = new NodeType;
```

```
newNode->info = item;
```

```
if (predLoc == NULL)
```

```
{
```

```
    newNode->next = listData;
```

```
    listData = newNode;
```

```
}
```

```
else
```

```
{
```

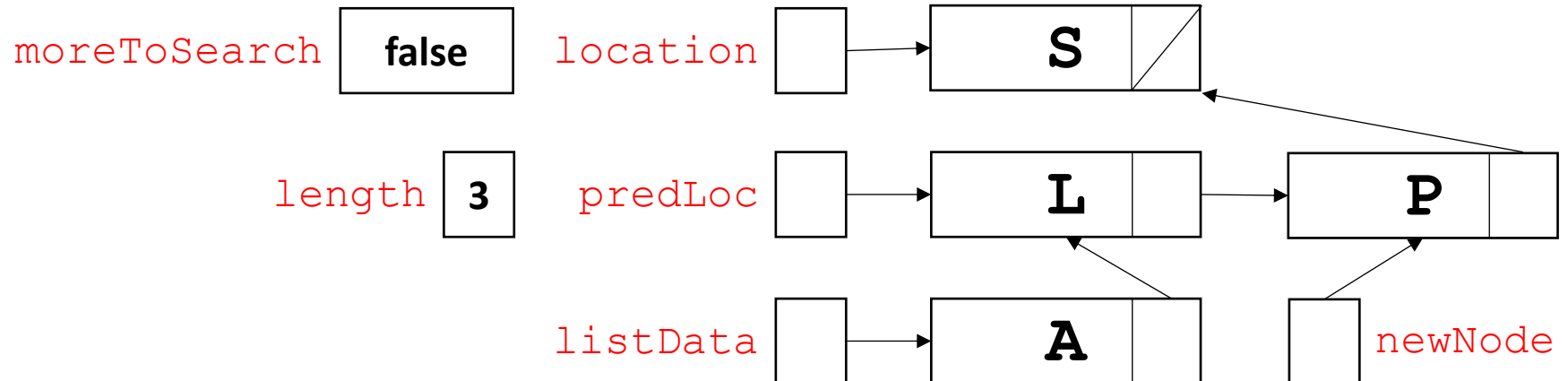
```
    newNode->next = location;
```

```
    predLoc->next = newNode;
```

```
}
```

```
length++;
```

```
}
```



InsertItem('P')

```
newNode = new NodeType;
```

```
newNode->info = item;
```

```
if (predLoc == NULL)
```

```
{
```

```
    newNode->next = listData;
```

```
    listData = newNode;
```

```
}
```

```
else
```

```
{
```

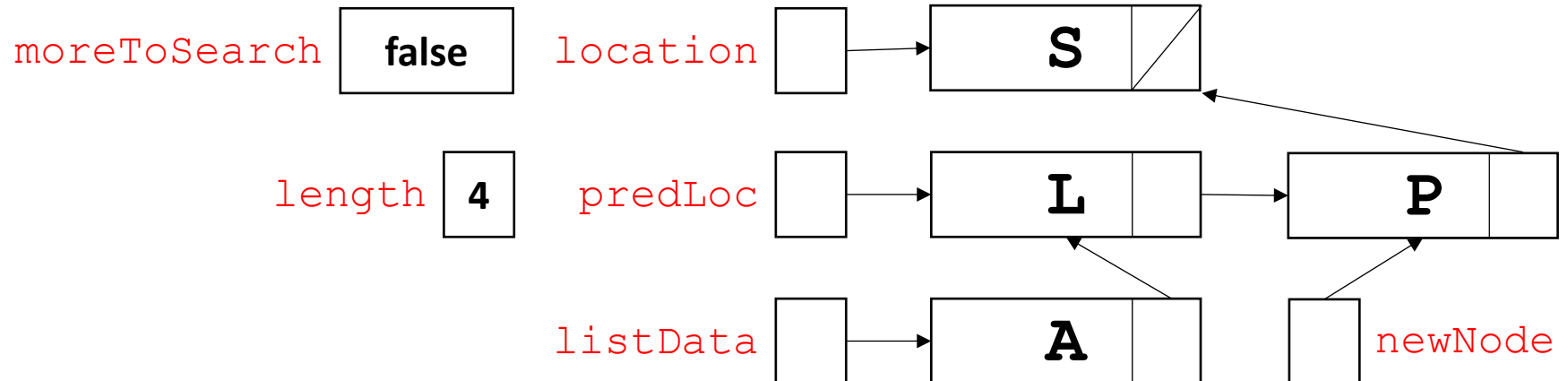
```
    newNode->next = location;
```

```
    predLoc->next = newNode;
```

```
}
```

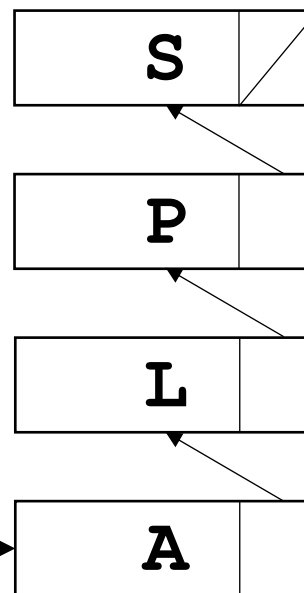
```
length++;
```

```
}
```



length 4

listData



sortedlinkedlist.cpp

```
template <class ItemType>
void SortedType<ItemType>::InsertItem(ItemType item)
{
    NodeType* newNode;
    NodeType* predLoc;
    NodeType* location;
    bool moreToSearch;

    location = listData;
    predLoc = NULL;
    moreToSearch = (location != NULL);
    while (moreToSearch)
    {
        if (location->info < item)
        {
            predLoc = location;
            location = location->next;
            moreToSearch = (location != NULL);
        }
        else moreToSearch = false;
    }
```

O(N)

```
        newNode = new NodeType;
        newNode->info = item;

        if (predLoc == NULL)
        {
            newNode->next = listData;
            listData = newNode;
        }
        else
        {
            newNode->next = location;
            predLoc->next = newNode;
        }
        length++;
    }
```

sortedlinkedlist.cpp

```
template <class ItemType>
void SortedType<ItemType>::DeleteItem(ItemType item)
{
    NodeType* location = listData;
    NodeType* tempLocation;
    if (item == listData->info)
    {
        tempLocation = location;
        listData = listData->next;
    }
    else
    {
        while (!(item==(location->next)->info))
            location = location->next;
        tempLocation = location->next;
        location->next = (location->next)->next;
    }
    delete tempLocation;
    length--;
}
```


sortedlinkedlist.cpp

```
template <class ItemType>
void SortedType<ItemType>::DeleteItem(ItemType item)
{
    NodeType* location = listData;
    NodeType* tempLocation;
    if (item == listData->info)
    {
        tempLocation = location;
        listData = listData->next;
    }
    else
    {
        while (!(item==(location->next)->info))
        {
            location = location->next;
            tempLocation = location->next;
            location->next = (location->next)->next;
        }
        delete tempLocation;
        length--;
    }
}
```

O(N)

sortedlinkedlist.cpp

```
template <class ItemType>
void SortedType<ItemType>::RetrieveItem(ItemType& item, bool& found)
{
    NodeType* location = listData;
    bool moreToSearch = (location != NULL);
    found = false;
    while (moreToSearch && !found)
    {
        if (item == location->info)
            found = true;
        else if (item < location->info)
        {
            location = location->next;
            moreToSearch = (location != NULL);
        }
        else
            moreToSearch = false;
    }
}
```

sortedlinkedlist.cpp

```
template <class ItemType>
void SortedType<ItemType>::RetrieveItem(ItemType& item, bool& found)
{
    NodeType* location = listData;
    bool moreToSearch = (location != NULL);
    found = false;
    while (moreToSearch && !found)
    {
        if (item == location->info)
            found = true;
        else if (item < location->info)
        {
            location = location->next;
            moreToSearch = (location != NULL);
        }
        else
            moreToSearch = false;
    }
}
```

O(N)